



INTELIGÊNCIA ARTIFICIAL SIMBÓLICA



RAFAEL ALBUQUERQUE PINTO



ACESSE AQUI ESTE
MATERIAL DIGITAL!

EXPEDIENTE

Coordenador(a) de Conteúdo

Greisse Moser Badalotti

Projeto Gráfico e Capa

Arthur Cantareli Silva

Editoração

Alan Diego Hordina e

Renata Sguissardi Coli

Design Educacional

Maíra Vanessa da Rocha

Revisão Textual

Cindy Luca

Ilustração

Andre Luis Azevedo da Silva,

Eduardo Aparecido Alves,

Geison Ferreira da Silva e

Wellington Vainer

Fotos

Shutterstock e Envato

FICHA CATALOGRÁFICA

N964 Núcleo de Educação a Distância. **PINTO**, Rafael Albuquerque.

Inteligência Artificial Simbólica / Rafael Albuquerque Pinto. -
Florianópolis, SC: Arquê, 2025.

264 p.

ISBN papel 978-65-279-0830-2

ISBN digital 978-65-279-0829-6

1. Inteligência 2. Artificial 3. Simbólica 4. EaD. I. Título.

CDD - 006.3

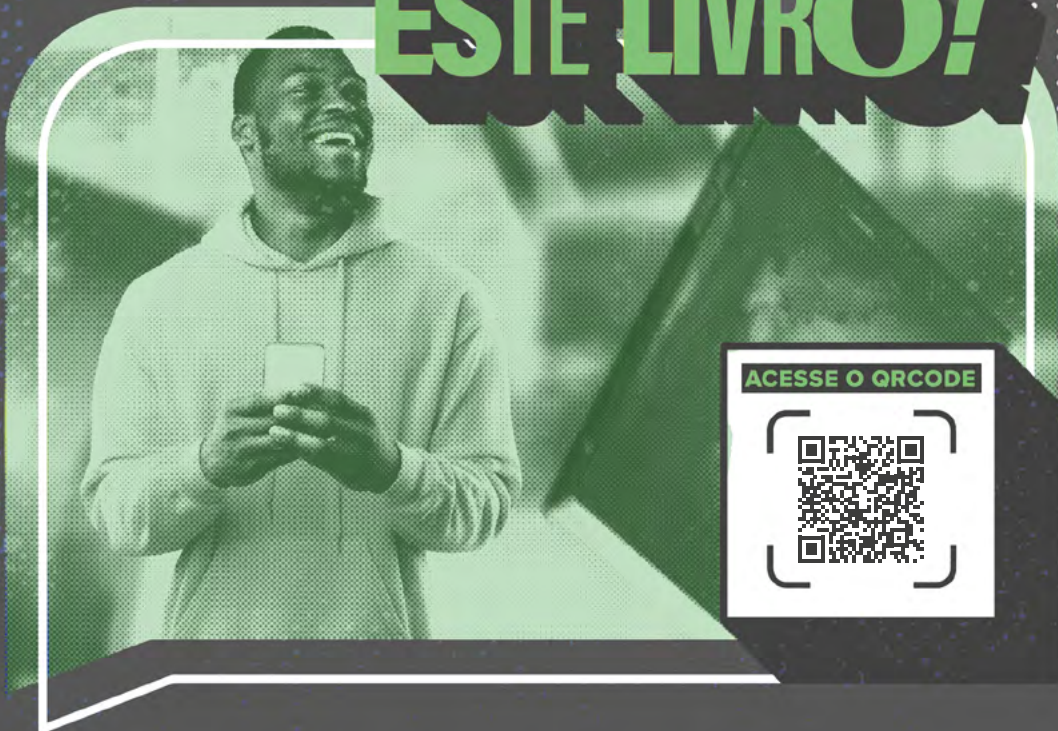
Bibliotecária: Leila Regina do Nascimento - CRB- 9/1722.

Ficha catalográfica elaborada de acordo com os dados fornecidos pelo(a) autor(a).

Impresso por:

Ano de Impressão:

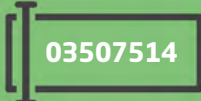
AVALIE ESTE LIVRO!



CRIAR MOMENTOS DE APRENDIZAGENS
INESQUECÍVEIS É O NOSSO OBJETIVO E POR ISSO,
GOSTARIAMOS DE SABER COMO FOI SUA EXPERIÊNCIA.

Conta para nós! leva menos de 2 minutos. Vamos lá?!

DIGITE O CÓDIGO

 03507514

 Aa

**RESPONDA A
PESQUISA**



 ...



RECURSOS DE IMERSÃO



PENSANDO JUNTOS

Este item corresponde a uma proposta de reflexão que pode ser apresentada por meio de uma frase, um trecho breve ou uma pergunta.



APROFUNDANDO

Utilizado para temas, assuntos ou conceitos avançados, levando ao aprofundamento do que está sendo trabalhado naquele momento do texto.



EU INDICO

Utilizado para agregar um conteúdo externo.



ZOOM NO CONHECIMENTO

Utilizado para desmistificar pontos que possam gerar confusão sobre o tema. Após o texto trazer a explicação, essa interlocução pode trazer pontos adicionais que contribuam para que o estudante não fique com dúvidas sobre o tema.

PRODUTOS AUDIOVISUAIS

Os elementos abaixo possuem recursos audiovisuais. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.



PLAY NO CONHECIMENTO

Professores especialistas e convidados, ampliando as discussões sobre os temas por meio de fantásticos podcasts.



EM FOCO

Utilizado para aprofundar o conhecimento em conteúdos relevantes utilizando uma linguagem audiovisual.



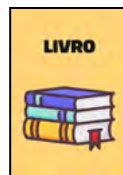
INDICAÇÃO DE FILME

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de filmes que se conectam com o tema do conteúdo.



INDICAÇÃO DE LIVRO

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de livros que agregarão muito na sua vida profissional.



SUMÁRIO

7

UNIDADE 1

HISTÓRICO E FUNDAMENTOS DA INTELIGÊNCIA ARTIFICIAL SIMBÓLICA 8

TEORIA DE PROBLEMAS E IA: MODELAGEM E SOLUÇÕES PRÁTICAS34

PARADIGMA SIMBÓLICO EM IA70

101

UNIDADE 2

MÉTODOS DE BUSCA EM IA102

REPRESENTAÇÃO DE CONHECIMENTO E INFERÊNCIA132

AGENTES INTELIGENTES156

185

UNIDADE 3

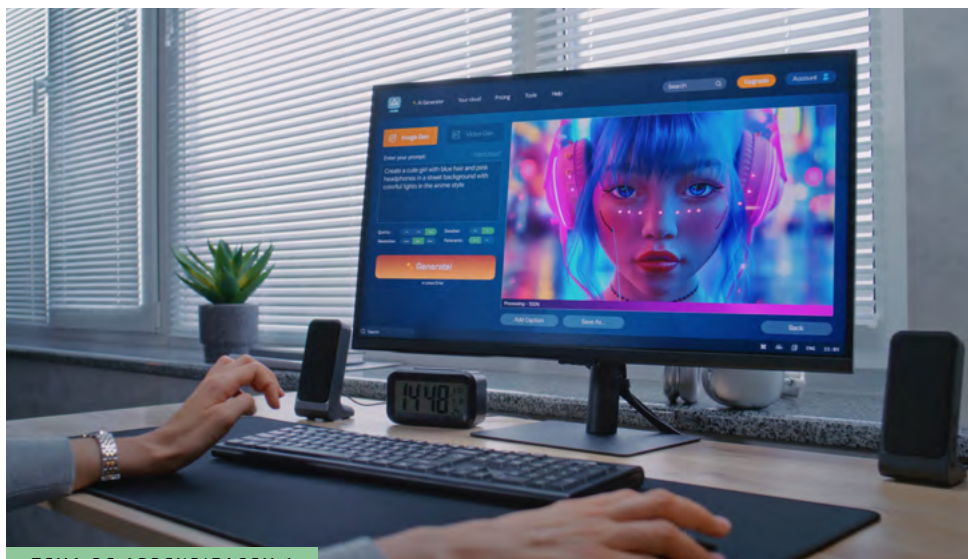
REDES SEMÂNTICAS EM IA SIMBÓLICA186

FRAMES E SISTEMAS BASEADOS EM CONHECIMENTO210

APLICAÇÕES PRÁTICAS DE IA SIMBÓLICA238

unidade





TEMA DE APRENDIZAGEM 1

HISTÓRICO E FUNDAMENTOS DA INTELIGÊNCIA ARTIFICIAL SIMBÓLICA

MINHAS METAS

- Explorar a evolução da IA simbólica desde os anos 1950 até hoje.
- Compreender os conceitos fundamentais da IA simbólica.
- Discutir a importância da IA simbólica na sociedade moderna.
- Examinar as bases teóricas, incluindo princípios matemáticos e lógicos.
- Configurar o ambiente de desenvolvimento Python para IA.
- Aplicar métodos matemáticos na IA simbólica.
- Implementar conceitos de IA simbólica em Python com exercícios práticos.

INICIE SUA JORNADA

Desde a invenção do computador, cientistas e engenheiros têm sonhado em criar máquinas que possam pensar e agir como seres humanos. Esse sonho deu origem ao campo da Inteligência Artificial (IA), especificamente à IA simbólica.

No entanto, quais são os desafios e as oportunidades que essa tecnologia oferece?

A IA simbólica se propõe a simular o raciocínio humano mediante a manipulação de símbolos e regras lógicas. Contudo, implementar essa capacidade em sistemas computacionais não é uma tarefa simples. Quais são as principais barreiras técnicas e éticas que devem ser superadas para que a IA simbólica atinja o potencial máximo?

A IA simbólica, com raízes profundas na lógica e na matemática, representa um dos pilares fundamentais para o **desenvolvimento da tecnologia moderna**. Ela não apenas ajuda a resolver problemas complexos de forma eficiente, mas também desempenha um papel crucial em diversas áreas, como medicina, transporte e comércio. Por exemplo, em diagnósticos médicos, a IA simbólica pode auxiliar na identificação de doenças, ao analisar sintomas e históricos de pacientes, oferecendo um suporte valioso aos profissionais de saúde. No setor de transporte, pode otimizar rotas de entrega, economizando tempo e recursos. Logo, compreender a importância da IA simbólica é essencial para quem deseja atuar na vanguarda tecnológica.

Suponha que você está trabalhando em uma empresa de logística que precisa otimizar as rotas de entrega para reduzir custos e aumentar a eficiência. Como você aplicaria a IA simbólica para resolver esse problema? Desenvolva um pequeno estudo de caso considerando os seguintes pontos: identificação do problema, coleta de dados relevantes, aplicação de algoritmos de busca para encontrar a rota mais eficiente e avaliação dos resultados. Esse exercício prático permitirá visualizar como a teoria da IA simbólica pode ser aplicada em situações reais, o que te preparará para enfrentar desafios semelhantes em sua carreira profissional.

Como a evolução da IA simbólica pode impactar a sua área de atuação? Quais são as possíveis aplicações e implicações éticas da IA no seu campo de trabalho? Reflita sobre como os conhecimentos adquiridos podem ser utilizados para melhorar processos e tomar decisões mais informadas. Considere também os desafios éticos, como a privacidade dos dados e a transparência nos processos decisórios. **É essencial que futuros profissionais não só dominem a tecnologia, mas também entendam as responsabilidades associadas ao respectivo uso.** Como você pode garantir que as soluções desenvolvidas sejam justas, éticas e benéficas para a sociedade?

**PLAY NO CONHECIMENTO**

Neste episódio do nosso podcast, exploramos as fascinantes origens da IA simbólica e destacamos os principais marcos históricos e figuras influentes que moldaram esse campo. Discutimos também as aplicações práticas e os impactos da IA simbólica em diversas áreas da sociedade. Venha descobrir como essa tecnologia revolucionária está transformando o mundo ao nosso redor! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para garantir que você esteja preparado(a) para compreender e aplicar os conceitos de Inteligência Artificial Simbólica, vamos relembrar alguns conteúdos essenciais. Primeiro, é importante entender a lógica proposicional, que é a base para a IA simbólica. A lógica proposicional inclui proposições, conectivos lógicos, tabelas-verdade e equivalências lógicas.

Além disso, expandir esse conhecimento para a lógica de predicados, que lida com quantificadores e relações mais complexas, é fundamental. Com uma base sólida desses conceitos, você estará mais bem preparado(a) para representar e manipular conhecimentos na IA simbólica. Finalmente, conhecer a história dos computadores ajuda a contextualizar a evolução da tecnologia que possibilitou os avanços em IA. Saiba mais no link a seguir: <https://www.youtube.com/watch?v=NJFhQU7jhpY>

DESENVOLVA SEU POTENCIAL

EVOLUÇÃO DA IA

Vamos embarcar em uma jornada fascinante pelo mundo da Inteligência Artificial (IA). Este tema de aprendizagem oferece uma visão abrangente e envolvente sobre a evolução, os conceitos-chave e a importância da IA no mundo moderno. Preparado(a)? Então, vamos lá!

A Inteligência Artificial é um campo da ciência da computação que tem como objetivo criar máquinas que possam realizar tarefas que, quando feitas por hu-

manos, requerem inteligência. Essas tarefas incluem desde resolver problemas matemáticos complexos até reconhecer rostos em uma foto ou compreender e responder a comandos de voz. A IA tem evoluído de maneira impressionante desde as origens, na década de 1950, passando por altos e baixos, mas sempre avançando em direção a um futuro em que as máquinas são capazes de desempenhar um papel cada vez mais significativo em nossas vidas.

Vamos começar entendendo como tudo começou. Logo, exploraremos os primeiros passos dados por pioneiros visionários que lançaram as bases para o que, hoje, conhecemos como IA. Em seguida, entenderemos como a IA evoluiu ao longo das décadas, superando desafios técnicos e conceituais, até se tornar uma parte indispensável da nossa sociedade moderna. Finalmente, vamos nos aprofundar nos conceitos-chave da IA simbólica, entendendo a respectiva importância no contexto atual, e explorar as bases teóricas que sustentam essa fascinante área do conhecimento.

Além disso, vamos fazer uma introdução à programação em Python, uma das linguagens mais populares e poderosas para desenvolver aplicações de IA, e aprender a configurar o ambiente de desenvolvimento para criarmos os nossos próprios algoritmos de IA.

Décadas de 1950 e 1960: origens e primeiros passos

No início da trajetória da Inteligência Artificial (IA) simbólica, as décadas de 1950 e 1960 se destacam como o marco inicial para o desenvolvimento das bases teóricas e práticas desse paradigma. Fundamentada na premissa de que o raciocínio humano pode ser modelado mediante símbolos e regras formais, essa abordagem propõe que máquinas sejam capazes de processar e interpretar informações de forma lógica e estruturada, aproximando-se do pensamento humano.

Imagine-se na década de 1950. A tecnologia estava começando a dar os primeiros grandes saltos, e a ideia de máquinas inteligentes começava a tomar forma. Em 1950, **Alan Turing**, um matemático britânico brilhante, publicou um artigo revolucionário chamado *Computing Machinery and Intelligence*. Nesse artigo, Turing propôs o famoso Teste de Turing, um método para determinar se uma máquina pode exibir comportamento inteligente semelhante ao humano. Esse trabalho seminal lançou as bases para a IA moderna e ainda é uma referência fundamental para os estudiosos da área (Russell; Norvig, 2021).



INDICAÇÃO DE FILME

O Jogo da Imitação

O *Jogo da Imitação* é um filme profundamente ligado aos fundamentos da Inteligência Artificial (IA) e da ciência da computação, assim como é discutido ao longo deste tema de aprendizagem. Alan Turing, cuja vida e trabalho são retratados no filme, é amplamente reconhecido como um dos pais da ciência da computação e da IA. O trabalho teórico de Turing, especialmente o artigo *Computing Machinery and Intelligence*, e a proposição do Teste de Turing, lançaram as bases para o desenvolvimento da IA moderna.



Agora, visualize a cena em 1956, durante a Conferência de Dartmouth. Um grupo de cientistas brilhantes, incluindo John McCarthy, Marvin Minsky, Nathaniel Rochester e Claude Shannon, reuniu-se para discutir uma nova e emocionante ideia: a criação de máquinas inteligentes. Foi nessa conferência que o termo **Inteligência Artificial** foi criado, marcando o nascimento oficial da IA como um campo acadêmico (Nilsson, 1982). Imagine a excitação desses pioneiros ao vislumbrarem um futuro em que as máquinas poderiam pensar e agir como humanos.

Nos anos de 1960, Allen Newell e Herbert Simon fizeram avanços significativos ao desenvolverem programas, como o Logic Theorist e o General Problem Solver (GPS). Esses programas buscavam imitar os processos de pensamento humano, provando teoremas de lógica matemática e resolvendo uma ampla gama de problemas. Imagine a sensação de conquista desses cientistas ao verem as próprias máquinas resolverem problemas complexos de maneira similar ao raciocínio humano (Poole; Mackworth; Goebel, 1998).

Décadas de 1970 e 1980: sistemas especialistas e o “Inverno da IA”

Os anos de 1970 foram marcados pelo desenvolvimento de sistemas especialistas, que constituem uma das aplicações mais notáveis da IA simbólica. Esses sistemas armazenavam o conhecimento em bases de dados e usavam mecanismos de inferência para tomar decisões com base em regras predefinidas.

Vamos avançar para a década de 1970, um período de grande inovação na IA, com o surgimento dos sistemas especialistas, que constituem uma das aplicações mais notáveis da IA simbólica. Esses sistemas armazenavam conhecimento em bases de dados e usavam mecanismos de inferência para tomar decisões com base em regras predefinidas. Exemplos notáveis incluem o MYCIN e o DENDRAL, ambos desenvolvidos na Universidade de Stanford. O MYCIN era capaz de diagnosticar infecções bacterianas e recomendar tratamentos, ajudando médicos a tomar decisões mais informadas. O DENDRAL, por sua vez, auxiliava químicos a determinar a estrutura molecular de compostos desconhecidos por meio da análise de dados de espectrometria de massa. Esses sistemas mostraram como a IA podia resolver problemas reais e práticos, demonstrando a respectiva aplicabilidade em áreas especializadas e ajudando profissionais a tomar decisões mais informadas (Nilsson, 1982).

No entanto, a década de 1980 trouxe o “inverno da IA”, um período de frustração e desafios. As expectativas não cumpridas e as limitações tecnológicas levaram a uma redução significativa no financiamento e no interesse pela pesquisa em IA. Imagine a decepção dos pesquisadores ao verem os próprios projetos perderem apoio e recursos. No entanto, mesmo durante esses tempos difíceis, a pesquisa em aprendizado de máquina e redes neurais continuou, o que estabeleceu as bases para futuros desenvolvimentos. É como uma história de superação, em que, mesmo diante das adversidades, a busca pelo conhecimento nunca cessou (Haykin, 2005).

Décadas de 1990 e 2000: ressurgimento e avanços significativos

Chegamos aos anos de 1990, um período de renascimento para a IA. Em 1997, o mundo assistiu a um evento histórico: o Deep Blue da International Business Machines Corporation (IBM) derrotou o campeão mundial de xadrez Garry Kasparov. Imagine a tensão e a emoção enquanto a máquina superava um dos melhores jogadores de xadrez do mundo, demonstrando a incrível capacidade das máquinas de realizar tarefas complexas e desafiadoras. Esse evento marcou um ponto de virada, mostrando que a IA tinha potencial para alcançar feitos extraordinários (Russell; Norvig, 2021).

**De 1990, um período
de renascimento
para a IA**

Nos anos 2000, avanços significativos em **hardware e a explosão de dados** disponíveis permitiram o desenvolvimento de técnicas sofisticadas de aprendizado profundo, também conhecido como *deep learning*. Imagine pesquisadores utilizando poderosos computadores para treinar redes neurais capazes de reconhecer imagens e processar linguagem natural com uma precisão incrível. Esses avanços transformaram a IA, tornando-a ainda mais relevante e aplicável em diversas áreas, desde a medicina até o entretenimento (Haykin, 2005).

Década de 2010 e além: IA Pervasiva

Finalmente, chegamos à década de 2010 e além. A IA se tornou uma parte essencial das nossas vidas diárias. Pense em como assistentes virtuais, como Siri e Alexa, ajudam-nos com tarefas cotidianas, desde definir lembretes até tocar nossas músicas favoritas. Veículos autônomos estão revolucionando o transporte, prometendo um futuro com menos acidentes e mais eficiência. Quem pode esquecer o momento em 2016, quando o AlphaGo da DeepMind venceu campeões mundiais no jogo de Go? Diante disso, foi demonstrada a incrível capacidade das redes neurais profundas de lidar com tarefas extremamente complexas e abstratas (Russell; Norvig, 2021).

Para entender melhor a evolução da Inteligência Artificial e conhecer as figuras proeminentes que moldaram a história e o desenvolvimento dela, desde os fundamentos matemáticos e teóricos até as aplicações práticas e impacto na sociedade contemporânea, vamos explorar alguns pioneiros e as respectivas contribuições.

Embora a década de 2010 tenha testemunhado o domínio do aprendizado profundo, a IA simbólica ainda desempenha um papel essencial. Integrações entre métodos simbólicos e estatísticos, como o raciocínio simbólico em sistemas híbridos, mostram a relevância contínua dessa abordagem. Por exemplo, na medicina, os sistemas que combinam regras simbólicas e aprendizado profundo são usados para diagnósticos mais precisos.

**Alan Turing**

Matemático britânico e pioneiro da computação. Propôs o Teste de Turing para avaliar a inteligência das máquinas.

**John McCarthy**

Cunhou o termo "Inteligência Artificial" e organizou a Conferência de Dartmouth, em 1956, ponto de partida da IA.

**Marvin Minsky**

Cofundador do MIT Media Lab, fez avanços na compreensão da Inteligência Artificial e da mente humana.



Nathaniel Rochester

Participou da Conferência de Dartmouth e trabalhou no desenvolvimento do computador IBM 701, contribuindo para a IA.



Claude Shannon

Pai da Teoria da Informação, as ideias de Shannon influenciaram profundamente a Teoria da Computação e a IA.



Allen Newell

Desenvolveu o Logic Theorist e o General Problem Solver (GPS), buscando imitar o pensamento humano.

**Herbert Simon**

Colaborou no desenvolvimento do Logic Theorist e do GPS. Recebeu o Nobel de Economia por pesquisas em teoria da decisão.

**Garry Kasparov**

Grande mestre de xadrez derrotado pelo Deep Blue, da IBM, em 1997, demonstrando a capacidade das máquinas.

Cada uma dessas décadas trouxe avanços significativos que moldaram o desenvolvimento da IA, o que consolida uma trajetória de inovação contínua e superação de desafios. Essa evolução rápida e constante é o que torna o campo da IA tão excitante e promissor, sempre com novas fronteiras a serem exploradas e conquistas a serem alcançadas.

FUNDAMENTOS E IMPORTÂNCIA DA IA

Vamos, agora, explorar os conceitos-chave da Inteligência Artificial (IA) e entender a devida importância no mundo moderno. A IA é um campo vasto e multifacetado. Para apreciá-la plenamente, precisamos nos aprofundar nas principais vertentes e aplicações.



IA Simbólica e Subsimbólica

Para começarmos, é essencial entendermos a diferença entre a IA simbólica e a IA subsimbólica. A IA simbólica, também conhecida como IA baseada em regras, utiliza símbolos e regras lógicas para representar e processar conhecimento. Imagine um médico usando um sistema de diagnóstico assistido por computador que faz perguntas específicas sobre os sintomas do paciente e, em seguida, usa regras predefinidas para sugerir diagnósticos. Esse é um exemplo clássico de IA simbólica. Esse tipo de IA é excelente para tarefas que requerem raciocínio lógico e manipulação explícita de conhecimento, como planejamento estratégico e diagnósticos médicos. A IA simbólica foi pioneira na resolução de problemas complexos e na automação de processos decisórios em várias indústrias (Nilsson, 1982).

Por outro lado, a IA subsimbólica envolve métodos que não utilizam representações explícitas de conhecimento, como redes neurais artificiais e algoritmos de aprendizado de máquina. Imagine um aplicativo de reconhecimento facial que pode identificar uma pessoa com base em milhares de fotos. Esse aplicativo não segue regras predefinidas, mas aprende padrões a partir dos dados que recebe. Esse tipo de IA é essencial para tarefas que envolvem o reconhecimento de padrões e aprendizado a partir de grandes volumes de dados, tais como re-

conhecimento de voz e imagem. A IA subsimbólica tem transformado áreas, tais como segurança, saúde e entretenimento, permitindo avanços que, antes, pareciam impossíveis (Haykin, 2005).

Observe, na tabela a seguir, as diferenças entre IA simbólica e IA subsimbólica.

CARACTERÍSTICA	IA SIMBÓLICA	IA SUBSIMBÓLICA
DEFINIÇÃO	Baseia-se na manipulação de símbolos e regras lógicas para representar e processar conhecimento.	Envolve métodos que não utilizam representações explícitas de conhecimento, como redes neurais artificiais e algoritmos de aprendizado de máquina.
MÉTODO DE FUNCIONAMENTO	Utiliza regras explícitas e lógicas para tomar decisões e resolver problemas.	Aprende padrões diretamente dos dados, ajustando-se com base em exemplos e experiências.
EXEMPLOS	Sistemas especialistas, como DENDRAL e MYCIN, e algoritmos de busca.	Redes neurais, aprendizado profundo (deep learning) e algoritmos de aprendizado de máquina.
PRINCIPAIS APLICAÇÕES	Diagnóstico médico, planejamento estratégico e resolução de teoremas.	Reconhecimento de imagem, processamento de linguagem natural e recomendação de produtos.
VANTAGENS	Alta explicabilidade e interpretabilidade das decisões. Facilidade de incorporação de conhecimento especializado.	Capacidade de lidar com grandes volumes de dados e identificar padrões complexos. Alta adaptabilidade.

CARACTERÍSTICA	IA SIMBÓLICA	IA SUBSIMBÓLICA
DESVANTAGENS	Dependência de conhecimento explícito e regras predefinidas. Dificuldade em lidar com dados não estruturados e variáveis.	Menor interpretabilidade. Necessidade de grandes volumes de dados para um treinamento eficaz.
EXEMPLOS DE FERRAMENTAS	Sistemas baseados em regras, motores de inferência.	TensorFlow, Keras, PyTorch (para redes neurais e deep learning).
REQUISITOS DE CONHECIMENTO	Requer conhecimento profundo em lógica e matemática.	Requer conhecimento em estatística, probabilidade e técnicas de aprendizado de máquina.
INTERPRETAÇÃO DE RESULTADOS	Resultados são facilmente interpretáveis e rastreáveis até as regras aplicadas.	Resultados podem ser vistos como "caixas pretas" devido à complexidade das redes neurais e modelos de aprendizado profundo.
HISTÓRIA	Desenvolvimento significativo nas décadas de 1950 e 1960, com a criação de sistemas especialistas.	Ganhou destaque nas décadas de 1980 e 1990, com o avanço do hardware e a explosão de dados, levando ao desenvolvimento de redes neurais profunda.

Tabela 1 – Diferenças entre IA simbólica e IA subsimbólica
Fonte: o autor.

Tanto a IA simbólica quanto a IA subsimbólica têm desempenhado papéis fundamentais no avanço da Inteligência Artificial. A IA simbólica trouxe as primeiras soluções para problemas complexos por meio de regras explícitas e lógica formal, enquanto a IA subsimbólica, com as capacidades de aprendizado de máquina e redes neurais, revolucionou a forma como lidamos com grandes volumes de dados e reconhecimento de padrões. Ambas as abordagens conti-

nuam a evoluir e a se complementar, abrindo caminho para novas aplicações e melhorias contínuas em diversas áreas da sociedade. Vamos, agora, explorar como a Inteligência Artificial está moldando a nossa sociedade e quais são os impactos e as aplicações mais significativos dela.

Fundamentos e importância da IA no mundo moderno

A importância da IA no mundo moderno não pode ser subestimada. Ela já está presente em muitos aspectos de nossas vidas diárias e continua a transformar indústrias inteiras. Vamos considerar algumas áreas específicas em que a IA tem um impacto significativo.

No setor da saúde, a IA está revolucionando o diagnóstico e o tratamento de doenças. Imagine um sistema de IA que pode analisar milhões de registros médicos e identificar padrões que um humano poderia facilmente perder. Esses sistemas ajudam médicos a fazer diagnósticos mais precisos e rápidos, melhorando significativamente os resultados aos pacientes. Além disso, a IA pode sugerir tratamentos personalizados com base no histórico médico de um paciente, tornando a medicina mais eficaz e personalizada (Russell; Norvig, 2021).

No campo da logística, a IA otimiza rotas de entrega, economizando tempo e recursos. Imagine uma empresa de transporte que precisa entregar pacotes em várias cidades. Com a IA, a empresa pode calcular a rota mais eficiente em tempo real, levando em consideração fatores, como trânsito e condições meteorológicas. Isso não apenas reduz os custos operacionais, mas também melhora a satisfação do cliente, ao garantir entregas mais rápidas e confiáveis (Nilsson, 1982).

Na área do comércio, a IA está mudando a forma como as empresas interagem com os clientes. Pense em sistemas de recomendação que sugerem produtos com base no histórico de compras de um cliente. Esses sistemas utilizam algoritmos de aprendizado de máquina para prever o que os clientes podem querer comprar a seguir, aumentando as vendas e melhorando a experiência do cliente. Além disso, chatbots de IA estão se tornando cada vez mais comuns no atendimento ao cliente, oferecendo suporte 24/7 e respostas rápidas a consultas (Haykin, 2005).

BASES TEÓRICAS DA IA

Para entender como a IA funciona, precisamos explorar as respectivas bases teóricas. A lógica proposicional e a lógica de predicados são fundamentais para a IA simbólica. Elas permitem a representação de conhecimento e a inferência lógica, sendo essenciais para a criação de sistemas que podem raciocinar sobre informações complexas. Imagine um sistema de IA que pode deduzir novas informações a partir de um conjunto de fatos conhecidos, semelhante à forma como um humano resolveria um quebra-cabeça lógico (Nilsson, 1982).

A teoria da computação também desempenha um papel crucial. Ela nos ajuda a entender as capacidades e as limitações dos algoritmos de IA. Por exemplo, a análise de algoritmos de busca é essencial para garantir que os sistemas de IA possam encontrar soluções eficientes para problemas complexos. Pense em como o Google encontra os resultados mais relevantes em uma fração de segundo. Isso é possível graças à aplicação de princípios de teoria da computação (Russell; Norvig, 2021).

Além disso, a estatística e a probabilidade são fundamentais para muitos métodos de aprendizado de máquina. Esses princípios permitem que os sistemas de IA infiram padrões e façam previsões a partir de dados, mesmo quando esses dados são incertos ou incompletos. Imagine um sistema de previsão do tempo que usa redes bayesianas para modelar a incerteza nos dados meteorológicos e fornecer previsões precisas (Poole; Mackworth; Goebel, 1998).



APROFUNDANDO

O que são redes bayesianas?

Redes bayesianas são modelos matemáticos que ajudam a entender como diferentes fatores se relacionam entre si e como essas relações afetam a probabilidade de certos eventos acontecerem (Luger, 2005). Imagine que você tem várias peças de informação, como a temperatura, a pressão do ar e a umidade. Uma rede bayesiana conecta essas peças, mostrando como uma pode influenciar a outra. Quando novos dados chegam, o modelo atualiza automaticamente as probabilidades dos eventos. Por exemplo, em previsão do tempo, uma rede bayesiana pode usar essas informações para calcular a chance de chover, ajustando-se conforme mais dados meteorológicos são recebidos. Esse tipo de modelo é muito útil para tomar decisões em situações em que os dados podem ser incertos ou incompletos.

Por fim, a filosofia e a epistemologia ajudam a abordar as questões profundas sobre a natureza do conhecimento e a representação da realidade. Essas disciplinas influenciam o modo como os sistemas de IA são projetados e compreendidos, garantindo que eles não apenas funcionem bem, mas também sejam éticos e transparentes. Imagine um sistema de IA que toma decisões importantes, como conceder um empréstimo bancário. É crucial que essas decisões sejam justas e baseadas em critérios transparentes. Aqui, é onde a filosofia e a epistemologia desempenham um papel vital (Luger, 2021). Na Figura 1, são mostradas as bases da IA.

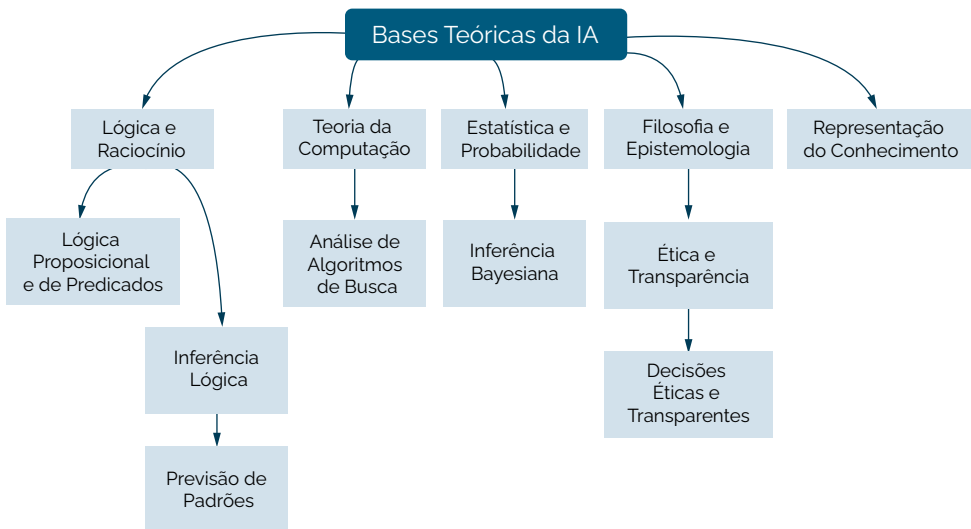


Figura 1 - Bases teóricas da IA / Fonte: o autor.

Descrição da Imagem: é exibido um diagrama que organiza os principais componentes teóricos que fundamentam a IA, dividindo-os em áreas específicas e subáreas relacionadas. Ele destaca como a lógica e o raciocínio, a teoria da computação, a estatística e probabilidade, a filosofia e a epistemologia, e a representação do conhecimento contribuem para o campo da IA. Há o seguinte conteúdo: Bases Teóricas da IA, Lógica e Raciocínio, Lógica Proposicional e de Predicados, Inferência Lógica, Previsão de Padrões, Teoria da Computação, Análise de Algoritmos de Busca, Estatística e Probabilidade, Inferência Bayesiana, Filosofia e Epistemologia, Ética e Transparência, Decisões Éticas e Transparentes, e Representação do Conhecimento. Fim da descrição.

Compreender esses conceitos-chave e as bases teóricas da IA nos ajuda a apreciar o poder e o potencial dessa tecnologia incrível. Vamos, agora, aprofundar-nos na maneira como podemos começar a programar IA utilizando Python, a linguagem favorita dos cientistas de dados e engenheiros de IA.



PROGRAMAÇÃO EM PYTHON: INTRODUÇÃO E CONFIGURAÇÃO DO AMBIENTE

Agora que temos uma boa compreensão da história e dos conceitos fundamentais da Inteligência Artificial (IA), vamos nos aventurar em uma parte prática: a programação em Python. Python é uma das linguagens de programação mais populares para o desenvolvimento de IA, devido à simplicidade, versatilidade e uma vasta quantidade de bibliotecas especializadas. Vamos configurar o nosso ambiente de desenvolvimento e dar os primeiros passos na criação de algoritmos de IA.



PENSANDO JUNTOS

Por que Python?

Você deve estar se perguntando: por que Python? A resposta é simples. Python é fácil de aprender e usar, tem uma sintaxe clara e legível, e é altamente eficiente. Além disso, a comunidade de Python é enorme e extremamente ativa, o que significa que há uma abundância de recursos, tutoriais e bibliotecas disponíveis para ajudar em praticamente qualquer projeto de IA. É a linguagem preferida de cientistas de dados e engenheiros de IA em todo o mundo.

Configurando o Ambiente: Passo a Passo

Vamos começar configurando o nosso ambiente de desenvolvimento Python. Imagine que você está prestes a montar uma oficina de robótica. Primeiro, você precisa garantir que tem todas as ferramentas necessárias à sua disposição. Com Python, isso significa instalar o Python em si e algumas bibliotecas essenciais.

1. Instalação do Python:

- Acesse o site oficial do Python (<https://www.python.org/>) e baixe a versão recomendada, que geralmente é a mais recente. Para a IA, recomendamos a versão 3.7 ou superior. Após o download, siga as instruções de instalação específicas para o seu sistema operacional.

2. Instalação de Bibliotecas Essenciais:

- O Python vem com um gerenciador de pacotes chamado pip. Ele facilita a instalação de bibliotecas adicionais. A seguir, estão algumas bibliotecas fundamentais para trabalhar com IA:
 - **numpy**: para operações numéricas e manipulação de arrays. É a base para muitas outras bibliotecas de ciência de dados (NumPy, 2025).
 - **pandas**: para manipulação e análise de dados. Facilita o trabalho com grandes conjuntos de dados (Pandas, 2025).
 - **scikit-learn**: para aprendizado de máquina. Inclui ferramentas simples e eficientes para análise de dados e mineração de dados (Scikit-Learn, 2025).
 - **tensorflow** e **keras**: para redes neurais e aprendizado profundo. São bibliotecas poderosas para construir e treinar modelos de aprendizado profundo (Tensorflow, 2025; Keras, 2025).
 - **matplotlib** e **seaborn**: para visualização de dados. Ajudam a criar gráficos e visualizações para melhor entender os dados (Matplotlib, 2025; Seaborn, 2025).

3. Instalando as Bibliotecas:

- Abra o seu terminal ou prompt de comando e execute os seguintes comandos:
`pip install numpy pandas scikit-learn tensorflow keras matplotlib seaborn.`

4. Escolhendo um IDE:

Um Ambiente de Desenvolvimento Integrado (IDE) facilita a escrita e a execução de códigos. O Jupyter Notebook é altamente recomendado para iniciantes em IA. Ele permite a escrita de código Python em células interativas, facilitando a experimentação e a visualização de resultados. Para instalar o Jupyter Notebook, basta rodar:

```
pip install notebook
```

Depois de instalado, você pode iniciar o Jupyter Notebook com o comando:
`jupyter notebook`

Alternativas de Ferramentas

Além do Jupyter Notebook, há outras ferramentas que podem ser extremamente úteis para desenvolver projetos de IA:

- **Anaconda:**

- Anaconda é uma distribuição gratuita e de código aberto do Python que inclui muitos pacotes úteis para ciência de dados e IA. Ela vem com o Jupyter Notebook e o JupyterLab pré-instalados, além de ferramentas adicionais, como o Spyder. Anaconda facilita a gestão de pacotes e ambientes, o que é muito útil para manter os seus projetos organizados e separados (Santos, 2023).
- Para instalar o Anaconda, você pode visitar o site oficial (<https://www.anaconda.com/>) e baixar a versão adequada para o seu sistema operacional. Uma vez instalado, você pode criar ambientes virtuais específicos para cada projeto, garantindo que cada um tenha as próprias dependências e configurações (Santos, 2023).

- **Google Colab:**

- O Google Colab é uma plataforma gratuita que permite escrever e executar código Python diretamente no seu navegador, sem precisar configurar nada localmente. É especialmente útil para quem está começando ou para aqueles que precisam de um ambiente de desenvolvimento rápido e acessível (Santos, 2023).
- Uma grande vantagem do Colab é que ele oferece acesso a GPUs gratuitas, o que pode acelerar significativamente o treinamento de modelos de aprendizado profundo. Você pode acessar o Colab a partir do link (<https://colab.research.google.com/>) e começar a codificar imediatamente, sem necessidade de instalação (Santos, 2023).

Neste tema, exploramos não apenas a teoria da Inteligência Artificial, mas também demos os primeiros passos práticos na programação em Python para IA. Python, com as poderosas bibliotecas e comunidade ativa, torna o desenvolvimento de IA acessível e eficiente. Ao configurar seu ambiente e começar a experimentar com código, você está no caminho certo para criar soluções de IA que podem ter um impacto significativo no mundo real. A aventura pelo mundo da IA está

apenas começando. Continue explorando, aprendendo e experimentando. O campo da IA é vasto e cheio de oportunidades emocionantes. Você, agora, está bem equipado para navegar por ele com confiança.

**EM FOCO**

Gostou desse assunto? Então, não perca tempo e dê o play na videoaula deste tema. Venha aprender mais! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Depois de explorarmos a história da Inteligência Artificial (IA), entendermos os conceitos-chave e configurarmos o ambiente de desenvolvimento em Python, estamos prontos para enfrentar novos desafios. Esta seção se concentra em conectar a teoria com a prática, preparando você para aplicar o conhecimento adquirido em situações reais e no ambiente profissional.

A IA não é apenas um campo de estudo acadêmico: as aplicações práticas dela são vastas e crescentes. Imagine estar no início de sua carreira e ter a oportunidade de aplicar IA em seu trabalho diário. Como a IA pode transformar o seu setor? Quais são os problemas práticos que você pode resolver com a IA? Vamos explorar alguns desses desafios e entender como a teoria se traduz em prática.

Setor de saúde: nesse setor, um dos grandes desafios é o diagnóstico preciso e precoce de doenças. A Inteligência Artificial, por meio de algoritmos de aprendizado de máquina, tem se mostrado uma ferramenta poderosa para superar esse obstáculo. Ao analisarem imagens médicas, como radiografias e tomografias, esses algoritmos são capazes de identificar padrões que indicam a presença de doenças, como o câncer, com alta precisão. As redes neurais convolucionais (CNNs) são especialmente eficazes nessa tarefa, graças à capacidade de extrair características relevantes de imagens.

Logística e transporte: otimizar rotas de entrega é um desafio constante para empresas de logística. A Inteligência Artificial oferece soluções inovadoras para esse problema. Mediante algoritmos de busca e otimização, é possível encontrar as rotas mais eficientes, considerando diversos fatores, como tráfego, condições climáticas e horários de entrega. Essa otimização resulta em redução de custos e melhora a eficiência das operações. Um exemplo prático é o uso de sistemas de otimização de rotas em empresas de logística.

Comércio e varejo: no setor de comércio e varejo, a personalização da experiência do cliente é fundamental para aumentar as vendas. A Inteligência Artificial, por meio de sistemas de recomendação, permite analisar o histórico de compras e o comportamento dos clientes para sugerir produtos relevantes. Algoritmos, como filtro colaborativo e aprendizado profundo, são comumente utilizados para criar recomendações personalizadas, aumentando as chances de conversão.

Segurança cibernética: a crescente complexidade das ameaças cibernéticas exige soluções cada vez mais sofisticadas. A Inteligência Artificial tem um papel fundamental na detecção e prevenção de ataques cibernéticos. Ao analisarem grandes volumes de dados de rede, os algoritmos de aprendizado de máquina podem identificar padrões anômalos que indicam a presença de atividades maliciosas. Sistemas de detecção de intrusão baseados em IA são cada vez mais comuns e eficazes na proteção de sistemas e dados.

Como a evolução da IA simbólica pode impactar a sua área de atuação? Quais são as possíveis aplicações e implicações éticas da IA no seu campo de trabalho? Reflita sobre como os conhecimentos adquiridos podem ser usados para melhorar processos e tomar decisões mais informadas. Considere também os desafios éticos, como a privacidade dos dados e a transparência nos processos decisórios. É crucial que futuros profissionais não só dominem a tecnologia, mas também entendam as responsabilidades associadas ao uso. Como você pode garantir que as soluções desenvolvidas sejam justas, éticas e benéficas para a sociedade?

AUTOATIVIDADE

1. Foi proposto um teste para determinar se uma máquina é capaz de exibir comportamento inteligente equivalente ao de um ser humano. No teste, um avaliador humano interage com uma máquina e um ser humano sem saber qual é qual, e tenta distinguir entre os dois.

Quem propôs o teste descrito e em que ano?

- a) John McCarthy, em 1956.
 - b) Marvin Minsky, em 1965.
 - c) Claude Shannon, em 1948.
 - d) Alan Turing, em 1950.
 - e) Herbert Simon, em 1960.
-
2. Nos anos de 1960 e 1970, a pesquisa em IA simbólica avançou com o desenvolvimento de sistemas especialistas. Um exemplo notável é o MYCIN, utilizado para diagnósticos médicos. Sistemas especialistas utilizam regras baseadas em conhecimento específico para resolver problemas em áreas bem definidas.

Considerando as aplicações práticas de sistemas especialistas, analise as afirmativas a seguir:

- I - Diagnóstico de doenças.
- II - Planejamento estratégico em empresas.
- III - Análise de sentimentos em textos.
- IV - Previsão de eventos climáticos.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) I e III, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

3. Ferramentas, como Anaconda e Google Colab, são amplamente utilizadas no desenvolvimento de projetos de IA devido às funcionalidades e facilidades de uso. Anaconda é uma plataforma que simplifica a gestão de pacotes e ambientes, permitindo aos desenvolvedores configurar e gerenciar bibliotecas necessárias para projetos de IA. Google Colab, por outro lado, é uma ferramenta baseada na web que oferece recursos computacionais avançados, como GPUs, facilitando o treinamento de modelos de IA diretamente no navegador.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - Anaconda facilita a instalação e o gerenciamento de bibliotecas necessárias para projetos de IA.

PORQUE

II - Anaconda oferece um ambiente pré-configurado, com várias bibliotecas úteis para IA, como numpy, pandas e tensorflow.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

HAYKIN, S. **Redes neurais**: princípios e prática. São Paulo: Bookman, 2005.

KERAS. The Python Deep Learning API. **Keras**, c2025. Disponível em: <https://keras.io>. Acesso em: 13 jan. 2025.

LUGER, G. F. **Artificial Intelligence**: Structures and Strategies for Complex Problem Solving. New Jersey: Pearson, 2005.

LUGER, G. F. **Knowing our World**: An Artificial Intelligence Perspective. New York: Springer, 2021.

MATPLOTLIB. Matplotlib: Visualization with Python. **Matplotlib**, c2025. Disponível em: <https://matplotlib.org>. Acesso em: 13 jan. 2025.

NILSSON, N. J. **Principles of Artificial Intelligence**. San Francisco: Morgan Kaufmann Publishers, 1982.

NUMPY. NumPy 2.2.0 released! **NumPy**, c2025. Disponível em: <https://numpy.org>. Acesso em: 29 nov. 2024.

PANDAS. Pandas: Python Data Analysis Library. **Pandas**, c2025. Disponível em: <https://pandas.pydata.org>. Acesso em: 13 jan. 2025.

POOLE, D.; MACKWORTH, A.; GOEBEL, R. **Computational Intelligence**: a logical approach. New York: Oxford University Press, 1998.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence**: A Modern Approach. New Jersey: Pearson, 2021.

SANTOS, R. M dos. Anaconda – Introdução e Instalação. **Brains**, 9 fev. 2023. Disponível em: <https://brains.dev/2023/anaconda-introducao-e-instalacao/>. Acesso em: 13 jan. 2025.

SANTOS, T. G. Google Colab: o que é, tutorial de como usar e criar códigos. **Alura**, 18 set. 2023. Disponível em: <https://www.alura.com.br/artigos/google-colab-o-que-e-e-como-usar>. Acesso em: 13 jan. 2025.

SCIKIT-LEARN. Scikit-learn: Machine Learning in Python. **Scikit-learn**, c2025. Disponível em: <https://scikit-learn.org>. Acesso em: 13 jan. 2025.

SEABORN. Seaborn: Statistical Data Visualization. **Seaborn**, c2025. Disponível em: <https://seaborn.pydata.org>. Acesso em: 13 jan. 2025.

TENSORFLOW. An end-to-end open-source machine learning platform. **TensorFlow**, c2025. Disponível em: <https://www.tensorflow.org>. Acesso em: 13 jan. 2025.

GABARITO

1. Opção D.

John McCarthy é conhecido por cunhar o termo "Inteligência Artificial", e não por ter proposto o Teste de Turing. Marvin Minsky contribuiu significativamente para a Inteligência Artificial, mas não foi ele quem propôs o Teste de Turing. Claude Shannon é conhecido como o "pai da teoria da informação" e as contribuições dele foram fundamentais para a teoria da computação, mas ele não propôs o Teste de Turing. Alan Turing propôs o Teste de Turing em 1950 como um critério para avaliar a inteligência de uma máquina. Esse teste consiste em um avaliador humano interagir com uma máquina e um ser humano, sem saber qual é qual, e determinar se a máquina pode ser diferenciada do humano. Se o avaliador não conseguir distinguir entre os dois, a máquina é considerada inteligente. Herbert Simon foi um pioneiro em Inteligência Artificial e colaborou com Allen Newell no desenvolvimento de programas de IA, mas não propôs o Teste de Turing.

2. Opção C.

Sistemas especialistas são utilizados em diversas aplicações práticas, como diagnóstico de doenças (I), exemplificado pelo MYCIN, e análise de sentimentos em textos (III), em que sistemas processam texto para identificar emoções e opiniões. Planejamento estratégico em empresas (II) e previsão de eventos climáticos (IV) são tarefas que geralmente envolvem variáveis complexas e dinâmicas, sendo mais adequadas para técnicas de aprendizado de máquina e modelagem estatística.

3. Opção A

As asserções I e II são verdadeiras, e a segunda justifica a primeira, porque Anaconda, ao oferecer um ambiente pré-configurado com bibliotecas, como numpy, pandas e tensorflow, simplifica o processo de instalação e gerenciamento de bibliotecas necessárias para projetos de IA. Isso garante que os usuários tenham as ferramentas necessárias para começar a desenvolver imediatamente.



TEMA DE APRENDIZAGEM 2

TEORIA DE PROBLEMAS E IA: MODELAGEM E SOLUÇÕES PRÁTICAS

MINHAS METAS

- Definir problemas na IA, incluindo o espaço de estados e objetivos.
- Modelar matematicamente problemas e soluções na IA.
- Entender a importância do espaço de estados na resolução de problemas.
- Descrever métodos para definir objetivos em problemas de IA.
- Implementar algoritmos básicos de resolução de problemas em Python.
- Desenvolver habilidades práticas na modelagem de problemas de IA.
- Aplicar conceitos teóricos em atividades práticas de IA.

INICIE SUA JORNADA

A Inteligência Artificial (IA) tem o poder de transformar a maneira como resolvemos problemas complexos. No centro da IA, está a capacidade de formular problemas de maneira precisa, definindo o espaço de estados e objetivos claros.

Portanto, como a IA pode ser usada para resolver problemas complexos de maneira eficiente? Entender a formulação de problemas é o primeiro passo para criar algoritmos eficazes que podem ter um impacto significativo em diversas áreas.

Considere, por exemplo, um **sistema de diagnóstico** médico que precisa identificar doenças a partir dos sintomas apresentados pelos pacientes. A IA pode ser usada para analisar grandes volumes de dados médicos, comparar sintomas e identificar padrões que levam a diagnósticos precisos. No entanto, como você abordaria a formulação desse problema? Para isso, é necessário pensar em como definir o problema, o espaço de estados, os possíveis objetivos e como modelar matematicamente a solução.

Para ilustrar melhor, pense no modo como você já aplicou algoritmos em problemas matemáticos, como a busca por soluções de equações complexas. De forma semelhante, a IA utiliza algoritmos de busca para explorar um espaço de estados e encontrar soluções para problemas definidos. Imagine resolver um quebra-cabeça no qual cada peça se encaixa apenas de uma maneira específica; da mesma forma, a IA precisa analisar diversas combinações e estados possíveis para encontrar a solução ideal. Ao aplicar esses conceitos conhecidos de forma prática, você poderá perceber como a IA modela e resolve problemas complexos, transformando informações teóricas em ferramentas eficazes para a solução de desafios reais.

Para concretizar essa ideia, imagine que você está desenvolvendo um sistema de recomendação de filmes que precisa sugerir os filmes mais relevantes para os usuários com base nas preferências deles. Nesse contexto, como você aplicaria a IA simbólica para resolver esse problema? Desenvolva um pequeno estudo de caso considerando os seguintes pontos: identificação do problema, coleta de dados relevantes, aplicação de algoritmos de busca para encontrar os filmes mais adequados e avaliação dos resultados. Esse exercício prático te permitirá compreender como a teoria da IA simbólica pode ser aplicada em situações reais, o que te preparará para enfrentar desafios semelhantes em sua carreira profissional.

Por fim, reflita sobre como a formulação de problemas na IA pode impactar a sua área de atuação. Pense nas aplicações práticas desses conceitos no seu campo de trabalho e como você pode usar o conhecimento adquirido para melhorar processos e resolver problemas complexos de maneira mais eficiente. Considere também as implicações éticas e a importância de desenvolver soluções justas e transparentes.



PLAY NO CONHECIMENTO

Neste episódio do podcast, exploramos como a IA formula e resolve problemas complexos de maneira eficiente. Além dos conceitos teóricos, discutiremos exemplos práticos e reais que mostram a aplicação da IA em diferentes setores. Acesse-o agora e expanda o seu conhecimento sobre essa tecnologia inovadora! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para garantir que você esteja preparado para compreender e aplicar os conceitos de formulação de problemas na IA, vamos relembrar alguns conteúdos essenciais. Primeiro, é importante obter uma compreensão clara do problema a ser resolvido, definindo-o de forma precisa e entendendo os respectivos contexto e impacto nas partes interessadas. Além disso, definir os objetivos finais e o plano do projeto é fundamental. Metas claras e mensuráveis orientam o desenvolvimento de soluções de IA. Colaborar com os usuários finais e as partes interessadas para formular questões relevantes garante que todas as questões importantes sejam abordadas. Finalmente, identificar as fontes de dados relevantes é essencial, porque fontes de dados precisas são a base para uma boa solução de IA.

<https://www.scielo.br/j/trans/a/xD6qrKMbCkNnm33Nkppmgz/?format=pdf>

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À FORMULAÇÃO DE PROBLEMAS NA IA

Vamos embarcar em uma jornada fascinante pelo mundo da formulação de problemas na Inteligência Artificial (IA). A formulação de problemas é uma das etapas mais essenciais no desenvolvimento de sistemas de IA eficientes e eficazes. Este tema de aprendizagem oferece uma visão abrangente e envolvente sobre como definir problemas na IA, modelagem matemática e implementação de algoritmos em **Python**. Entender como os problemas são formulados e resolvidos pode abrir portas para uma ampla gama de aplicações práticas, desde recomendações de filmes até diagnósticos médicos e otimização de rotas logísticas. Preparado? Então, vamos lá!

A formulação de problemas na IA começa com a definição clara do problema. Isso envolve identificar os elementos principais, como o estado inicial, o espaço de estados e o estado objetivo. A modelagem matemática, que utiliza ferramentas, como grafos e diagramas de estados, é fundamental para representar esses problemas de forma estruturada. Finalmente, a implementação prática em Python, uma das linguagens mais populares para IA, permite que esses conceitos sejam aplicados em situações reais, criando soluções que podem ter um impacto significativo em diversas áreas.

Compreendendo a formulação de problemas

A palavra **problema** possui raízes profundas na história da humanidade, remontando à Grécia Antiga. Derivada do grego (*próblema*), que significa **obstáculo** ou **algo lançado à frente**, a palavra originalmente designava uma barreira ou um desafio a ser superado. Na filosofia e matemática gregas, um problema era uma questão que exigia uma solução lógica ou matemática (Aristóteles, 2021).

Durante a Idade Média, o conceito se expandiu para incluir desafios práticos e teológicos, com estudiosos europeus sistematizando a respectiva resolução mediante métodos científicos e racionais. No Renascimento e na Revolução Científica, o termo “problema” começou a ser usado em contextos científicos e técnicos mais amplos, com metodologias rigorosas desenvolvidas para defini-lo e resolvê-lo (Koetsier, 2019).

A definição formal de problema em termos científicos e matemáticos começou a tomar forma no século XVIII, com o desenvolvimento do cálculo e da física newtoniana (Guicciardini, 2003). Um problema passou a ser visto como uma questão bem definida, a qual podia ser abordada por meio de métodos analíticos e quantitativos.

Com o advento da computação no século XX, a definição de problema evoluiu para incluir desafios que podiam ser resolvidos por máquinas. Matemáticos e cientistas da computação formularam problemas de maneira que computadores pudessem processar e encontrar soluções. A lógica formal e a teoria dos algoritmos se tornaram fundamentais para a definição e a resolução de problemas computacionais.

Na segunda metade do século XX, com o surgimento da Inteligência Artificial (IA), a definição de problema se tornou ainda mais sofisticada. A IA busca não apenas resolver problemas por intermédio de métodos predefinidos, mas também mediante aprendizado e adaptação. Um problema em IA pode ser definido formalmente por cinco componentes (Russell; Norvig, 2021):

1. ESTADO INICIAL

Este é o ponto de partida do problema, representando a situação atual antes de qualquer ação ser tomada. No caso do sistema de recomendação de filmes, o estado inicial inclui os dados sobre filmes, como gênero, diretor, atores principais e avaliações. Esses dados são coletados e organizados para serem utilizados pelos algoritmos de IA.

2. AÇÕES POSSÍVEIS

Descrevem todas as ações disponíveis para o sistema em um determinado estado. Por exemplo, as ações possíveis podem ser "Recomendar Filme A", "Recomendar Filme B" etc. Cada ação deve ser aplicável no estado atual.

3. MODELO DE TRANSIÇÃO

Especifica o efeito de cada ação, indicando qual será o próximo estado após a execução de uma ação. Por exemplo, se o estado atual inclui que um filme foi recomendado e aceito pelo usuário, o próximo estado incluirá a atualização das preferências do usuário com base na aceitação dessa recomendação. O modelo de transição ajuda a definir o **espaço de estados** do problema, que é o conjunto de todos os estados acessíveis a partir do estado inicial.

4. TESTE DE OBJETIVO

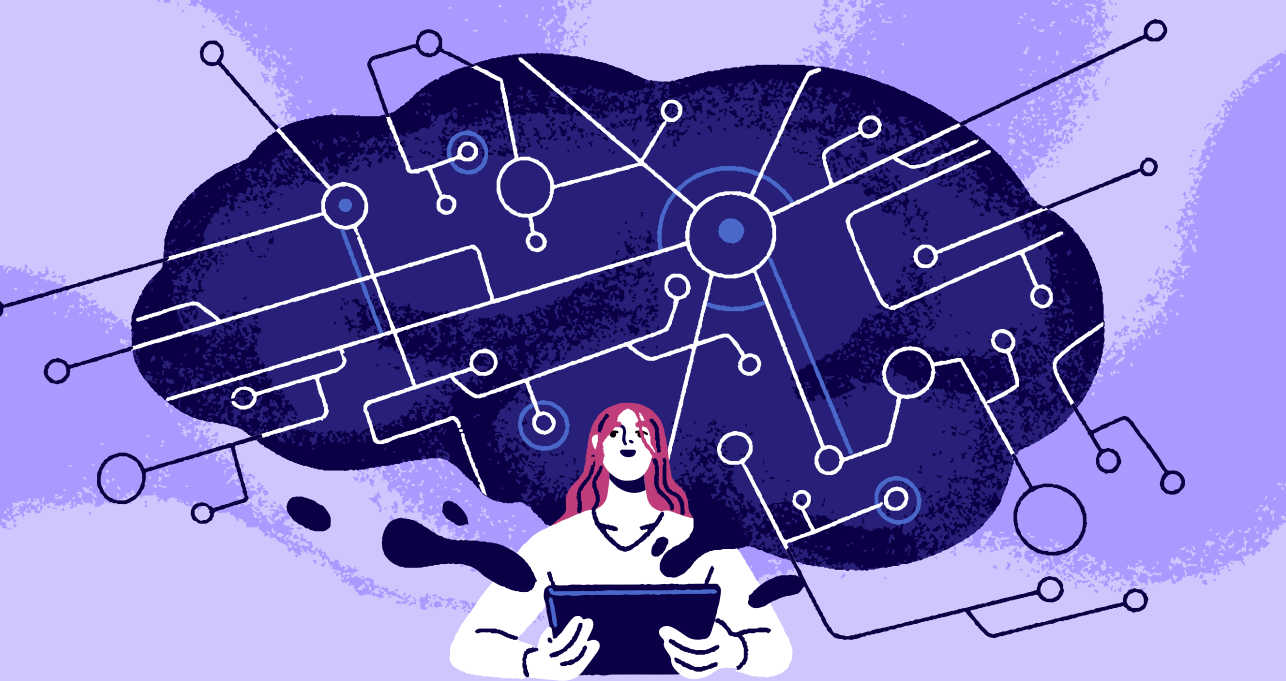
Determina se um estado é um estado objetivo. No caso do sistema de recomendação de filmes, o objetivo pode ser definido como a satisfação do usuário com as recomendações fornecidas.

5. FUNÇÃO DE CUSTO DE CAMINHO

Atribui um custo numérico a cada caminho ou sequência de ações. A função de custo pode refletir a precisão das recomendações, minimizando a insatisfação do usuário.

Usando esses componentes, formulamos um modelo para o sistema de recomendação. Embora esse modelo seja uma boa representação, ele é abstrato e simplificado. Logo, não reflete completamente a complexidade do mundo real. Na prática, precisamos lidar com muitos detalhes adicionais, como preferências pessoais, popularidade dos filmes e tendências de visualização. Para gerenciar essa complexidade, usamos a abstração, que objetiva remover detalhes desnecessários e focar no essencial (Russell; Norvig, 2021).

Por exemplo, ao contrário de considerar todas as possíveis variáveis do usuário e do filme, focamos apenas em alguns dados críticos, como avaliações e gêneros. Além disso, simplificamos as ações para apenas recomendações diretas, ignorando outros possíveis efeitos, tais como mudanças na popularidade do filme ou novas tendências.



APROFUNDANDO

A abstração em Inteligência Artificial (IA) é o processo de simplificar a representação de um problema, removendo detalhes irrelevantes para a tarefa em questão (Russell; Norvig, 2021). Isso permite que os agentes de IA se concentrem nos aspectos essenciais do problema e encontrem soluções mais eficientes. Em outras palavras, a abstração é fundamental em IA, especialmente ao lidar com problemas do mundo real, como navegação de robôs e dirigir um táxi. Esses problemas envolvem espaços de estados e ações contínuos, com infinitas possibilidades. A abstração ajuda a gerenciar essa complexidade, tornando o problema computável.

A **abstração** é válida se conseguirmos expandir qualquer solução abstrata para uma solução detalhada no mundo real. Por exemplo, recomendar “Filme A”, depois, “Filme B” e, finalmente, “Filme C” deve funcionar tanto no modelo abstrato quanto no detalhado. A abstração deve garantir que as ações sejam simples o suficiente para serem executadas sem a necessidade de planejamento adicional.

Ao formularmos problemas em IA, especialmente em sistemas de recomendação, criamos uma representação simplificada do mundo real, removendo detalhes desnecessários. Isso permite que as nossas soluções sejam aplicadas de forma eficiente. Sem a capacidade de abstrair, seria impossível lidar com a complexidade do mundo real nos sistemas de IA.



INDICAÇÃO DE FILME

Ex_Machina

O filme, embora não seja sobre Inteligência Artificial (IA), aborda a vida do matemático John Nash, cuja teoria de jogos é fundamental para a modelagem de problemas em IA. Ele é reconhecido pelas contribuições à teoria dos jogos, especialmente o Equilíbrio de Nash, que oferece uma estrutura matemática essencial para a análise de interações estratégicas em IA. Essa teoria é utilizada em áreas, como aprendizado de máquina e sistemas multiagentes, nas quais a definição correta de problemas é crucial para o sucesso dos algoritmos de IA.



A resolução de problemas em IA tem sido aplicada a uma vasta gama de ambientes de tarefas. Vamos explorar alguns dos problemas mais conhecidos, distinguindo entre problemas simplificados e do mundo real (Russell; Norvig, 2021). Um problema simplificado é projetado para ilustrar ou exercitar vários métodos de resolução de problemas. Ele pode ser descrito de forma concisa e exata, sendo adequado como referência para pesquisadores compararem o desempenho de algoritmos.

Já um problema do mundo real, como a navegação de robôs, é aquele cujas soluções são realmente usadas pelas pessoas e cuja formulação é idiossincrática, não padronizada. Isso acontece, pois, por exemplo, cada robô tem sensores diferentes, os quais produzem dados diferentes. Para ilustrarmos as diferenças entre um problema padronizado e um problema do mundo real, usaremos um exemplo de navegação.

**MUNDO SIMPLIFICADO:
NAVEGAÇÃO DO ASPIRADOR DE PÓ****Estado Inicial:**

- O aspirador está em uma posição inicial em uma grade 2x2.
- Posições das sujeiras são fixas.

Ações Possíveis:

- Esquerda, Direita, Cima, Baixo, Aspirar.

Modelo de Transição:

- O aspirador se move para a célula adjacente ou aspira a sujeira na célula atual.

Teste de Objetivo:

- Todas as células estão limpas.

Função de Custo de Caminho:

- Cada movimento ou ação custa 1 unidade.

Exemplo:

- O aspirador começa na célula (1,1) e deve limpar todas as células. Se a célula (1,2) estiver suja, o aspirador se move para (1,2) e a limpa. Repete esse processo para todas as células.

**MUNDO REAL:
NAVEGAÇÃO DE UM ROBÔ EM UM
ARMAZÉM****Estado Inicial:**

- Posição inicial do robô em um armazém.
- Layout dinâmico com prateleiras móveis, pessoas e outros robôs.

Ações Possíveis:

- Movimentos complexos como avançar, retroceder, girar, levantar objetos, manipular braços e pernas.

Modelo de Transição:

- O robô se move para uma nova posição, ajustando componentes como braços e pernas, levando em conta a precisão dos sensores e a possibilidade de colisões.

Teste de Objetivo:

- Chegar à posição alvo específica ou completar uma tarefa como pegar um item e entregá-lo em um local designado.

Função de Custo de Caminho:

- Pode incluir tempo de movimento, energia consumida, precisão das ações, potencial desgaste do robô.

Exemplo:

- O robô começa na entrada do armazém e precisa pegar um item na prateleira B3 e entregar na estação de embalagem. Ele deve evitar colisões com pessoas e outros robôs, recalcular rotas se prateleiras forem movidas, e ajustar sua trajetória conforme novos obstáculos aparecem.

Observe, nos Quadros 1 e 2, as diferenças entre problemas simplificados e problemas do mundo real com base nos cinco componentes essenciais de um problema de IA:

COMPONENTE	PROBLEMAS PADRONIZADOS	PROBLEMAS DO MUNDO REAL
ESTADO INICIAL	Localização do agente e objetos fixos e conhecidos.	Localização inicial variável, layout e condições dinâmicas.
AÇÕES POSSÍVEIS	Movimentos básicos e ações específicas.	Movimentos complexos e variados, controle de múltiplos fatores.
MODELO DE TRANSIÇÃO	Simples e previsível, com poucas variáveis.	Complexo, com múltiplas variáveis e influências externas.
TESTE DE OBJETIVO	Metas fixas e bem definidas.	Metas variáveis e complexas, dependentes de múltiplos fatores.
FUNÇÃO DE CUSTO DE CAMINHO	Custo fixo e simples (como custo de movimento = 1).	Custo variável, incluindo múltiplos fatores (como tempo, energia e condições).

Quadro 1 - Diferenças entre problemas padronizados e problemas do mundo real
Fonte: adaptado de Russell e Norvig (2021).

ASPECTO	PROBLEMAS PADRONIZADOS	PROBLEMAS DO MUNDO REAL
COMPLEXIDADE E VARIABILIDADE	Simples e fixos, configuração estática e bem definida.	Complexos e dinâmicos, com muitas variáveis e incertezas.
APLICABILIDADE	<i>Benchmarks</i> para testar algoritmos em condições controladas.	Aplicações práticas, exigindo adaptações contínuas às condições reais.
DETALHAMENTO E ABSTRAÇÃO	Abstrações simplificadas, focando em elementos essenciais.	Considerações detalhadas, formuladas para lidar com variabilidade e incerteza.

Quadro 2 - Aspectos e aplicabilidade dos problemas padronizados e problemas do mundo real
Fonte: adaptado de Russell e Norvig (2021).

Agora que compreendemos como a formulação de problemas varia entre mundos simplificados e reais, é fundamental explorarmos a modelagem matemática desses problemas. A modelagem matemática fornece uma base sólida para representar problemas de IA, permitindo a aplicação de métodos analíticos e computacionais para encontrar soluções ótimas.

Vamos, então, aprofundar o nosso entendimento sobre a modelagem matemática de problemas na IA, explorando as técnicas e as ferramentas que nos ajudam a traduzir problemas do mundo real em modelos matemáticos precisos e manejáveis.

MODELAGEM MATEMÁTICA DE PROBLEMAS NA IA

A modelagem matemática é essencial para a formulação de problemas na IA. Isso envolve a representação de problemas de maneira que os algoritmos possam operá-los. Ferramentas, como grafos e diagramas de estados, são extremamente úteis para representar problemas de forma estruturada e eficiente.



Definição de variáveis

Na modelagem matemática de problemas na IA, as variáveis representam os diferentes elementos que compõem o problema. Por exemplo, no sistema de recomendação de filmes, as variáveis podem incluir:

- **Filmes:** cada filme está disponível na base de dados.
- **Usuários:** cada usuário que interage com o sistema.
- **Preferências:** as preferências dos usuários, como gêneros favoritos ou diretores preferidos.
- **Ações:** as possíveis ações que o sistema pode tomar, como recomendar um filme específico.

Essas variáveis são usadas para descrever o estado atual do sistema e as possíveis transições entre estados.

Funções de custo

A função de custo é usada para avaliar a qualidade das soluções possíveis. Ela atribui um valor numérico a cada solução possível, permitindo comparar e escolher a melhor opção (Luger, 2005). No exemplo relativo à recomendação de filmes, a função de custo pode medir a satisfação do usuário com as recomendações recebidas.

- **Definição:** $C(s,a)$, em que s é o estado atual, e a é a ação tomada. O custo pode ser calculado com base em fatores, como a precisão das recomendações, o tempo de processamento ou a relevância das sugestões para o usuário (Luger, 2005).
- **Minimização:** o objetivo é minimizar o custo total, encontrando a sequência de ações que resulta na menor função de custo possível, garantindo a satisfação do usuário (Luger, 2005).
- **Exemplo:** o custo é calculado com base em diferentes fatores, como a precisão das recomendações (P), o tempo de processamento (T) e a relevância das sugestões (R). Cada fator contribui de forma acumulativa ao custo total: $C(s,a) = (1 - P) + T + (1 - R)$. Vamos calcular o custo total com base em uma recomendação específica:

- **Precisão (P):** o modelo acertou 90% das recomendações ($P=0.9$). Logo, o custo da precisão é $1-0.9=0.1$.
- **Tempo de processamento (T):** a recomendação foi gerada em 0.2 segundos ($T=0.2$).
- **Relevância (R):** o filme recomendado foi 80% relevante para os interesses do usuário ($R=0.8$). Logo, o custo da relevância é $1-0.8=0.2$.

Substituindo os valores na fórmula, temos:

$$C(s,a) = (1 - P) + T + (1 - R) = 0.1 + 0.2 + 0.2 = 0.5$$

Restrições

As restrições definem as condições que as soluções devem satisfazer. Elas são essenciais para garantir que as soluções sejam viáveis e realistas.

- **Exemplos:** no sistema de recomendação de filmes, uma restrição pode exigir que um filme não seja recomendado mais de uma vez ao mesmo usuário. Outra restrição pode ser que apenas filmes com certa avaliação mínima sejam recomendados.
- **Formulação:** restrições são expressas matematicamente e devem ser satisfeitas por todas as soluções viáveis. Por exemplo, $R(x) \leq b$, em que R é uma função de restrição e b é um limite que não deve ser excedido.

Função heurística

A função heurística é uma estimativa do custo necessário para alcançar o objetivo a partir de um estado dado. Ela é utilizada para guiar a busca por soluções, tornando o processo mais eficiente (Russell; Norvig, 2021).

- **Definição:** $h(n)$, em que n é o estado atual. A função heurística estima o custo de alcançar o estado objetivo a partir de n .
- **Utilização:** em algoritmos, como A^* , a função heurística $h(n)$ é combinada com a função de custo real $g(n)$ para formar a função de avaliação $f(n)$, que guia a busca pelo melhor caminho:

$$f(n)=h(n)+g(n)$$

A heurística deve ser admissível, ou seja, nunca deve superestimar o custo real para garantir que o algoritmo encontre a solução ótima.

- **Exemplo:** imagine que você está implementando um sistema para encontrar o caminho mais curto em um mapa entre duas cidades, A (origem) e D (destino). O custo real $g(n)$ entre as cidades é baseado na distância percorrida até o momento, e a heurística $h(n)$ é baseada na distância em linha reta até o destino.

Quadro com os custos entre as cidades:

CIDADE ATUAL (N)	G(N) (CUSTO REAL ATÉ N)	H(N)(DISTÂNCIA EM LINHA RETA ATÉ D)	F(N)=G(N)+H(N)
A	0	50	50
B	10	30	40
C	15	20	35
D(Objetivo)	25	0	25

Quadro 3 - Custos de distância entre cidades / Fonte: o autor.

Começamos na cidade A, onde o custo acumulado $g(A)$ é 0 e a estimativa heurística para alcançar o destino $h(A)$ é 50, resultando em $f(A)=50$. Avaliamos as conexões de A e calculamos $f(n)$ para cada cidade conectada. A cidade C apresenta o menor $f(C)=35$, sendo escolhida como o próximo nó a ser expandido. A partir de C, repetimos o processo avaliando as conexões, até alcançar o destino D, onde o custo total acumulado é $g(D)=25$. Utilizando a função heurística, o algoritmo prioriza caminhos mais promissores, garantindo que o custo final encontrado seja o menor possível.

Grafos e espaço de estados

Um grafo é uma estrutura matemática que representa o espaço de estados de um problema. Cada nó (vértice) representa um estado, e cada aresta (arco) representa uma ação que leva de um estado a outro (Luger, 2005).

- **Nós (Vértices):** representam os estados possíveis do problema. No sistema de recomendação de filmes, cada nó pode representar um conjunto específico de preferências do usuário.
- **Arestas (Arcos):** representam as ações que podem ser tomadas para mover de um estado para outro. No exemplo do sistema de recomendação de filmes, a ação de recomendar um filme altera o estado das preferências do usuário.

Grafos são úteis para visualizar todas as possíveis configurações de um problema e as transições entre essas configurações. Eles fornecem uma maneira clara de mapear o espaço de estados e analisar os caminhos possíveis para alcançar o objetivo.

Representação matemática:

Um grafo G é definido como:

$$G=(V,E)$$

Em que:

- V é o conjunto de vértices (estados).
- E é o conjunto de arestas (transições), representado por pares ordenados de vértices $(u,v) \in V$.

EXEMPLIFICANDO

Exemplo prático

Imagine um sistema de recomendação de filmes em que o estado inicial (S_0) é o conjunto de preferências iniciais do usuário. Cada recomendação altera o estado, criando uma transição representada por uma aresta.

Configuração do grafo:

- **Nós (V):** estados do usuário após cada recomendação ($S_0, S_1, S_2, \dots$).
- **Arestas (E):** recomendações específicas ($F_1, F_2, \dots$).

Por exemplo:

- S_0 : preferências iniciais do usuário (gênero A e B).
- F_1 : recomenda o filme X.
- S_1 : preferências ajustadas para incluir gênero C.

Representação matemática do grafo:

- $V = \{S_0, S_1, S_2, S_3\}$
- $E = \{(S_0, S_1), (S_1, S_2), (S_2, S_3)\}$

Nesse exemplo, o grafo mostra como cada recomendação altera o estado e permite analisar a sequência de ações que leva ao objetivo final, tais como maximizar a satisfação do usuário. Essa estrutura facilita a aplicação de algoritmos de busca para encontrar as melhores recomendações.

Diagramas de estados

Os diagramas de estados são representações visuais do espaço de estados. Eles ajudam a visualizar o modo como os estados estão conectados e quais ações levam de um estado a outro (Harel, 1987).

- **Visualização de transições:** mostra como as ações (recomendações) afetam o estado das preferências do usuário. Cada transição entre estados pode ser representada por uma seta direcionada que indica a ação tomada.

- **Rastreamento de caminhos:** permite rastrear os caminhos que levam ao estado objetivo. No exemplo do sistema de recomendação, isso pode ajudar a identificar quais sequências de recomendações são mais eficazes para satisfazer o usuário.

A modelagem matemática permite a representação de problemas complexos de forma estruturada e lógica, facilitando a aplicação de algoritmos de IA para encontrar soluções eficientes. Ao concluirmos a nossa exploração sobre a formulação de problemas na IA, estamos prontos para avançar para a próxima etapa essencial: a implementação prática de algoritmos.

IMPLEMENTAÇÃO DE ALGORITMOS BÁSICOS PARA A RESOLUÇÃO DE PROBLEMAS

Chegamos ao momento prático de nossa jornada pela Inteligência Artificial: a implementação de algoritmos básicos usando Python. Vamos nos concentrar na criação de um sistema simples de recomendação de filmes, um exemplo clássico de aplicação de algoritmos de IA para resolver problemas reais. Contudo, antes, vamos conhecer o passo a passo para a implementação de algoritmos básicos em Python. Também abordaremos algumas bibliotecas essenciais para a resolução de problemas em Python.

- Passos para a implementação de algoritmos básicos:

1. DEFINIÇÃO DO PROBLEMA:

- Identifique claramente o problema que você deseja resolver.
- Defina os objetivos e os critérios de sucesso.

2. COLETA DE DADOS:

- Reúna dados relevantes que serão utilizados para resolver o problema.
- Certifique-se de que os dados são de boa qualidade e relevantes para o problema.

3. PRÉ-PROCESSAMENTO DE DADOS:

- Limpe e prepare os dados para análise.
- Trate valores ausentes, normalize os dados e realize a codificação necessária.

4. ESCOLHA DO ALGORITMO:

- Selecione um algoritmo apropriado para resolver o problema.
- Considere a simplicidade, a eficiência e a adequação ao problema específico.

5. IMPLEMENTAÇÃO DO ALGORITMO:

- Codifique o algoritmo em Python usando bibliotecas apropriadas.
- Garanta que o código seja claro, modular e reutilizável.

6. AVALIAÇÃO DO ALGORITMO:

- Avalie a performance do algoritmo utilizando métricas apropriadas.
- Realize testes e ajustes para melhorar a precisão e a eficiência.

7. GERAÇÃO DE SOLUÇÕES:

- Utilize o algoritmo treinado para gerar soluções ou recomendações.
- Verifique se as soluções geradas atendem aos critérios de sucesso definidos.

8. ITERAÇÃO E MELHORIA:

- Itere sobre o processo, refinando o algoritmo e a abordagem com base no feedback e nos resultados obtidos.

Agora, vamos implementar um sistema simples de recomendação de filmes seguindo os passos mencionados. Para que você possa seguir os passos mencionados e

implementar o sistema de recomendação de filmes, vou fornecer exemplos de dados CSV para os arquivos **movies.csv** e **ratings.csv**. Você pode usar esses exemplos para testar o código em seu ambiente. Observe uma amostra nos Quadros 4 e 5.

MOVIEID	TITLE	GENRES
1	Toy Story (1995)	Adventure
2	Jumanji (1995)	Adventure
3	Grumpier Old Men (1995)	Comedy
4	Waiting to Exhale (1995)	Comedy
5	Father of the Bride Part II (1995)	Comedy
6	Heat (1995)	Action
7	Sabrina (1995)	Comedy
8	Tom and Huck (1995)	Adventure
9	Sudden Death (1995)	Action
10	GoldenEye (1995)	Action

Quadro 4 - Amostra de dados do arquivo movies.csv / Fonte: <https://www.kaggle.com/datasets/ashukr/movie-rating-data?select=ratings.csv>. Acesso em: 5 fev. 2025.

USERID	MOVIEID	RATING	TIMESTAMP
1	1	4.0	964982703
1	3	4.0	964981247
1	6	4.0	964982224
1	47	5.0	964983815
1	50	5.0	964982931
2	1	3.0	964982400
2	2	3.0	964982400
2	3	1.0	964982400
2	4	2.0	964982400
2	5	3.0	964982400
3	1	4.0	964982900
3	7	5.0	964982900
3	8	3.0	964982900
3	9	3.5	964982900
3	10	4.0	964982900

Quadro 5 - Amostra de dados do arquivo ratings.csv / Fonte: <https://www.kaggle.com/code/muskan9698/movie-recommendation>. Acesso em: 5 fev. 2025.

Certifique-se de ter os arquivos `movies.csv` e `ratings.csv` no mesmo diretório onde está rodando o script Python.

Passos para a implementação de algoritmos básicos

1. Definição do problema

A definição clara e precisa do problema é o primeiro passo para o desenvolvimento de um algoritmo eficaz. Segundo Russell e Norvig (2021), a formalização do problema envolve a especificação dos estados iniciais, das ações possíveis e do objetivo a ser alcançado.

- **Objetivo:** recomendar filmes aos usuários com base nas avaliações anteriores.
- **Critérios de sucesso:** precisão e relevância das recomendações.

2. Coleta de dados

Para iniciar o trabalho com os dados do sistema de recomendação, é necessário carregar as informações dos filmes e das avaliações dos usuários. A biblioteca `pandas` foi utilizada para manipular os dados em formato de tabelas (`DataFrames`). Primeiramente, os dados dos filmes foram carregados a partir do arquivo `movies.csv` utilizando o método `pd.read_csv`. Em seguida, as avaliações dos usuários foram carregadas a partir do arquivo `ratings.csv` usando o mesmo método.

Após carregar os dados, as primeiras linhas de cada conjunto foram exibidas com o método `head()` para uma visualização inicial. Esse procedimento é útil para garantir que os arquivos foram carregados corretamente e obter uma visão geral da estrutura dos dados.

```
import pandas as pd
```

```
movies = pd.read_csv('movies.csv')
```

```
ratings = pd.read_csv('ratings.csv')
```

```
print(movies.head())
```

```
print(ratings.head())
```

3. Pré-processamento de dados

Antes de prosseguir com a análise dos dados, é essencial verificar se há valores ausentes nos conjuntos de dados, pois a presença de dados incompletos pode impactar os resultados. Para isso, foi usado o método *isnull()* em combinação com *sum()*, que retorna a contagem de valores ausentes em cada coluna dos DataFrames de avaliações (ratings) e filmes (movies). Caso valores ausentes sejam encontrados, eles serão tratados utilizando o método *dropna()*, que remove linhas com dados incompletos. Esse tratamento é importante para garantir a consistência e a integridade dos dados durante o processamento.

```
# Verificar valores ausentes
```

```
print(ratings.isnull().sum())
```

```
print(movies.isnull().sum())
```

```
# Tratar valores ausentes, se necessário
```

```
ratings = ratings.dropna()
```

```
movies = movies.dropna()
```

```
# Exibir tipos de dados
```

```
print(ratings.dtypes)
```

```
print(movies.dtypes)
```

4. Escolha do algoritmo

Selecione um algoritmo apropriado para o problema. Nesse caso, usaremos a média das avaliações dos filmes.

5. Implementação do algoritmo

Para calcular a média das avaliações atribuídas a cada filme, utilizamos a biblioteca *pandas*, que oferece ferramentas eficientes para a manipulação e a análise de dados tabulares. A média foi obtida agrupando as avaliações pelo identificador único de cada filme (*movieId*) e, em seguida, aplicando a função de cálculo da média aos valores de avaliação (*rating*) dentro de cada grupo. Esse processo é realizado com o método *groupby* combinado com *.mean()*. Após o cálculo, as médias das avaliações foram exibidas utilizando o método *head()*, que retorna as primeiras linhas do conjunto resultante, permitindo uma rápida verificação dos valores calculados.

```
# Calcular a média das avaliações de cada filme

average_ratings = ratings.groupby('movieId')
['rating'].mean()

# Exibir as médias das avaliações

print(average_ratings.head()) # Exibe as médias das
avaliações dos primeiros filmes
```

6. Avaliação do algoritmo

Para analisar a quantidade de avaliações recebidas por cada filme, utilizamos o método *groupby* da biblioteca *pandas*, agrupando os dados pelo identificador único dos filmes (*movieId*). Em seguida, aplicamos a função *.count()* à coluna de avaliações (*rating*), o que nos fornece o número total de avaliações para cada filme.

```
# Verificar número de avaliações por filme

num_ratings = ratings.groupby('movieId')['rating'].
count()
```

```
# Exibir as primeiras linhas
```

```
print(num_ratings.head())
```

7. Geração de soluções

Para recomendar filmes personalizados a um usuário específico, foi desenvolvida uma função chamada *recommend_movies*. Inicialmente, a função identifica os filmes já avaliados pelo usuário, filtrando o conjunto de avaliações (*ratings*) com base no identificador do usuário (*userId*). Os filmes avaliados são extraídos como uma lista, armazenada em *watched_movies*. Em seguida, o sistema recomenda filmes que o usuário ainda não assistiu. Para isso, utiliza-se o conjunto de médias de avaliações previamente calculado (*average_ratings*) e remove-se qualquer filme presente na lista *watched_movies*. Os filmes restantes são ordenados em ordem decrescente com base nas médias de avaliação, e os primeiros *nnn* filmes, determinados pelo parâmetro *n_recommendations*, são selecionados com o método *nlargest*. Por fim, a função retorna os detalhes dos filmes recomendados, ao buscar, no conjunto de dados de filmes (*movies*), os registros correspondentes aos identificadores selecionados. A função foi utilizada para recomendar cinco filmes ao usuário com identificador 1, e os resultados foram exibidos para validação.

```
def recommend_movies(user_id, n_recommendations=5):
```

```
# Filmes que o usuário já avaliou
```

```
user_ratings = ratings[ratings['userId'] == user_id]
```

```
watched_movies = user_ratings['movieId'].tolist()
```

```
# Recomendar filmes que o usuário não assistiu ainda, ordenados pela média de avaliação
```

```
recommendations = average_ratings[~average_ratings.index.isin(watched_movies)]
```

```
recommended_movies = recommendations.nlargest(n_
recommendations).index.tolist()
```

```
return movies[movies['movieId'].isin(recommended_
movies)]
```

```
# Recomendar filmes para um usuário específico
```

```
recommended_movies = recommend_movies(user_id=1, n_
recommendations=5)
```

```
print("Filmes recomendados:")
```

```
print(recommended_movies)
```

8. Iteração e melhoria

Itere sobre o processo, refinando o algoritmo e a abordagem com base no feedback e nos resultados obtidos. A função *recommend_movies_refined* aprimora o processo de recomendação, ao incluir critérios de popularidade, garantindo que os filmes recomendados tenham um número mínimo de avaliações confiáveis. Inicialmente, ela identifica os filmes já avaliados pelo usuário e calcula a lista de filmes não assistidos. Em seguida, filtra os filmes populares com base em um número mínimo de avaliações (por padrão, 50). Caso o número de filmes populares seja insuficiente para as recomendações, o critério é relaxado, retornando os filmes mais bem avaliados. A função exibe os resultados e retorna os detalhes dos filmes recomendados, oferecendo recomendações mais confiáveis e relevantes.

```
def recommend_movies_refined(user_id, n_recommendations=5, min_num_ratings=50):
```

```
# Filmes que o usuário já avaliou
```

```
user_ratings = ratings[ratings['userId'] == user_id]
```

```

watched_movies = user_ratings['movieId'].tolist()

# Filmes não assistidos

unwatched_movies = average_ratings[~average_ratings.index.isin(watched_movies)]

print(f"Total de filmes não assistidos: {len(unwatched_movies)}") # Debug

# Considerar apenas filmes com um número mínimo de avaliações

popular_movies = num_ratings[num_ratings > min_num_ratings].index.tolist()

unwatched_popular_movies = unwatched_movies[unwatched_movies.index.isin(popular_movies)]

print(f"Total de filmes populares não assistidos: {len(unwatched_popular_movies)}") # Debug

# Se não houver filmes populares suficientes, relaxar o critério de popularidade

if len(unwatched_popular_movies) < n_recommendations:

    recommended_movies = unwatched_movies.sort_values(ascending=False).head(n_recommendations).index.tolist()

else:

    recommended_movies = unwatched_popular_movies.

```

```
sort_values(ascending=False).head(n_recommendations).
index.tolist()
```

```
print(f"Filmes recomendados IDs: {recommended_
movies}") # Debug
```

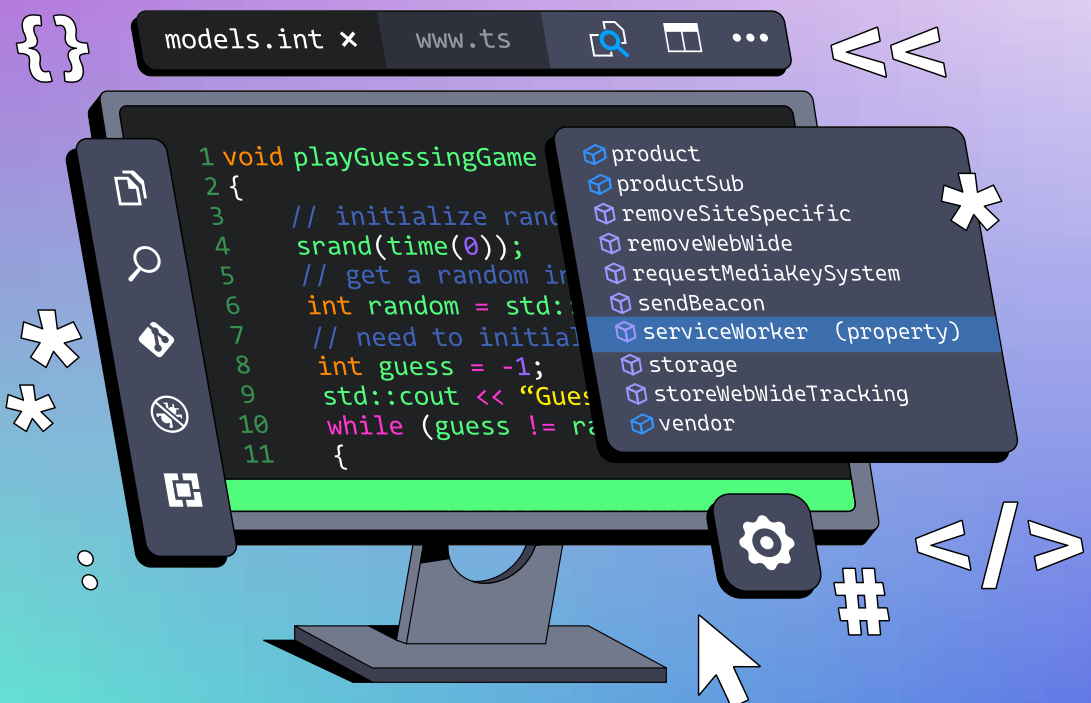
```
return movies[movies['movieId'].isin(recommended_
movies)]
```

```
# Recomendar filmes refinados para um usuário específico
```

```
recommended_movies_refined = recommend_movies_
refined(user_id=1, n_recommendations=5)
```

```
print("Filmes recomendados (refinados):")
```

```
print(recommended_movies_refined)
```



**APROFUNDANDO****Explicação do código**

1. Importação de bibliotecas: importamos a biblioteca pandas para manipulação de dados.
2. Carregamento de dados: carregamos os arquivos CSV movies.csv e ratings.csv.
3. Visualização e limpeza de dados: verificamos e limpamos valores ausentes, se necessário.
4. Cálculo de médias e contagens: calculamos a média das avaliações e a contagem de avaliações por filme.
5. Função de recomendação: implementamos funções para recomendar filmes com base nas avaliações.
6. Teste da função: testamos as funções de recomendação com um usuário específico.

Com esses exemplos de arquivos CSV e o código completo, você terá um sistema de recomendação de filmes funcionando em Python.

Neste tema de aprendizagem, exploramos a teoria e a prática da formulação de problemas na Inteligência Artificial. Aprendemos a definir problemas, identificar o espaço de estados e estabelecer objetivos claros. Compreendemos a importância da modelagem matemática para representar problemas de forma estruturada e eficiente utilizando ferramentas, como grafos e diagramas de estados.

Além disso, abordamos a implementação prática de algoritmos em Python, destacando a criação de um sistema de recomendação de filmes como exemplo.

**EM FOCO**

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Agora que exploramos como problemas são formulados na IA, modelamos matematicamente esses problemas e implementamos algoritmos básicos para resolução, é hora de conectar esses conhecimentos ao ambiente profissional e entender as respectivas aplicações práticas. Esta seção visa preparar você para aplicar o conhecimento adquirido em situações reais e no mercado de trabalho.

A formulação de problemas na IA é um passo essencial que influencia diretamente a eficácia das soluções desenvolvidas. Vamos explorar como esses conceitos são aplicados em diferentes setores e como eles podem transformar seu campo de atuação.

No setor da saúde, diagnosticar doenças com maior precisão utilizando IA envolve definir os sintomas relatados e o histórico médico como estado inicial, proporcionar possíveis diagnósticos como espaço de estados, e fornecer o diagnóstico correto como objetivo. Algoritmos analisam dados de pacientes, resultando em diagnósticos mais rápidos e eficientes, melhorando tratamentos e salvando vidas.

Empresas que utilizam IA para **logística e transporte** conseguem otimizar rotas de entrega para reduzir custos. Aumentar a eficiência com IA envolve definir a localização dos veículos como estado inicial, todas as possíveis rotas como espaço de estados, e a rota mais eficiente como objetivo. A modelagem matemática usa grafos e algoritmos de busca para encontrar a solução ótima, economizando tempo e recursos e melhorando a satisfação do cliente.

No comércio e varejo, é utilizada para aumentar vendas mediante recomendações personalizadas com IA, que envolve definir o histórico de compras do cliente como estado inicial, todos os produtos disponíveis como espaço de estados, e produtos que o cliente provavelmente comprará como objetivo. Algoritmos de recomendação analisam dados para gerar recomendações personalizadas, aumentando vendas e melhorando a experiência do cliente.

Por fim, um último exemplo de aplicação está na área de segurança cibernética, em que detectar e prevenir ataques cibernéticos usando IA envolve identificar padrões anômalos em dados de rede. A formulação do problema inclui definir atividade de rede normal como estado inicial, possíveis anomalias como espaço de estados, e a detecção de ataques como objetivo, protegendo informações sensíveis e mantendo a segurança dos dados em empresas e instituições.

Como a formulação de problemas na IA pode impactar a sua área de atuação? Pense nas aplicações práticas desses conceitos no seu campo de trabalho. Reflita sobre como você pode usar o conhecimento adquirido para melhorar processos e resolver problemas complexos de maneira mais eficiente. Considere também as implicações éticas e a importância de desenvolver soluções justas e transparentes.

AUTOATIVIDADE

1. Na IA, a formulação de problemas envolve a identificação de vários elementos principais. Esses elementos incluem: o estado inicial, que descreve a condição inicial do problema; o espaço de estados, que abrange todas as possíveis configurações do problema; e o estado objetivo, que define a condição desejada ou a meta a ser alcançada. A formulação adequada desses elementos é crucial para o desenvolvimento de algoritmos eficientes que possam encontrar soluções ótimas.

Qual das alternativas descreve corretamente um exemplo de estado inicial em um sistema de recomendação de filmes?

- a) A lista de filmes recomendados ao usuário.
 - b) As ações possíveis que o sistema pode tomar.
 - c) Os dados sobre filmes, como gênero, diretor, atores principais e avaliações.
 - d) A função de custo, que minimiza a insatisfação do usuário.
 - e) O objetivo de satisfazer o usuário com as recomendações.
2. As restrições são condições que as soluções devem satisfazer em um problema de IA. Elas garantem que as soluções propostas sejam viáveis e estejam dentro dos limites aceitáveis para o problema em questão. Restrições podem incluir requisitos específicos, tais como não repetir recomendações ou manter certo padrão de qualidade. Em um sistema de recomendação, por exemplo, as restrições ajudam a assegurar que as recomendações sejam relevantes e diversas.

Considerando as possíveis restrições em um sistema de recomendação de filmes, analise as afirmativas a seguir:

- I - Um filme não deve ser recomendado mais de uma vez ao mesmo usuário.
- II - Apenas filmes com certa avaliação mínima devem ser recomendados.
- III - Recomendar o maior número possível de filmes.
- IV - Analisar os dados de todos os usuários em tempo real.

É correto o que se afirma em:

- a) I e II, apenas.
- b) II e III, apenas.
- c) I e III, apenas.
- d) I, II e IV, apenas.
- e) I, II, III e IV.

3. Python é uma das linguagens mais populares para implementar algoritmos de IA devido à simplicidade e à vasta quantidade de bibliotecas disponíveis. Ferramentas, tais como NumPy e pandas, facilitam o desenvolvimento de algoritmos complexos, proporcionando funções e métodos prontos para uso que aceleram o processo de desenvolvimento. Além disso, Python possui uma comunidade ativa que contribui para a constante evolução e melhoria das bibliotecas e frameworks voltados para IA.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - Python é amplamente utilizada para a implementação de algoritmos de IA.

PORQUE

II - Python oferece bibliotecas e ferramentas poderosas, como NumPy e pandas, que facilitam o desenvolvimento de algoritmos de IA.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

ARISTÓTELES. **Metafísica**. Lisboa: Edições 70, 2021.

GUICCIARDINI, N. **The Development of Newtonian Calculus in Britain, 1700-1800**. Cambridge: Cambridge University Press, 2003.

HAREL, D. Statecharts: A Visual Formalism for Complex Systems. **Science of Computer Programming**, v. 8, n. 3, p. 231-274, 1987.

KOETSIER, T. **The Ascent of GIM, the Global Intelligent Machine**. [S. l.]: Springer, 2019.

LUGER, G. F. **Artificial Intelligence**: Structures and Strategies for Complex Problem Solving. [S. l.]: Pearson, 2005.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence**: A Modern Approach. [S. l.]: Pearson, 2021.

1. Opção C.

A lista de filmes recomendados ao usuário seria uma parte da solução, e não do estado inicial. As ações possíveis que o sistema pode tomar são parte da formulação do problema, mas não do estado inicial. O estado inicial em um sistema de recomendação de filmes inclui os dados sobre filmes, como gênero, diretor, atores principais e avaliações. A função de custo é usada para avaliar as soluções, e não para definir o estado inicial. O objetivo de satisfazer o usuário com as recomendações é uma meta, e não um estado inicial.

2. Opção A.

As restrições presentes nas afirmativas I e II são exemplos claros de condições que as soluções devem satisfazer em um sistema de recomendação de filmes. A restrição localizada na afirmativa I garante que um filme não seja recomendado repetidamente ao mesmo usuário, enquanto a restrição presente na afirmativa II assegura que apenas filmes com uma avaliação mínima sejam recomendados. Recomendar o maior número possível de filmes não é uma restrição típica, pois não necessariamente melhora a qualidade das recomendações. Analisar os dados de todos os usuários em tempo real, embora útil, não é uma restrição direta sobre as recomendações de filmes.

3. Opção A.

A Python é amplamente utilizada na IA devido à simplicidade e à disponibilidade de bibliotecas poderosas, como NumPy e pandas, que facilitam o desenvolvimento de algoritmos. A justificativa presente na asserção II é correta para a asserção I. Logo, ambas as asserções são verdadeiras, e a relação entre elas é correta.



TEMA DE APRENDIZAGEM 3

PARADIGMA SIMBÓLICO EM IA

MINHAS METAS

- Entender o paradigma simbólico na IA.
- Aprender a lógica proposicional e a lógica de predicados.
- Configurar o ambiente de desenvolvimento Python.
- Criar estruturas de dados para a manipulação de símbolos.
- Implementar expressões lógicas em Python.
- Aplicar conceitos teóricos em atividades práticas.
- Desenvolver habilidades de raciocínio lógico.

INICIE SUA JORNADA

Imagine um mundo em que máquinas não apenas executam tarefas, mas também raciocinam como seres humanos. Esse cenário, que, antes, parecia ficção científica, está se tornando realidade com a evolução da Inteligência Artificial (IA). Nesse vasto campo, a IA simbólica se destaca pela capacidade de simular o raciocínio humano mediante a manipulação de símbolos e regras lógicas. No entanto, transformar essa visão em realidade envolve desafios significativos. Como podemos criar sistemas que pensem de forma lógica e estruturada? Quais são as barreiras técnicas que devem ser superadas para que a IA simbólica atinja o potencial máximo?

A abordagem simbólica da IA, fundamentada em uma **lógica formal**, permite criar sistemas que podem realizar inferências lógicas e tomar decisões baseadas em regras definidas. Compreender esses conceitos é essencial para desenvolver aplicações que necessitam de raciocínio estruturado e preciso, como sistemas de recomendação personalizados, diagnósticos médicos e planejamento estratégico.

A lógica proposicional e a lógica de predicados são os alicerces da IA simbólica (Russell; Norvig, 2021). Elas permitem analisar dados e inferir novos conhecimentos de maneira precisa e eficiente. Em diagnósticos médicos, por exemplo, a IA simbólica pode identificar doenças, ao analisar sintomas e históricos de pacientes, oferecendo suporte valioso aos profissionais de saúde. Em sistemas de recomendação, essa abordagem pode personalizar ofertas com base nas preferências dos usuários, enquanto, no planejamento estratégico, pode prever os impactos de diferentes decisões, tornando-se uma ferramenta essencial para quem deseja estar na vanguarda tecnológica.

Suponha que você está trabalhando em um banco e precisa desenvolver um sistema que auxilie na concessão de empréstimos, avaliando automaticamente a viabilidade de novos pedidos com base em dados financeiros e históricos de crédito dos clientes. Como você aplicaria a IA simbólica para resolver esse problema? Desenvolva um pequeno estudo de caso considerando os seguintes pontos: identificação do problema, coleta de dados relevantes, aplicação das lógicas proposicional e de predicados, a fim de avaliar a viabilidade dos empréstimos, e avaliação dos resultados. Esse exercício prático permitirá visualizar como a teoria da IA simbólica pode ser aplicada em situações reais, o que te preparará para enfrentar desafios semelhantes em sua carreira profissional.

Pense em como a IA simbólica pode transformar não apenas o seu campo de atuação, mas também a sua própria carreira. Ao dominar essas técnicas, você pode se tornar um(a) líder inovador(a), capaz de implementar soluções que melhoram significativamente a eficiência e a precisão das decisões em sua área. Considere, também, o impacto social das tecnologias que você desenvolve. Como as suas habilidades em IA simbólica podem contribuir para criar sistemas mais justos e acessíveis? Reflita sobre a responsabilidade que vem com o poder de moldar o futuro mediante a tecnologia e como você pode usar esse conhecimento para promover mudanças positivas.



PLAY NO CONHECIMENTO

No episódio de nosso podcast, mergulhamos em inovações recentes no campo da Inteligência Artificial, explorando como novas técnicas e aplicações estão transformando diversos setores. Descubra como a IA está revolucionando a indústria, a educação e até mesmo o entretenimento. Acesse o podcast e fique por dentro das últimas tendências e descobertas em IA. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para garantir que você esteja preparado(a) para compreender e aplicar os conceitos do paradigma simbólico na Inteligência Artificial, vamos relembrar alguns conteúdos essenciais. Primeiro, é importante entender a lógica proposicional e a lógica de predicados, que são fundamentais para a representação simbólica do conhecimento. Essas formas de lógica permitem criar regras e inferências lógicas que são a base da IA simbólica. Além disso, familiarizar-se com a biblioteca SymPy em Python é fundamental, pois ela será usada para manipular expressões lógicas e símbolos. SymPy permite realizar cálculos simbólicos e simplificações de expressões lógicas de forma eficiente.

Revisar estruturas de dados em Python também é essencial, já que a manipulação de símbolos e expressões lógicas requer um bom entendimento de como organizar e acessar dados de maneira eficiente. <https://cutt.ly/Ze2g8gEn>

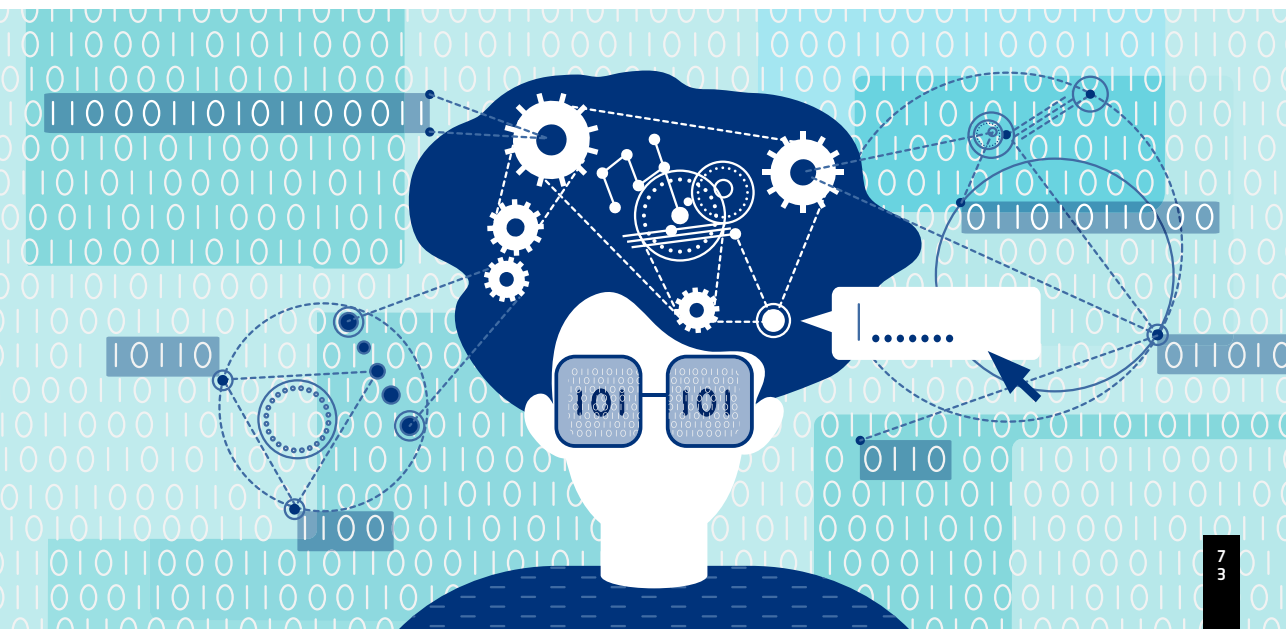
DESENVOLVA SEU POTENCIAL

INTRODUÇÃO AO PARADIGMA SIMBÓLICO NA IA

Vamos embarcar em uma jornada fascinante pelo mundo do paradigma simbólico na Inteligência Artificial (IA). Teremos uma visão abrangente e envolvente sobre como o conhecimento é representado mediante símbolos, explorando a lógica proposicional e de predicados como base para essa representação, além da respectiva implementação prática em Python. Compreender como esses conceitos são aplicados pode abrir portas para uma ampla gama de aplicações, desde diagnósticos médicos até planejamento estratégico. Preparado(a)? Então, vamos lá!

O paradigma simbólico na IA começa com a representação clara do conhecimento a partir de símbolos. Esses símbolos podem ser letras, palavras ou frases manipulados por meio de regras lógicas para inferir novos conhecimentos (Russell; Norvig, 2021). Essa abordagem se baseia na lógica formal e na teoria dos conjuntos, envolvendo a identificação de elementos principais, tais como proposições e predicados, e a aplicação de regras lógicas para manipular esses símbolos (Souza, 2008).

A lógica formal, que inclui a lógica proposicional e a lógica de predicados, é essencial para representar problemas de forma estruturada. Finalmente, a implementação prática em Python permite aplicar esses conceitos em situações reais, criando soluções eficientes e eficazes.



REPRESENTAÇÃO DO CONHECIMENTO MEDIANTE SÍMBOLOS

A representação do conhecimento em IA simbólica é baseada na utilização de símbolos que carregam significados específicos (Russell; Norvig, 2021). Esses símbolos podem ser variáveis, constantes, relações ou funções que, juntos, formam a estrutura fundamental para a manipulação lógica e a inferência de novos conhecimentos.

Estruturas simbólicas

Estruturas simbólicas são formas de representação do conhecimento que utilizam símbolos para expressar conceitos, objetos, relações e funções. Esses símbolos são abstrações que carregam significados específicos em um contexto determinado, permitindo a manipulação lógica e a inferência de novos conhecimentos.

Em **Inteligência Artificial (IA)**, as estruturas simbólicas são fundamentais para organizar e processar informações de maneira que possam ser combinadas e transformadas segundo regras lógicas predefinidas. Elas incluem: variáveis, que podem assumir diferentes valores; constantes, que representam valores fixos; relações, que descrevem associações entre símbolos; e funções, que mapeiam conjuntos de símbolos para outros conjuntos, possibilitando a realização de operações complexas (Souza, 2008).

Essas estruturas são essenciais para a clareza, a flexibilidade e a eficiência na representação e na manipulação do conhecimento em sistemas de IA (Russell; Norvig, 2021):

SÍMBOLOS

São representações abstratas de conceitos ou objetos que carregam significado dentro de um contexto específico.

VARIÁVEIS

São símbolos que podem assumir diferentes valores.

CONSTANTES

São símbolos que representam valores fixos e definidos.

Para compreender melhor como os símbolos operam na IA, é essencial entender as principais características. A seguir, apresentamos uma tabela com as principais características dos símbolos e as respectivas descrições (Russell; Norvig, 2021).

CARACTERÍSTICA	DESCRIÇÃO
ABSTRAÇÃO	Símbolos são abstratos, o que significa que podem representar algo sem ser o próprio objeto ou conceito. Por exemplo, a letra "A" pode representar um conceito, como "gato".
INTERPRETAÇÃO	O significado de um símbolo é dado pela interpretação dentro de um sistema ou contexto. A mesma letra "A" pode ter diferentes significados em diferentes contextos.
MANIPULAÇÃO	Símbolos podem ser combinados e manipulados de acordo com regras lógicas para gerar novos conhecimentos. Isso é feito usando operadores lógicos e inferências.

Tabela 1 - Principais características dos símbolos / Fonte: adaptada de Russell e Norvig (2021).

Além das características dos símbolos, é importante considerar as **relações** e as **funções** que eles podem representar. **Relações** descrevem como os símbolos estão associados uns aos outros. Por exemplo, “Gato está no tapete” representa uma relação entre o “Gato” e o “tapete”. **Funções**, por outro lado, mapeiam um conjunto de símbolos a outro conjunto de símbolos, permitindo a definição de operações sobre esses símbolos. Por exemplo, uma função pode mapear um conjunto de entradas (variáveis) para uma saída (resultado).

Existem diferentes formas de representar o conhecimento, cada uma com vantagens e aplicações específicas. As representações **numéricas**, por exemplo, são usadas para valores quantitativos e são manipuladas por operações aritméticas. Em contraste, os símbolos podem representar conceitos qualitativos e são manipulados por operações lógicas.

Representações **gráficas**, tais como gráficos e diagramas, são visuais e usadas para ilustrar relações e padrões. Símbolos podem ser usados dentro de representações gráficas, mas a manipulação deles ocorre em um nível lógico e abstrato. Observe, na Tabela 2, as vantagens e as desvantagens de cada representação.

TIPO DE REPRESENTAÇÃO	VANTAGENS	DESVANTAGENS
NUMÉRICA	Precisão e clareza em cálculos quantitativos; fácil manipulação mediante operações aritméticas.	Limitada a informações quantitativas; não adequada para representar conceitos abstratos.
SIMBÓLICA	Representação de conceitos qualitativos; permite inferência lógica e manipulação estruturada de conhecimento.	Pode ser complexa de implementar e requer sistemas lógicos sofisticados; a interpretação pode variar conforme o contexto.
GRÁFICA	Visualização intuitiva de relações e padrões; facilita a compreensão rápida de dados complexos.	A manipulação lógica pode ser limitada; depende da clareza dos gráficos e diagramas para a correta interpretação dos dados.

Tabela 2 – Tipos de representação do conhecimento / Fonte: o autor.

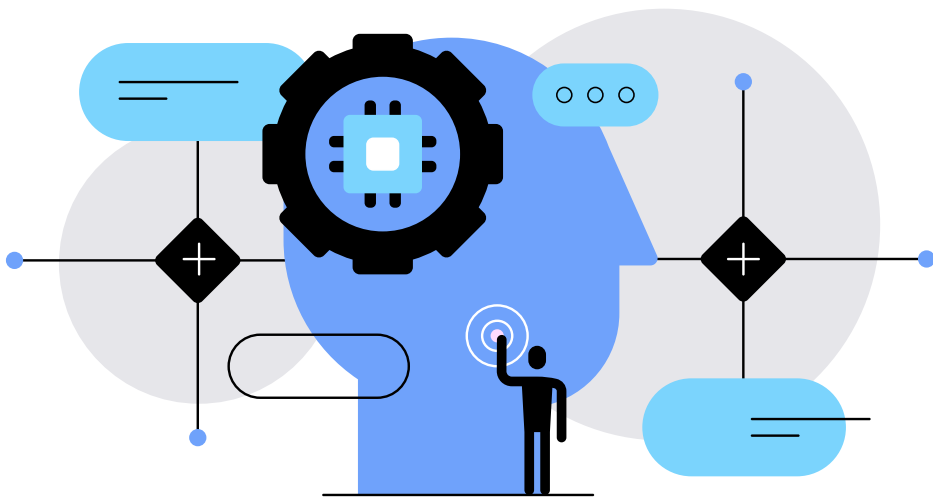
 **APROFUNDANDO**

Símbolos são amplamente utilizados em diversos campos além da Inteligência Artificial. Na matemática, símbolos, como "+", "-", "=" e "x", são usados para operações matemáticas. Esses símbolos são fundamentais para a construção de expressões e equações que descrevem relações quantitativas e operações aritméticas.

APROFUNDANDO

Em linguagem natural, palavras e frases são símbolos usados para a comunicação humana. Cada palavra é um símbolo que representa um conceito ou objeto, e a combinação de palavras forma frases que transmitem significados complexos. Em computação, variáveis e operadores são símbolos usados para representar e manipular dados. As variáveis podem armazenar diferentes tipos de dados, e os operadores definem como esses dados devem ser processados (Fais, 2020).

Os símbolos são, portanto, elementos essenciais em vários campos de estudo, proporcionando uma forma estruturada e eficiente de representar e manipular o conhecimento. Seja na resolução de problemas matemáticos, na comunicação entre pessoas ou na programação de computadores, os símbolos permitem que conceitos complexos sejam expressos de forma clara e manipulados de acordo com regras bem definidas.



Regras lógicas

As regras lógicas são essenciais para a manipulação e a inferência de novos conhecimentos a partir de símbolos. Elas determinam como os símbolos podem ser combinados e transformados para gerar conclusões e resolver problemas.

Regras de produção especificam como novos símbolos podem ser inferidos a partir de símbolos existentes. Por exemplo, se “A” e “B” são verdadeiros, então, “C” também é verdadeiro. Sistemas de inferência utilizam essas regras para derivar novos conhecimentos a partir de um conjunto de símbolos e relações existentes, como sistemas especialistas, que diagnosticam doenças com base em sintomas fornecidos (Russell; Norvig, 2021).

Por exemplo, considere um sistema de diagnóstico médico que usa regras de produção para identificar doenças. Se um paciente apresenta sintomas, como febre e tosse, e se há uma regra que afirma que, “se um paciente tem febre e tosse, então, ele pode ter gripe”, o sistema pode inferir que o paciente pode estar com gripe. Outro exemplo pode ser encontrado em sistemas de recomendação, no qual as regras de inferência podem ser usadas para sugerir produtos aos usuários com base nas preferências e nos comportamentos anteriores. Exemplo: “se um usuário comprou livros de ficção científica e fantasia, então, ele pode gostar de livros de fantasia épica”.

Essas regras de produção e sistemas de inferência são ferramentas poderosas na **IA simbólica**, permitindo a derivação de novos conhecimentos e a tomada de decisões com base em informações existentes (Souza, 2008). Além disso, a aplicação dessas regras pode ser automatizada, tornando os sistemas de IA mais eficientes e eficazes em várias tarefas, desde diagnósticos médicos até planejamento estratégico.

Compreender a representação simbólica é apenas uma parte da exploração do paradigma simbólico na IA. Para aplicar efetivamente esses conceitos, é fundamental ter uma base sólida em lógica matemática, que fornece as ferramentas necessárias para manipular e inferir novos conhecimentos a partir de símbolos. A lógica proposicional e a lógica de predicados são dois pilares dessa abordagem matemática (Russell; Norvig, 2021).

ABORDAGEM MATEMÁTICA

Para entender e aplicar o paradigma simbólico na Inteligência Artificial, é essencial dominar os fundamentos da lógica matemática. A lógica fornece as ferramentas necessárias para representar, manipular e inferir conhecimento a partir de símbolos. Nesta seção, exploraremos a lógica proposicional e a lógica de predicados, abordagens fundamentais para a manipulação simbólica e a construção de sistemas de inferência complexos.

Lógica Proposicional

A lógica proposicional é uma forma de lógica que se concentra em proposições, que são sentenças declarativas que podem ser classificadas como verdadeiras ou falsas. Diferentemente de outras formas de lógica, a lógica proposicional não se preocupa com o conteúdo interno das proposições, mas com a forma como elas podem ser combinadas e manipuladas utilizando operadores lógicos. Esse enfoque permite construir proposições mais complexas a partir de proposições simples e analisar as inter-relações e implicações lógicas (Souza, 2008).

Na lógica proposicional, proposições simples são combinadas usando operadores lógicos para formar proposições compostas. Na Tabela 3, são expostos os operadores lógicos básicos (Souza, 2008).

OPERADOR	SÍMBOLO	DESCRIÇÃO
AND (E)	$\wedge$	A conjunção de duas proposições, verdadeira apenas se ambas as proposições forem verdadeiras.
OR (OU)	$\vee$	A disjunção de duas proposições, verdadeira se, pelo menos, uma das proposições for verdadeira.
NOT (NÃO)	$\neg$	A negação de uma proposição, verdadeira se a proposição for falsa.

IMPLIES (IMPLICA)	→	A implicação entre duas proposições, verdadeira se a primeira proposição implica a segunda.
EQUIVALENT (EQUIVALENTE)	↔	A equivalência entre duas proposições, verdadeira se ambas as proposições tiverem o mesmo valor de verdade.

Tabela 3 – Operadores lógicos básicos / Fonte: adaptada de Souza (2008).

Para aplicar e entender melhor a lógica proposicional, é fundamental utilizar tabelas verdade e equivalências lógicas. Essas ferramentas permitem visualizar e analisar todas as combinações possíveis de valores verdade para proposições, facilitando a verificação de validade e a identificação de relações lógicas entre diferentes expressões (Abe; Scalzitti; Silva Filho, 2002).

Tabelas Verdade e Equivalências Lógicas

As tabelas verdade são ferramentas essenciais na lógica proposicional. Elas permitem visualizar todas as combinações possíveis de valores verdade das proposições e os resultados das operações lógicas aplicadas a elas. São utilizadas para verificar a validade de expressões lógicas e identificar equivalências entre diferentes expressões (Fais, 2020).

Para construir uma tabela verdade, siga estes passos (Souza, 2020):

1

Identifique as proposições envolvidas:
Liste todas as proposições simples (P, Q, R etc.) que serão usadas.

2

Liste todas as combinações possíveis de valores verdade:
Cada proposição pode ser verdadeira (T) ou falsa (F). Para n proposições, há 2n combinações possíveis.

3

Aplique os operadores lógicos:
Para cada combinação de valores verdade, calcule o resultado dos operadores lógicos utilizados na expressão.

4

Preencha a tabela:
Organize as informações em uma tabela, com colunas para cada proposição, operadores e resultado final.

Vamos observar um exemplo prático para elaborar uma tabela verdade. Considere duas proposições, P e Q:

- P: está chovendo.
- Q: eu vou levar um guarda-chuva.

Vamos construir a Tabela 4 verdade para as proposições P e Q usando os operadores AND, OR e NOT.

P	Q	$P \wedge Q$	$P \vee Q$	$\neg P$
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Tabela 4 - Tabela verdade com os operadores AND, OR e NOT / Fonte: o autor.

Na Tabela 4, a primeira coluna representa a proposição P (“Está chovendo”), enquanto a segunda coluna representa a proposição Q (“Eu vou levar um guarda-chuva”). As combinações de valores verdade para P e Q são listadas nas linhas seguintes. As colunas seguintes mostram os resultados das operações lógicas AND, OR e NOT aplicadas às proposições. Além disso, podemos usar a lógica proposicional para inferir novas proposições a partir das existentes. Por exemplo, a proposição “Se está chovendo, então, eu vou levar um guarda-chuva” ($P \rightarrow Q$) pode ser analisada da seguinte maneira:

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Tabela 5 – Tabela verdade de implicação / Fonte: o autor.

A implicação ($P \rightarrow Q$) é verdadeira em todas as combinações, exceto quando P é verdadeira e Q é falsa. Isso reflete a lógica de que, se a primeira proposição é verdadeira, então, a segunda também deve ser verdadeira, para que a implicação seja válida (Souza, 2008).

Lógica de Predicados

A lógica de predicados é uma extensão da lógica proposicional que inclui quantificadores e permite expressar relações mais complexas entre objetos (Souza, 2020). Enquanto a lógica proposicional lida com proposições simples que podem ser verdadeiras ou falsas, a lógica de predicados adiciona uma camada de profundidade, ao permitir a especificação das propriedades de objetos e as relações entre eles.



ZOOM NO CONHECIMENTO

Na lógica de predicados, utilizamos quantificadores para generalizar as proposições sobre conjuntos de objetos. Os dois quantificadores principais são (Souza, 2008):

- **Quantificador Universal ($\forall$):** afirma que uma proposição é verdadeira para todos os elementos de um conjunto. Por exemplo, "Todos os gatos são mamíferos" pode ser formalizado como $\forall x (\text{Gato}(x) \rightarrow \text{Mamífero}(x))$, em que x representa um elemento do conjunto de todos os gatos.
- **Quantificador Existencial ($\exists$):** afirma que uma proposição é verdadeira para, pelo menos, um elemento de um conjunto. Por exemplo, "Existe um gato que é preto" pode ser formalizado como $\exists x (\text{Gato}(x) \wedge \text{Preto}(x))$, indicando que há, pelo menos, um gato no conjunto que possui a propriedade de ser preto.

Predicados são funções que retornam valores verdade e são usados para expressar propriedades e relações entre objetos. Por exemplo:

- **Gato(x):** pode representar "x é um gato".
- **Mamífero(x):** pode representar "x é um mamífero".

Vamos construir uma Tabela 6 verdade para o exemplo "Todos os gatos são mamíferos" ($\forall x, \text{Gato}(x) \rightarrow \text{Mamífero}(x)$) e "Existe um gato que é preto" ($\exists x, \text{Gato}(x) \wedge \text{Preto}(x)$).

X	GATO(X)	MAMÍFERO(X)	PRETO(X)	$GATO(X) \rightarrow MAMÍFERO(X)$	$GATO(X) \wedge PRETO(X)$
Gato1	V	V	F	V	F
Gato2	V	V	V	V	V
Gato3	F	V	V	V	F
Gato4	V	V	F	V	F

Gato1: é um gato (V), é um mamífero (V), não é preto (F).

Gato2: é um gato (V), é um mamífero (V), é preto (V).

Gato3: não é um gato (F), é um mamífero (V), é preto (V).

Gato4: é um gato (V), é um mamífero (V), não é preto (F).

Tabela 6 – Tabela verdade para o exemplo “Todos os gatos são mamíferos” e “Existe um gato que é preto”
Fonte: o autor.

Na coluna “ $Gato(x) \rightarrow Mamífero(x)$ ”, a proposição é verdadeira sempre que $Gato(x)$ é falso ou $Mamífero(x)$ é verdadeiro. Na coluna “ $Gato(x) \wedge Preto(x)$ ”, a proposição é verdadeira apenas se ambos, $Gato(x)$ e $Preto(x)$, forem verdadeiros.

Formas Normais

Para simplificar e estruturar as expressões lógicas, muitas vezes, convertemos essas expressões em formas normais. As duas formas normais mais comuns são a Forma Normal Conjuntiva (CNF) e a Forma Normal Disjuntiva (DNF) (Souza, 2020).

Forma Normal Conjuntiva (CNF): a CNF é uma conjunção de disjunções de literais. Em outras palavras, é uma expressão que consiste em várias cláusulas (grupos de literais) conectadas por operadores AND ($\wedge$), em que cada cláusula é uma disjunção (OR, $\vee$) de literais (Souza, 2008). Por exemplo, a expressão $(P \vee Q) \wedge (\neg R \vee S)$ está em CNF. Cada cláusula $(P \vee Q)$ e $(\neg R \vee S)$ é uma disjunção de literais, e a conjunção das cláusulas forma a CNF.

EXPRESSÃO	FORMA
$P \vee Q \wedge R$	Não está em CNF
$(P \vee Q) \wedge (\neg R \vee S)$	CNF

Tabela 7 - Normalização Conjuntiva (CNF) / Fonte: adaptada de Souza (2008).

Forma Normal Disjuntiva (DNF): a DNF é uma disjunção de conjunções de literais. Em outras palavras, é uma expressão que consiste em várias cláusulas conectadas por operadores OR ($\vee$), em que cada cláusula é uma conjunção (AND, $\wedge$) de literais (Souza, 2008). Por exemplo, a expressão $(P \wedge Q) \vee (\neg R \wedge S)$ está em DNF. Cada cláusula $(P \wedge Q)$ e $(\neg R \wedge S)$ é uma conjunção de literais, e a disjunção das cláusulas forma a DNF.

EXPRESSÃO	FORMA
$P \wedge (Q \vee R)$	Não está em DNF
$(P \wedge Q) \vee (\neg R \wedge S)$	DNF

Tabela 8 - Normalização Disjuntiva (DNF) / Fonte: adaptada de Souza (2008).

Vantagens das Formas Normais:

Converter expressões em CNF ou DNF pode facilitar a resolução de problemas lógicos, permitindo a aplicação de algoritmos de inferência de forma mais eficiente (Russell; Norvig, 2021). Essas formas são particularmente úteis em sistemas de verificação automática de teoremas e em algoritmos de satisfatibilidade booleana (SAT). Além disso, a transformação para formas normais ajuda a padronizar expressões lógicas, tornando mais simples a análise e a manipulação delas.

Aplicações Práticas:

1. **Sistemas de Verificação Automática de Teoremas:** utilizam formas normais para simplificar a prova de teoremas complexos. Transformando expressões em CNF, é possível aplicar algoritmos de resolução que sistematicamente buscam inconsistências.
2. **Algoritmos de Satisfatibilidade Booleana (SAT):** SAT frequentemente transformam problemas em CNF para determinar se existe uma atribuição de verdade que satisfaça a uma fórmula booleana. A padronização em CNF permite a aplicação de técnicas de poda e busca mais eficientes.
3. **Análise de Circuitos Lógicos:** a transformação de expressões lógicas em DNF pode ser útil para simplificar e otimizar a análise e o design de circuitos lógicos digitais, facilitando a identificação de mintermos e maxtermos.

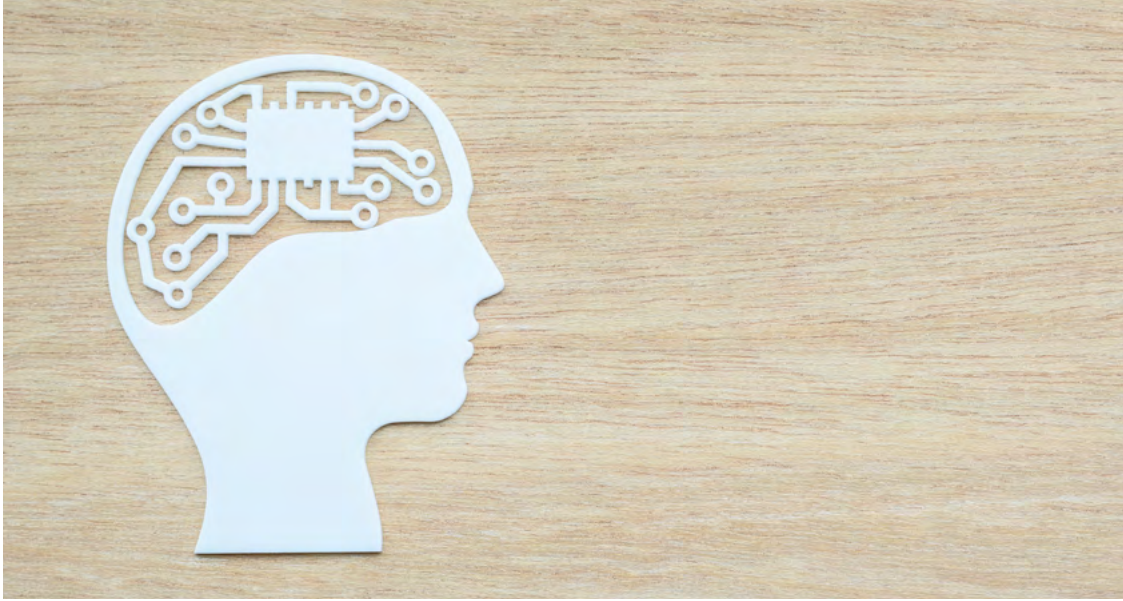
Exemplo de Conversão:

Considere a expressão lógica $(\neg P \vee Q) \wedge (R \vee \neg S)$. Para verificar se essa expressão está em CNF ou DNF, analisamos a estrutura:

- **Forma Normal Conjuntiva (CNF):** verificamos se a expressão é uma conjunção de disjunções. Nesse caso, $(\neg P \vee Q) \wedge (R \vee \neg S)$ está em CNF, porque é uma conjunção (AND) de duas disjunções (OR).
- **Forma Normal Disjuntiva (DNF):** para a mesma expressão ser DNF, ela teria que ser uma disjunção de conjunções. $(\neg P \vee Q) \wedge (R \vee \neg S)$ não está em DNF, porque as subexpressões dentro da conjunção não são conjunções de literais.

Esses exemplos mostram como a lógica de predicados pode ser utilizada para expressar e verificar propriedades e relações complexas, tornando-a uma ferramenta poderosa na IA simbólica.

Com essa base sólida em lógica proposicional e lógica de predicados, estamos prontos para explorar como essas abordagens matemáticas podem ser implementadas e utilizadas em Python para manipulação simbólica.



FERRAMENTAS E BIBLIOTECAS EM PYTHON

Para implementar e manipular expressões lógicas em Python, várias ferramentas e bibliotecas estão disponíveis (Menezes, 2019). Elas facilitam a definição, a avaliação e a manipulação de expressões lógicas, proposicionais e de predicados. Antes de explorarmos essas bibliotecas, vamos entender como essas expressões podem ser manipuladas usando Python puro, sem bibliotecas externas.

Manipulação de expressões lógicas

Em Python, podemos definir e avaliar expressões lógicas usando operadores lógicos básicos disponíveis na linguagem (Downey, 2019):

- **AND (e):** and
- **OR (ou):** or
- **NOT (não):** not
- **IMPLIES (implica):** não há operador direto, mas podemos usar not P or Q
- **EQUIVALENT (equivalente):** não há operador direto, mas podemos usar (P and Q) or (not P and not Q)

Agora, vamos definir proposições simples e combiná-las usando esses operadores lógicos.

Exemplo de Uso:

```
# Definição de proposições
```

```
P = True
```

```
Q = False
```

```
# Operadores lógicos
```

```
and_result = P and Q
```

```
or_result = P or Q
```

```
not_result = not P
```

```
implies_result = not P or Q
```

```
equivalent_result = (P and Q) or (not P and not Q)
```

```
print("P and Q:", and_result)
```

```
print("P or Q:", or_result)
```

```
print("not P:", not_result)
```

```
print("P implies Q:", implies_result)
```

```
print("P equivalent Q:", equivalent_result)
```

Lógica Proposicional em Python

Para implementar a lógica proposicional em Python, podemos criar funções para representar operadores lógicos e avaliar proposições.

Exemplo:

```
# Funções para operadores lógicos

def implies(p, q):

    return not p or q

def equivalent(p, q):

    return (p and q) or (not p and not q)

# Definição de proposições

P = True

Q = False

# Avaliação de proposições

implies_result = implies(P, Q)

equivalent_result = equivalent(P, Q)

print("P implies Q:", implies_result)

print("P equivalent Q:", equivalent_result)
```

Avaliação de Expressões Proposicionais:

```
# Definição de proposições
```

```
P = True
```

```
Q = False
```

```
# Expressão proposicional: (P AND (NOT P OR Q))
```

```
expression = P and (not P or Q)
```

```
print("Expression (P and (not P or Q)):", expression)
```

Lógica de Predicados em Python

A lógica de predicados permite expressar relações mais complexas entre objetos utilizando predicados e quantificadores. Podemos criar funções para representar predicados e quantificadores.

Exemplo:

```
# Definição de predicados como funções
```

```
def Gato(x):
```

```
    return x in ["Felix", "Tom"]
```

```
def Mamifero(x):
```

```

return x in ["Felix", "Tom", "Elefante"]

# Definição de quantificadores

def forall(pred, domain):

    return all(pred(x) for x in domain)

def exists(pred, domain):

    return any(pred(x) for x in domain)

# Domínio de objetos

dominio = ["Felix", "Tom", "Elefante"]

# Quantificador universal: Todos os gatos são mamíferos

quant_universal = forall(lambda x: not Gato(x) or Mamifero(x), dominio)

# Quantificador existencial: Existe um gato que não é mamífero

quant_existencial = exists(lambda x: Gato(x) and not Mamifero(x), dominio)

print("Quantificador Universal (Todos os gatos são mamíferos):", quant_universal)

print("Quantificador Existencial (Existe um gato que não é mamífero):", quant_existencial)

```

Implementação de Expressões de Predicados:

```
# Definição de predicados
```

```
def Gato(x):
```

```
    return x == "Felix"
```

```
def Preto(x):
```

```
    return x == "Felix"
```

```
# Expressões de predicados
```

```
expr1 = Gato("Felix")
```

```
expr2 = Gato("Felix") and Preto("Felix")
```

```
print("Gato(Felix):", expr1)
```

```
print("Gato(Felix) and Preto(Felix):", expr2)
```

Agora que implementamos essas funcionalidades em Python puro, vamos utilizar uma biblioteca que facilita a manipulação simbólica.

Biblioteca SymPy

Para facilitar a manipulação simbólica de expressões lógicas, podemos utilizar a biblioteca SymPy. SymPy oferece uma ampla gama de ferramentas para a manipulação simbólica, incluindo variáveis simbólicas, operadores lógicos e ferramentas, para simplificação de expressões.

Instalação e Configuração:

```
pip install sympy
```

Estruturas Básicas e Funções:

```
from sympy import symbols, And, Or, Not, Implies
```

```
# Definição de variáveis simbólicas
```

```
P, Q = symbols('P Q')
```

```
# Criação de expressões lógicas
```

```
expr1 = And(P, Q)
```

```
expr2 = Or(P, Not(Q))
```

```
expr3 = Implies(P, Q)
```

```
# Avaliação de expressões lógicas
```

```
print(expr1)
```

```
print(expr2)
```

```
print(expr3)
```

Essas ferramentas e bibliotecas fornecem uma base sólida para a manipulação simbólica e a aplicação de lógica proposicional e de predicados em Python. Ao utilizarmos SymPy para a manipulação simbólica geral, podemos aplicar de forma eficiente os conceitos aprendidos em lógica matemática e resolver problemas complexos de forma mais prática e intuitiva.

NOVOS DESAFIOS

A exploração do paradigma simbólico na IA, a lógica proposicional, a lógica de predicados e a programação em Python são fundamentais para o ambiente profissional. A representação simbólica permite codificar o conhecimento de maneira estruturada, o que é essencial para sistemas inteligentes e suporte à decisão.

No mercado, essas habilidades são valiosas em consultoria empresarial, engenharia de software e análise de dados, em que a modelagem matemática de problemas ajuda a desenvolver algoritmos eficazes para a otimização de processos e análise preditiva. Python, com as próprias bibliotecas, tais como SymPy, facilita a criação e a manipulação de expressões simbólicas, automatizando tarefas complexas e otimizando processos.

A **integração de lógica simbólica** com tecnologias emergentes, como aprendizado de máquina e IoT, amplia as possibilidades de aplicação, permitindo o desenvolvimento de sistemas híbridos que combinam regras simbólicas com modelos de aprendizado. Isso é altamente valorizado em setores, como automação, TI e Indústria 4.0.

À medida que a tecnologia avança, a necessidade de profissionais que conectem teoria e prática se torna essencial. A lógica simbólica e a programação em Python são habilidades essenciais que permitem abordar desafios complexos de maneira estruturada e eficiente. A demanda por essas competências aumenta com a crescente adoção de IA e tecnologias relacionadas.

Como a lógica proposicional e a lógica de predicados, aliadas à programação em Python, podem impactar a sua área de atuação? Pense nas aplicações práticas desses conceitos no seu campo de trabalho. Reflita sobre como você pode usar o conhecimento adquirido para melhorar processos e resolver problemas de maneira mais eficiente. Considere também as implicações éticas e a importância de desenvolver soluções justas e transparentes.

A compreensão e a aplicação desses conceitos proporcionam uma base sólida para uma carreira de sucesso, atendendo às demandas do mercado de trabalho e preparando profissionais para as exigências futuras.

AUTOATIVIDADE

1. A lógica proposicional é uma forma de lógica que se concentra em proposições que são sentenças declarativas que podem ser classificadas como verdadeiras ou falsas. Na lógica proposicional, proposições simples são combinadas usando operadores lógicos para formar proposições compostas. Os operadores lógicos básicos incluem AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$) e EQUIVALENT ($\leftrightarrow$). A lógica proposicional é usada para construir proposições mais complexas a partir de proposições simples e analisar as inter-relações e implicações lógicas.

Qual dos seguintes operadores lógicos representa a conjunção de duas proposições?

- a) OR ($\vee$)
 - b) AND ($\wedge$)
 - c) NOT ($\neg$)
 - d) IMPLIES ($\rightarrow$)
 - e) EQUIVALENT ($\leftrightarrow$)
2. A lógica de predicados é uma extensão da lógica proposicional que inclui quantificadores e permite expressar relações mais complexas entre objetos. Enquanto a lógica proposicional lida com proposições simples que podem ser verdadeiras ou falsas, a lógica de predicados adiciona uma camada de profundidade ao permitir a especificação de propriedades de objetos e as relações entre eles. Na lógica de predicados, utilizamos quantificadores para generalizar proposições sobre conjuntos de objetos.

Considerando os quantificadores na lógica de predicados, analise as afirmativas a seguir:

- I - O quantificador universal ($\forall$) afirma que uma proposição é verdadeira para todos os elementos de um conjunto.
- II - O quantificador existencial ($\exists$) afirma que uma proposição é verdadeira para pelo menos um elemento de um conjunto.
- III - O quantificador universal ($\forall$) afirma que uma proposição é verdadeira para pelo menos um elemento de um conjunto.
- IV - O quantificador existencial ($\exists$) afirma que uma proposição é verdadeira para todos os elementos de um conjunto.

AUTOATIVIDADE

É correto o que se afirma em:

- a) I e II, apenas.
- b) II e III, apenas.
- c) I, II e III, apenas.
- d) I, III e IV, apenas.
- e) I, II, III e IV.

3. A lógica proposicional se concentra em proposições que são sentenças declarativas que podem ser classificadas como verdadeiras ou falsas. A lógica proposicional permite combinar proposições simples para formar proposições compostas e utiliza operadores lógicos, como AND, OR, e NOT, para manipular proposições.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - A lógica proposicional permite combinar proposições simples para formar proposições compostas.

PORQUE

II - A lógica proposicional utiliza operadores lógicos, como AND, OR, e NOT, para manipular proposições.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

ABE, J. M.; SCALZITTI, A.; SILVA FILHO, J. I. da. **Introdução à Lógica para a Ciência da Computação**. Rio de Janeiro: Arte & Ciência, 2002.

DOWNEY, A. B. **Pense em Python**: pense como um cientista da computação. São Paulo: Novatec, 2019.

FAIS, R. M. **Lógica Computacional**. Rio de Janeiro: Clube de Autores, 2020.

MENEZES, N. N. C. **Introdução à programação com Python**: algoritmos e lógica de programação para iniciantes. 3. ed. São Paulo: Novatec, 2019.

RUSSELL, S.; NORVIG, P. **Artificial intelligence**: a modern approach. [S. l.]: Pearson, 2021.

SOUZA, J. N. de. **Lógica para Ciência da Computação e Áreas Afins**: uma introdução concisa sobre os fundamentos da lógica. Rio de Janeiro: Elsevier, 2020.

SOUZA, J. N. de. **Lógica para Ciência da Computação**. Rio de Janeiro: Elsevier, 2008.

GABARITO

1. Opção B.

O operador AND ($\wedge$) representa a conjunção de duas proposições, sendo verdadeiro apenas se ambas as proposições forem verdadeiras. O operador OR ($\vee$) representa a disjunção, que é verdadeiro se pelo menos uma das proposições for verdadeira. O operador NOT ($\neg$) representa a negação de uma proposição. O operador IMPLIES ($\rightarrow$) representa a implicação, que é falso apenas se a primeira proposição for verdadeira e a segunda for falsa. O operador EQUIVALENT ($\leftrightarrow$) representa a equivalência entre proposições, que é verdadeiro se ambas as proposições têm o mesmo valor de verdade.

2. Opção A.

O quantificador universal ($\forall$) afirma que uma proposição é verdadeira para todos os elementos de um conjunto. O quantificador existencial ($\exists$) afirma que uma proposição é verdadeira para pelo menos um elemento de um conjunto. As afirmativas III e IV estão incorretas, porque invertem as definições dos quantificadores.

3. Opção A.

A lógica proposicional permite combinar proposições simples para formar proposições compostas. Além disso, utiliza operadores lógicos, como AND, OR, e NOT, para manipular proposições. Ambas as asserções são verdadeiras, e a segunda justifica a primeira, ao explicar como as proposições são combinadas.





TEMA DE APRENDIZAGEM 4

MÉTODOS DE BUSCA EM IA

MINHAS METAS

- Compreender os diferentes algoritmos de busca utilizados em IA.
- Formalizar matematicamente os algoritmos de busca.
- Analisar a complexidade dos algoritmos de busca.
- Implementar algoritmos de busca em Python.
- Comparar a performance dos algoritmos de busca.
- Aplicar algoritmos de busca em problemas práticos.
- Relacionar os algoritmos de busca às aplicações reais no mercado de trabalho.

INICIE SUA JORNADA

Em um mundo cada vez mais interconectado, a eficiência logística se tornou um fator essencial para o sucesso das empresas. Com o crescimento do comércio eletrônico e a expectativa de entregas rápidas, as empresas enfrentam o desafio constante de otimizar as rotas de entrega em meio a cidades congestionadas, condições climáticas imprevisíveis e restrições de horário. Esses obstáculos resultam em atrasos, aumento de custos e insatisfação dos clientes. Para resolver esses problemas, a Inteligência Artificial (IA) oferece uma ampla gama de algoritmos de busca.

No entanto, é importante destacar que nem todos os algoritmos são adequados para um roteamento eficiente. Por exemplo, a busca em largura (BFS) pode ser útil quando todas as arestas têm o mesmo custo, mas não é ideal para problemas com custos variáveis. Já a busca em profundidade (DFS) não é apropriada para encontrar rotas mínimas ou soluções otimizadas, pois pode explorar caminhos ineficientes e não garante a melhor solução. Ao contrário disso, alguns algoritmos, como Dijkstra e A*, são mais indicados, pois consideram custos e distâncias variáveis, garantindo uma otimização eficaz das rotas de entrega.

Métodos de busca em IA, como busca em largura, busca em profundidade e busca heurística, podem revolucionar a forma como as rotas são planejadas, tornando-as mais eficientes e econômicas. No entanto, a implementação desses algoritmos em sistemas computacionais não é simples. Além dos desafios técnicos, como a necessidade de lidar com grandes volumes de dados em tempo real, há a questão da escalabilidade. Como garantir que esses algoritmos funcionem de forma eficiente em diferentes contextos e com diferentes volumes de dados? Superar essas barreiras é essencial para explorar todo o potencial dos algoritmos de busca em IA e aplicá-los de maneira eficaz e responsável.

Os métodos de busca em IA, fundamentados na **teoria dos grafos** e na lógica matemática, são ferramentas poderosas e versáteis que encontram aplicação em diferentes áreas da Inteligência Artificial. Eles não apenas resolvem problemas complexos de forma eficiente, mas também são fundamentais em setores, como transporte, medicina e entretenimento. Na logística, por exemplo, os algoritmos de busca podem otimizar rotas de entrega, economizando tempo e recursos, e melhorando significativamente a eficiência operacional.

Imagine que você está trabalhando em uma empresa de logística que precisa otimizar as rotas de entrega para reduzir custos e aumentar a eficiência. Como você aplicaria os métodos de busca em IA para resolver esse problema?

Desenvolva um pequeno estudo de caso considerando os seguintes pontos: identificação do problema, visto que a empresa enfrenta desafios na otimização de rotas devido a congestionamentos, mudanças climáticas e restrições de horário; coleta de dados relevantes, incluindo dados sobre mapas da cidade, zonas de voo restritas, condições de trânsito e previsões meteorológicas; aplicação de algoritmos de busca, implementando e testando algoritmos de busca em largura, profundidade e A* para encontrar as rotas mais eficientes; e avaliação dos resultados, medindo a eficiência das rotas otimizadas em termos de tempo de entrega, consumo de energia e segurança, comparando os resultados dos diferentes algoritmos para determinar qual é o mais eficiente.

Esse exercício prático permitirá visualizar como a teoria dos algoritmos de busca pode ser aplicada em situações reais, o que te preparará para enfrentar desafios semelhantes em sua carreira profissional.

Como a evolução dos métodos de busca em IA pode impactar a sua área de atuação? Quais são as possíveis aplicações e implicações desses algoritmos no seu campo de trabalho? Reflita sobre como os conhecimentos adquiridos podem ser utilizados para melhorar processos e tomar decisões mais informadas. Considere também os desafios de escalabilidade e como garantir que os algoritmos funcionem eficientemente em diferentes contextos e com volumes variados de dados.

É essencial que os futuros profissionais não só dominem a tecnologia, mas também compreendam as responsabilidades associadas ao respectivo uso. Como você pode garantir que as soluções desenvolvidas sejam eficazes, escaláveis e benéficas para a sociedade? Reflita sobre essas questões e prepare-se para ser um protagonista na transformação digital e tecnológica em seu campo de atuação.

**PLAY NO CONHECIMENTO**

Neste episódio do nosso podcast, exploramos os métodos de busca em Inteligência Artificial (IA) e entendemos como eles estão transformando a logística moderna. Venha descobrir como a IA está revolucionando o setor logístico e impactando positivamente outras áreas! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para garantir que você esteja preparado para compreender e aplicar os conceitos de algoritmos de busca em IA, vamos relembrar alguns conteúdos essenciais. Primeiro, é importante entender a teoria dos grafos, que é a base para muitos algoritmos de busca. A teoria dos grafos inclui conceitos, tais como nós, arestas, caminhos e ciclos. Além disso, revisar os princípios de complexidade computacional ajudará a entender as limitações e os desafios de cada algoritmo. Saiba mais! <http://www.obmep.org.br/docs/apostila5.pdf>

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO AOS ALGORITMOS DE BUSCA EM IA

A resolução de problemas é uma parte central da Inteligência Artificial (IA), já que os agentes buscam soluções para alcançar objetivos definidos. O processo de busca é essencial para explorar o espaço de estados, encontrar caminhos ótimos e tomar decisões informadas (Russell; Norvig, 2013). Existem várias estratégias de busca, cada uma com próprias características e aplicações.

Busca não informada

A busca não informada, também conhecida como busca cega, explora o espaço de estados sem qualquer conhecimento adicional sobre a disposição dos estados ou o custo associado (Nilsson, 1982). Exemplos clássicos dessa categoria incluem:

- **Busca em Largura (Breadth-First Search - BFS):** examina todos os nós de um nível antes de passar para o próximo, garantindo a solução de menor custo em termos de número de arestas (Russell; Norvig, 2013).
- **Busca em Profundidade (Depth-First Search - DFS):** explora uma ramificação até o máximo possível antes de retroceder, o que pode ser eficiente em espaços de estados com poucas soluções (Nilsson, 1982).

Busca informada (heurística)

A busca informada, ou busca heurística, utiliza conhecimento adicional para guiar a exploração do espaço de estados (Russell; Norvig, 2013). Esse conhecimento é frequentemente representado por uma função heurística que estima o custo ou a distância até o objetivo. Exemplos de algoritmos:

- **A*:** combina a busca em largura com a heurística para encontrar o caminho mais curto de forma eficiente (Russell; Norvig, 2013).



A busca informada é geralmente mais rápida e eficiente do que a busca não informada, especialmente em espaços de estados grandes e complexos (Luger, 2021).

Busca local

A busca local se concentra em encontrar soluções por meio de melhorias iterativas, começando com uma solução inicial e fazendo pequenas alterações para encontrar soluções melhores (Taulli, 2019). Métodos comuns incluem:

- **Subida de Encosta (Hill Climbing):** melhora iterativamente a solução atual com base em um critério local (Russell; Norvig, 2013).
- **Algoritmos Genéticos:** utilizam princípios de evolução natural, como seleção, cruzamento e mutação, para explorar um conjunto de soluções simultaneamente (Haykin, 2005).

A seguir, apresentaremos uma análise detalhada dos principais algoritmos de busca utilizados em IA. Começaremos pela busca em largura, seguida pela busca em profundidade e, finalmente, chegaremos à busca heurística. Cada seção conterá a explicação do conceito, a formalização matemática, a implementação em Python e a análise de complexidade.

BUSCA EM LARGURA (BREADTH-FIRST SEARCH - BFS)

A busca em largura é um algoritmo de busca não-informada que explora todos os nós de um nível antes de passar para o próximo. Esse algoritmo utiliza uma estrutura de dados chamada fila, na qual os nós são enfileirados à medida que são descobertos e desenfileirados para exploração. A busca em largura garante que a solução encontrada será a de menor custo em termos de número de arestas, porque explora todos os nós de um nível antes de avançar para o próximo (Russell; Norvig, 2013).

Imagine que você está em um labirinto e quer encontrar a saída. A busca em largura começa na posição inicial e explora todas as posições adjacentes (nós) antes de mover para a próxima camada de posições. Isso garante que você encontrará a saída com o menor número de passos possíveis.

Características:

- **Completa:** sempre encontra a solução se ela existir, pois explora todos os nós possíveis (Nilsson, 1982).
- **Ótima:** encontra a solução de menor custo se todos os passos tiverem o mesmo custo (Russell; Norvig, 2013).
- **Aplicações:** ideal para problemas nos quais a solução está próxima do nó inicial, tais como em jogos de tabuleiro simples e labirintos (Russell; Norvig, 2013).

Formalização matemática

A busca em largura pode ser formalizada como uma exploração em níveis de um grafo utilizando uma fila para armazenar os nós a serem explorados (Nilsson, 1982). Considere o exemplo a seguir, na Figura 1:

- **Entrada:**
 - Grafo $G(V,E)$, em que:
 - $V=\{A,B,C,D,E,F\}$ são os vértices.
 - $E=\{(A,B),(A,C),(B,D),(B,E),(C,F),(E,F)\}$ são as arestas.
 - Nó inicial: $s=A$
 - Nó alvo: $t=F$
- **Saída:** Nó alvo t ('F') e caminho p de s a t ($[A, 'C', 'F']$).

Busca em Largura (BFS) no Grafo Simples

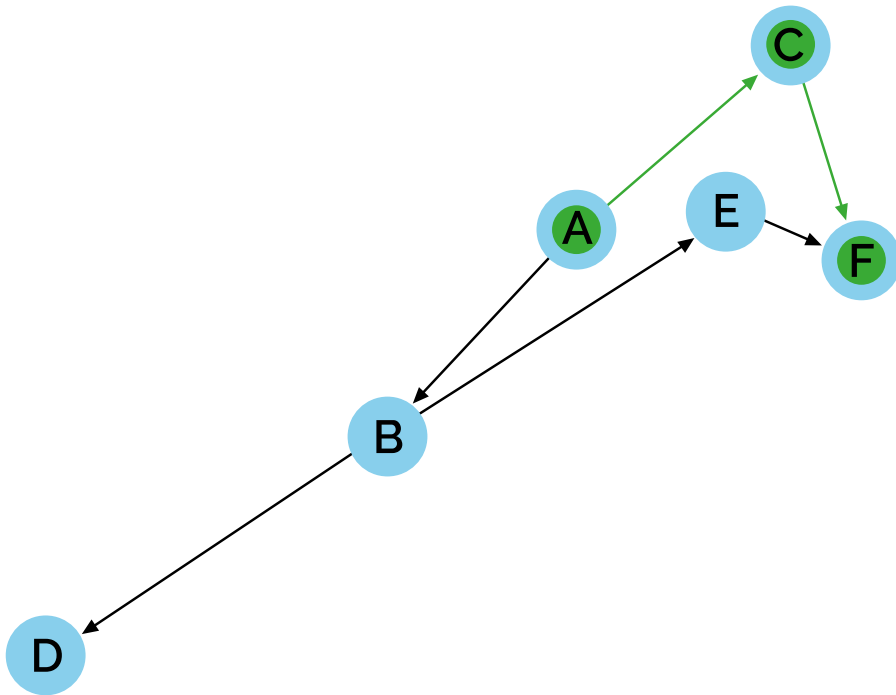


Figura 1 - Busca em largura / Fonte: o autor.

Descrição da Imagem: é exibido um grafo ilustrando a busca em largura (BFS). Os nós são representados por círculos azuis rotulados com letras (A, B, C, D, E e F). As arestas entre os nós são indicadas por linhas pretas e verdes, mostrando as conexões entre eles. O grafo apresenta uma estrutura ramificada, com o nó A conectado diretamente aos nós B e C. O nó B está ligado ao nó D, enquanto o nó E se conecta ao nó F. As arestas verdes destacam um possível caminho descoberto pela BFS, mostrando conexões de A para C e de C para F. Fim da descrição.

O caminho encontrado é destacado em verde-claro no grafo. Esse exemplo ilustra como a BFS explora todos os nós de um nível antes de avançar para o próximo, garantindo a menor quantidade de passos do nó inicial ao nó alvo.

Implementação em Python

O código implementa o algoritmo de busca em largura (BFS) para encontrar o caminho mais curto em termos de número de arestas entre dois nós em um grafo. A seguir, é apresentada a explicação detalhada do código Python fornecido.

```
from collections import deque
```

```
def bfs(graph, start, goal):
```

```
    queue = deque([start])
```

```
    visited = set([start])
```

```
    parent = {start: None}
```

```
    while queue:
```

```
        current = queue.popleft()
```

```
        if current == goal:
```

```
            path = []
```

```
            while current:
```

```
                path.append(current)
```

```
                current = parent[current]
```

```
            return path[::-1]
```

```
    for neighbor in graph[current]:
```

```
        if neighbor not in visited:
```

```
            visited.add(neighbor)
```

```
            parent[neighbor] = current
```

```
            queue.append(neighbor)
```

```
    return None
```

```
# Exemplo de uso
```

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['D', 'E'],
```

```
    'C': ['F'],
```

```

'D': [],
'E': ['F'],
'F': []
}

print(bfs(graph, 'A', 'F'))

```

No exemplo fornecido, um grafo é definido com nós e as respectivas conexões. A função **bfs** é chamada com o nó inicial **A** e o nó alvo **F**. O algoritmo percorre o grafo e encontra o caminho **[A, C, F]**, que é, então, impresso como resultado.

A BFS explora todos os nós de um nível antes de passar para o próximo, garantindo que o caminho mais curto em termos de número de arestas seja encontrado (Russell; Norvig, 2013). Esse método é eficiente e evita ciclos, ao marcar nós como visitados. A fila (deque) e o conjunto (visited) são usados para gerenciar a ordem de exploração e evitar a revisita de nós, respectivamente, enquanto o dicionário (parent) rastreia os pais dos nós para reconstruir o caminho final.



APROFUNDANDO

Análise de complexidade

A análise de complexidade de algoritmos é fundamental para entender o desempenho e a eficiência das soluções computacionais. No caso da **busca em largura (BFS)**, um dos algoritmos mais usados para a exploração de grafos, o tempo de execução e o consumo de memória dependem diretamente da estrutura do grafo. Segundo Cormen (2009), a complexidade da BFS pode ser descrita da seguinte forma:

- Tempo: $O(|V| + |E|)$, em que $|V|$ é o número de vértices e $|E|$ é o número de arestas.
- Espaço: $O(|V|)$, para armazenar todos os nós na fila e as marcas de visitação.
- Compreendido o funcionamento da busca em largura, passamos agora para a análise da busca em profundidade.

BUSCA EM PROFUNDIDADE (DEPTH-FIRST SEARCH - DFS)

A busca em profundidade é um algoritmo de busca não-informada que explora o máximo possível ao longo de cada ramificação antes de retroceder. Diferentemente da busca em largura, que utiliza uma fila, a busca em profundidade utiliza uma pilha para armazenar os nós a serem explorados. Esse método é eficiente em termos de uso de memória e é útil para encontrar todas as soluções possíveis em um espaço de busca (Russell; Norvig, 2013).

Em outras palavras, imagine que você está em uma caverna com várias bifurcações. A busca em profundidade simula uma exploração na qual você segue cada caminho até o fim antes de voltar e tentar um novo caminho. Isso pode ser útil para explorar todas as possibilidades em profundidade antes de considerar alternativas mais próximas do ponto de partida.

Características:

- **Não Completa:** pode falhar em encontrar a solução em espaços de busca infinitos, pois pode entrar em um ciclo infinito (Nilsson, 1982).
- **Não Ótima:** pode encontrar uma solução que não é a de menor custo, pois pode seguir uma ramificação longa antes de encontrar a solução (Russell; Norvig, 2013).
- **Aplicações:** útil em problemas em que todas as soluções precisam ser exploradas ou quando a profundidade da solução é grande, como em quebra-cabeças complexos e jogos (Russell; Norvig, 2013).

Formalização matemática

A busca em profundidade pode ser formalizada como uma exploração recursiva de cada ramificação de um grafo usando uma pilha (implícita ou explícita) para armazenar os nós a serem explorados (Nilsson, 1982). Observe um exemplo na Figura 2.

Entrada: grafo $G(V,E)$ e nó inicial s ('A').

Saída: nó alvo t ('F') e caminho p de s a t (['A', 'C', 'F']).

Busca em Profundidade (DFS) no Grafo Simples

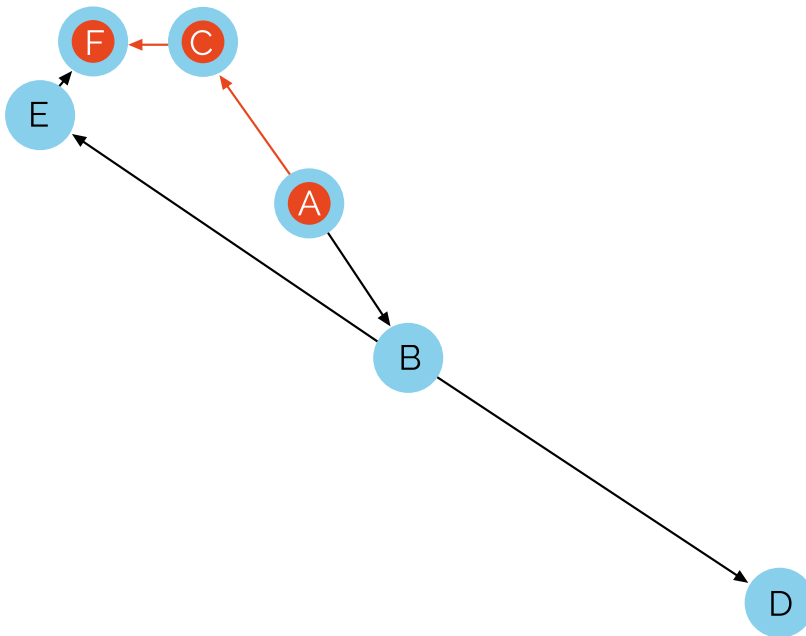


Figura 2 - Busca em profundidade / Fonte: o autor.

Descrição da Imagem: é exibido um grafo simples com seis nós rotulados como A, B, C, D, E e F. Os nós estão conectados por linhas que representam as arestas do grafo. Há um caminho destacado do nó A para o nó C e, em seguida, para o nó F, que está colorido em vermelho. Acima do grafo, há um texto em português que diz "Busca em Profundidade (DFS) no Grafo Simples". Fim da descrição.

O caminho encontrado é destacado em vermelho-claro no grafo da Figura 2. Esse exemplo ilustra como a DFS explora uma ramificação ao máximo antes de retroceder, utilizando uma pilha para armazenar os nós a serem explorados.

Implementação em Python

O código implementa o algoritmo de busca em profundidade (DFS) para encontrar um caminho entre dois nós em um grafo. A seguir, é apresentada a explicação detalhada do código Python fornecido.

```

def dfs(graph, start, goal):
    stack = [start]
    visited = set([start])
    parent = {start: None}

    while stack:
        current = stack.pop()
        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                stack.append(neighbor)

    return None

# Exemplo de uso
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],

```

```
'F': []
}

print(dfs(graph, 'A', 'F'))
```

No exemplo fornecido, um grafo é definido com nós e as respectivas conexões. A função **dfs** é chamada com o nó inicial 'A' e o nó alvo 'F'. O algoritmo percorre o grafo e encontra o caminho ['A', 'C', 'F'], que é, então, impresso como resultado.

A DFS explora ao máximo cada ramificação antes de retroceder, utilizando uma pilha para gerenciar os nós a serem explorados. Esse método é eficiente para explorar todos os caminhos possíveis até encontrar o nó alvo e é especialmente útil em grafos em que as soluções estão mais profundas.

Análise de complexidade

A **busca em profundidade (DFS)** se destaca pela abordagem sistemática de exploração de grafos, seguindo cada caminho até o final antes de retroceder. Esse comportamento influencia diretamente a complexidade computacional, tanto em tempo quanto em espaço, tornando-a eficiente para diversas aplicações, como detecção de ciclos e busca de componentes conexas. De acordo com Cormen (2009), a complexidade da DFS pode ser descrita da seguinte forma:

- Tempo: $O(|V| + |E|)$, em que $|V|$ é o número de vértices e $|E|$ é o número de arestas.
- Espaço: $O(|V|)$ em uma implementação iterativa ou até $O(h)$ em uma implementação recursiva, em que h é a altura da árvore.

Após entendermos a busca em profundidade, avançamos para a busca heurística, com foco no algoritmo A^* .



BUSCA HEURÍSTICA

A busca heurística usa informações adicionais para guiar a busca, visando encontrar soluções mais rapidamente que os algoritmos não informados. O algoritmo A^* é um exemplo clássico de busca heurística (Russell; Norvig, 2013).

Em outras palavras, suponha que você está tentando encontrar o caminho mais rápido para chegar a um destino em uma cidade. A busca heurística, como o algoritmo A^* , utiliza uma estimativa da distância (heurística) até o destino para escolher o caminho mais promissor a cada passo, combinando a distância já percorrida com a estimativa do que ainda falta percorrer.

Características:

- **Completa e Ótima:** dependente da heurística utilizada (Russell; Norvig, 2013).
- **Algoritmos Comuns:** A^* e Greedy (Russell; Norvig, 2013).
- **Aplicações:** ideal para problemas em que se pode usar uma função heurística para guiar a busca, como em jogos ou roteamento de redes (Luger, 2021).

Formalização matemática

A busca A^* combina a busca em largura e a busca heurística para encontrar o caminho mais curto para o objetivo. Logo, usa uma função de custo que considera tanto o custo já incorrido quanto uma estimativa heurística do custo restante (Russell; Norvig, 2013). Observe, na Figura 3, um exemplo da busca A^* em combinação com a busca heurística. Exemplo:

- **Entrada:** grafo $G(V,E)$, nó inicial s (A), nó alvo t (J), função heurística h .
- **Saída:** caminho de s a t ($[A, B, D, G, J]$)

Busca A* no grafo grande

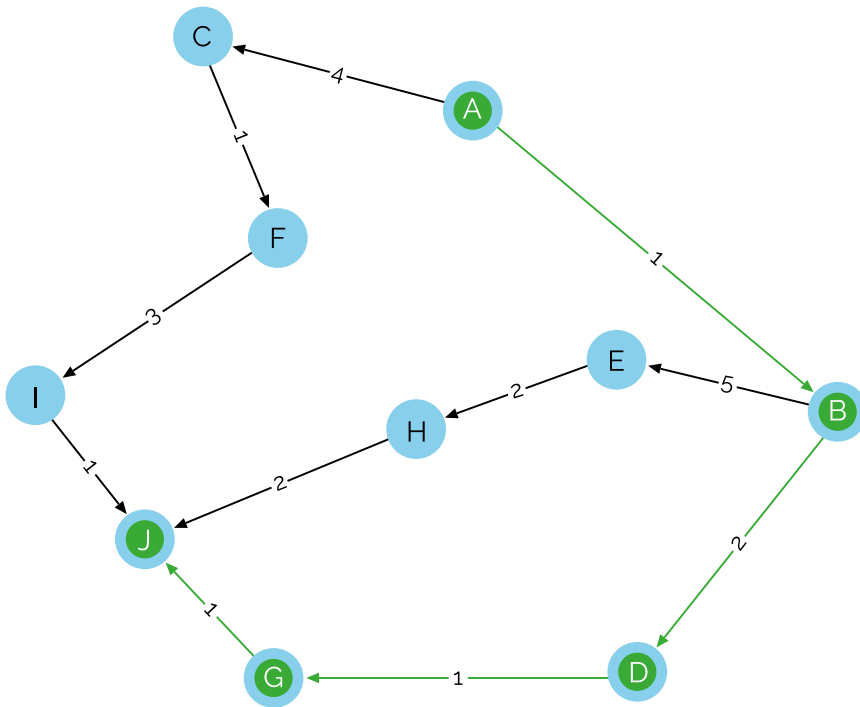


Figura 3 - Busca A* / Fonte: o autor.

Descrição da Imagem: é mostrado um grafo com nós e arestas representando uma rede ou conexões entre pontos. Os nós são rotulados com as letras A até J. Há um título no topo da imagem que diz "Busca A* no Grafo Grande", sugerindo que se trata de uma ilustração relacionada ao algoritmo de busca A* em um grafo grande. As arestas entre os nós têm números, indicando, talvez, o custo ou a distância associada à viagem de um nó para outro. Fim da descrição.

O caminho encontrado é destacado em verde-claro no grafo. Esse exemplo ilustra como a A* utiliza a combinação de custo já incorrido e a heurística para encontrar o caminho mais curto até o objetivo.

Implementação em Python

O código implementa o algoritmo de busca A* (A-star) para encontrar o caminho mais curto entre dois nós em um grafo. A seguir, é apresentada a explicação detalhada do código Python fornecido.

```

import heapq

def heuristic(node, goal):
    return abs(ord(node) - ord(goal))

def a_star(graph, start, goal):
    queue = []
    heapq.heappush(queue, (0, start))
    costs = {start: 0}
    parent = {start: None}

    while queue:
        _, current = heapq.heappop(queue)
        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor, cost in graph[current].items():
            new_cost = costs[current] + cost
            if neighbor not in costs or new_cost < costs[neighbor]:
                costs[neighbor] = new_cost
                priority = new_cost + heuristic(neighbor, goal)
                heapq.heappush(queue, (priority, neighbor))
                parent[neighbor] = current

    return None

```

```
# Exemplo de uso

graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 2, 'E': 5},
    'C': {'F': 1},
    'D': {'G': 1},
    'E': {'F': 1},
    'F': {'I': 3},
    'G': {'J': 1},
    'H': {'J': 2},
    'I': {'J': 1},
    'J': {}
}

print(a_star(graph, 'A', 'J'))
```

No exemplo fornecido, um grafo é definido com nós, vizinhos e os custos das arestas. A função **a_star** é chamada com o nó inicial **A** e o nó alvo **J**. O algoritmo percorre o grafo e encontra o caminho [**A**, **B**, **D**, **G**, **J**], que é, então, impresso como resultado.

A busca **A*** utiliza uma combinação de custos acumulados e estimativas heurísticas para encontrar o caminho mais curto de um nó inicial a um nó alvo. Esse método é eficiente para encontrar caminhos ótimos em grafos, especialmente em situações em que a heurística pode fornecer boas estimativas dos custos restantes.

Análise de complexidade

A **busca A*** (*A-star*) é um algoritmo eficiente para encontrar caminhos mínimos em grafos ponderados, ao combinar a busca heurística com a estratégia de custo uniforme. A complexidade dela está diretamente relacionada ao fator de ramificação do grafo (**b**) e à profundidade da solução (**d**). Segundo Cormen (2009), a complexidade da busca **A*** pode ser descrita da seguinte forma:

- **Tempo:** $O(b^d)$, em que b é o fator de ramificação e d é a profundidade da solução.
- **Espaço:** $O(b^d)$, devido ao armazenamento de todos os nós na memória.

Agora que examinamos cada algoritmo individualmente, vamos comparar os respectivos desempenhos e características em uma tabela comparativa.

CRITÉRIOS DE COMPARAÇÃO

Para comparar os algoritmos de busca, utilizamos critérios, como tempo de execução, uso de memória, completude e otimalidade. Esses critérios são essenciais para avaliar a eficiência e a aplicabilidade de cada algoritmo em diferentes cenários, de acordo com Russell e Norvig (2013):

TEMPO DE EXECUÇÃO
O tempo necessário para o algoritmo encontrar a solução.
USO DE MEMÓRIA
A quantidade de memória usada durante a execução do algoritmo.
COMPLETUDE
A capacidade do algoritmo de encontrar uma solução se ela existir.
OTIMALIDADE
A capacidade do algoritmo de encontrar a solução de menor custo.

Utilizando um grafo grande e complexo, medimos o tempo de execução de cada algoritmo usando o módulo **time**, comparando os desempenhos em tempo de execução.

```
import time

def measure_time(func, *args):

    start_time = time.time()

    result = func(*args)

    end_time = time.time()

    return result, end_time - start_time

# Exemplo de grafo grande e complexo

graph_large = {

    'A': {'B': 1, 'C': 4},

    'B': {'D': 2, 'E': 5},

    'C': {'F': 1},

    'D': {'G': 1},

    'E': {'H': 2},

    'F': {'I': 3},

    'G': {'J': 1},

    'H': {'J': 2},
```

```

'I': {'J': 1},

'J': {}

}

bfs_result, bfs_time = measure_time(bfs, graph_large, 'A',
'J')

dfs_result, dfs_time = measure_time(dfs, graph_large, 'A',
'J')

a_star_result, a_star_time = measure_time(a_star, graph_
large, 'A', 'J')

print(f"BFS: {bfs_result} in {bfs_time} seconds")

print(f"DFS: {dfs_result} in {dfs_time} seconds")

print(f"A*: {a_star_result} in {a_star_time} seconds")

```

Comparar algoritmos de busca com base em critérios, como tempo de execução, uso de memória, completude e otimalidade, é essencial para identificar o mais adequado em cada situação. Ao aplicar esses algoritmos a um grafo grande e complexo, surgiram diferenças notáveis de desempenho, especialmente no tempo de execução. Esses achados oferecem uma base confiável para selecionar o algoritmo mais eficiente e eficaz, considerando a complexidade e as necessidades específicas de cada problema.

Comparação

Os algoritmos de busca em grafos desempenham um papel fundamental na resolução de problemas computacionais, variando em eficiência e aplicabilidade conforme a estrutura do grafo e os requisitos da busca. A busca em largura (BFS)

é ideal para encontrar caminhos mínimos em grafos não ponderados, enquanto a busca em profundidade (DFS) é útil para exploração exaustiva e problemas, como detecção de ciclos. Já a busca A^* (*A-star*) se destaca pelo uso de heurísticas, tornando-a mais eficiente para encontrar caminhos ótimos em grafos ponderados. A seguir, comparamos essas abordagens em termos de tempo de execução, consumo de memória, completude e otimalidade (Cormen, 2009):

TEMPO DE EXECUÇÃO

- **BFS:** o tempo de execução é $O(|V|+|E|)$, em que $|V|$ é o número de vértices e $|E|$ é o número de arestas. Isso ocorre, porque a BFS explora todos os nós e todas as arestas no pior caso.
- **DFS:** o tempo de execução é $O(|V|+|E|)$ pelos mesmos motivos da BFS, mas a DFS pode ser mais rápida em alguns casos específicos, nos quais a solução está próxima da raiz.
- **A^* :** o tempo de execução é $O(b^d)$, em que b é o fator de ramificação e d é a profundidade da solução. Isso porque o A^* explora muitos nós, mas pode ser mais rápido devido ao uso de heurísticas que guiam a busca.

USO DE MEMÓRIA

- **BFS:** o uso de memória é $O(|V|)$, pois todos os nós de um nível devem ser armazenados na fila.
- **DFS:** o uso de memória é $O(|V|)$ em uma implementação iterativa ou $O(h)$ em uma implementação recursiva, em que h é a altura da árvore. A DFS utiliza uma pilha de recursão que pode, no pior caso, armazenar todos os nós.
- **A^* :** o uso de memória é $O(b^d)$, pois o algoritmo deve armazenar todos os nós na memória devido ao armazenamento dos custos e das heurísticas.

COMPLETUDE

- **BFS:** a BFS é completa, ou seja, sempre encontrará uma solução se uma existir, pois explora todos os nós de um nível antes de passar para o próximo.
- **DFS:** a DFS não é completa, pois pode entrar em ciclos infinitos ou explorar caminhos sem saída em grafos infinitos.
- **A^* :** o A^* é completo, desde que a heurística utilizada seja admissível (nunca superestima o custo).

OTIMALIDADE

- **BFS:** a BFS é ótima se todos os passos tiverem o mesmo custo, pois encontra o caminho com o menor número de arestas.
- **DFS:** a DFS não é ótima, pois pode encontrar uma solução que não é a de menor custo, dependendo da ordem de exploração.
- **A*:** o A* é ótimo se a heurística utilizada for admissível e consistente, garantindo que o caminho encontrado é o de menor custo possível.

Cada algoritmo tem vantagens e desvantagens, e a escolha do melhor algoritmo depende do problema específico e das características do grafo. A BFS é eficiente para encontrar o caminho mais curto em termos de número de arestas. Já a DFS é útil para explorar todos os caminhos em profundidade, enquanto o A* é poderoso para encontrar caminhos ótimos utilizando heurísticas informadas.



EM FOCO

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A exploração dos **algoritmos de busca**, como BFS, DFS e A*, e a programação em Python são fundamentais para o ambiente profissional. A aplicação desses algoritmos permite resolver problemas complexos de maneira estruturada e eficiente, essencial para sistemas inteligentes e suporte à decisão. No mercado, essas habilidades são valiosas em diversas áreas, como desenvolvimento de software, Inteligência Artificial, engenharia de dados e ciência de dados.

A modelagem matemática de problemas com esses algoritmos ajuda a desenvolver soluções eficazes para a otimização de processos e a análise preditiva. Python, com suas bibliotecas e estruturas de dados eficientes, facilita a implementação desses algoritmos, automatizando tarefas complexas e otimizando processos.

A integração de algoritmos de busca com tecnologias emergentes, como aprendizado de máquina e IoT, amplia as possibilidades de aplicação, permitindo o desenvolvimento de sistemas híbridos que combinam heurísticas inteligentes com modelos de aprendizado. Isso é altamente valorizado em setores, como automação, TI e Indústria 4.0.

À medida que a tecnologia avança, a necessidade de profissionais que conectem teoria e prática se torna essencial. A compreensão e a aplicação dos algoritmos de busca são habilidades essenciais que permitem abordar desafios complexos de maneira estruturada e eficiente. A demanda por essas competências aumenta com a crescente adoção de IA e tecnologias relacionadas.

Como os algoritmos de busca podem impactar a sua área de atuação? Pense nas aplicações práticas desses conceitos no seu campo de trabalho. Reflita sobre como você pode usar o conhecimento adquirido para melhorar processos e resolver problemas de maneira mais eficiente. Considere também as implicações éticas e a importância de desenvolver soluções justas e transparentes.

A compreensão e a aplicação desses conceitos proporcionam uma base sólida para uma carreira de sucesso, atendendo às demandas do mercado de trabalho e preparando profissionais para as exigências futuras.



AUTOATIVIDADE

1. A busca em largura é um algoritmo de busca não-informada que explora todos os nós de um nível antes de passar para o próximo, garantindo uma cobertura completa de cada camada do espaço de busca. Esse método é especialmente útil em problemas em que é importante encontrar a solução mais próxima do ponto de partida.

Qual das seguintes alternativas é verdadeira para a busca em largura?

- a) Não garante encontrar a solução de menor custo.
 - b) Explora uma ramificação até o máximo possível antes de retroceder.
 - c) Utiliza uma fila para armazenar os nós a serem explorados.
 - d) É eficiente em termos de uso de memória
 - e) Utiliza uma função heurística para guiar a busca.
2. A busca informada, ou busca heurística, é uma técnica avançada que utiliza conhecimento adicional para guiar a exploração do espaço de estados de forma mais eficiente. Ao empregar uma função heurística que estima o custo ou a distância até o objetivo, o algoritmo pode priorizar os caminhos mais promissores, reduzindo o tempo de busca e o uso de recursos. Esse tipo de busca é especialmente útil em problemas complexos em que a eficiência é crucial, como na navegação de robôs e na resolução de problemas de otimização.

Considerando os algoritmos de busca informada, analise as afirmativas a seguir:

- I - Busca em Largura (BFS).
- II - A*.
- III - Busca em Profundidade (DFS).
- IV - Busca Gulosa (Greedy).

É correto o que se afirma em:

- a) I e II, apenas.
- b) II e IV, apenas.
- c) I, II e III, apenas.
- d) I, III e IV, apenas.
- e) I, II, III e IV.

AUTOATIVIDADE

3. A busca em largura (BFS) e a busca em profundidade (DFS) se diferem significativamente em relação às complexidades de tempo e espaço. A BFS explora todos os nós de um nível antes de avançar, garantindo a descoberta da solução mais próxima, mas exige um maior uso de memória. Já a DFS segue um caminho até o fim antes de retroceder. Usa menos memória, mas pode ser menos eficiente em alguns casos, especialmente em espaços de busca grandes ou infinitos.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - A busca em largura (BFS) tem complexidade de tempo $O(|V|+|E|)$, em que $|V|$ é o número de vértices e $|E|$ é o número de arestas.

PORQUE

II - A busca em profundidade (DFS) tem complexidade de espaço $O(|V|)$ em uma implementação iterativa.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

CORMEN, T. H. **Introduction to algorithms**. 3. ed. Cambridge: MIT Press, 2009.

HAYKIN, S. **Redes neurais**: princípios e prática. [S. l.]: Bookman, 2005.

LUGER, G. F. **Knowing our world**: an Artificial Intelligence perspective. [S. l.]: Springer, 2021.

NILSSON, N. J. **Principles of Artificial Intelligence**. [S. l.]: Elsevier, 1982.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence**: a modern approach. [S. l.]: Prentice Hall, 2013.

TAULLI, T. **Artificial Intelligence Basics**: a non-technical introduction. [S. l.]: Apress, 2019.

GABARITO

1. Opção C.

A busca em largura (BFS) utiliza uma fila para explorar todos os nós de um nível antes de passar para o próximo. É completa e ótima se todos os passos tiverem o mesmo custo. As alternativas A, B, D e E estão incorretas, porque descrevem características de outros tipos de busca ou incorretamente descrevem a BFS.

2. Opção B.

A* e busca gulosa (greedy) são exemplos de algoritmos de busca informada que utilizam heurísticas para guiar a busca. As afirmativas I e III estão incorretas, porque a busca em largura (BFS) e a busca em profundidade (DFS) são algoritmos de busca não-informada.

3. Opção B.

A asserção I é verdadeira, pois a BFS tem complexidade de tempo $O(|V|+|E|)$ devido à necessidade de explorar todos os vértices e arestas. A asserção II também é verdadeira, porque a DFS tem complexidade de espaço $O(|V|)$ em uma implementação iterativa. No entanto, a asserção II não justifica a asserção I, pois elas descrevem características de complexidade diferentes para BFS e DFS.



TEMA DE APRENDIZAGEM 5

REPRESENTAÇÃO DE CONHECIMENTO E INFERÊNCIA

MINHAS METAS

- Compreender os principais métodos de representação de conhecimento em IA.
- Explorar a aplicação da teoria dos grafos na representação de conhecimento.
- Entender como a inferência lógica é realizada a partir do conhecimento representado.
- Aplicar a lógica proposicional e a lógica de predicados em sistemas de inferência.
- Desenvolver habilidades práticas em programação para implementar sistemas de inferência simples.
- Utilizar Python para criar e manipular redes semânticas e grafos.
- Integrar conceitos teóricos e práticos para resolver problemas reais utilizando sistemas de inferência.

INICIE SUA JORNADA

Em um ambiente de trabalho cada vez mais orientado por dados, a capacidade de transformar grandes volumes de informação em conhecimento útil e acionável é um diferencial competitivo. Empresas de tecnologia, saúde, finanças e outras áreas enfrentam desafios significativos na organização e utilização de dados para tomar decisões estratégicas.

Como garantir que sistemas de IA consigam processar, interpretar e utilizar esses dados de maneira eficaz para apoiar decisões críticas? A necessidade de uma representação de conhecimento eficiente e de inferências lógicas precisas se torna cada vez mais evidente à medida que o mercado exige respostas rápidas e bem-informadas.

A representação de conhecimento e a inferência não são apenas conceitos teóricos, mas ferramentas práticas que ajudam a organizar e a aplicar conhecimento em situações do mundo real. Ao dominarem essas técnicas, os profissionais se tornam capazes de desenvolver sistemas que não só armazenam dados, mas que também “entendem” e usam essas informações para **resolver problemas complexos**.

Imagine um cenário em que você trabalha em uma startup de tecnologia focada em otimização de processos industriais. A sua equipe foi encarregada de desenvolver um sistema de IA que analisa dados de produção para identificar falhas operacionais e sugerir melhorias. Para que esse sistema seja eficaz, você precisa estruturar o conhecimento sobre os processos industriais de forma que o sistema possa inferir quais ajustes são necessários em cada etapa da produção. A criação de uma rede semântica que relacione dados de desempenho, possíveis falhas e soluções pode permitir que o sistema sugira automaticamente as melhores práticas, otimizando a operação e reduzindo custos.

Como esses conceitos podem ser aplicados no seu campo de atuação? Quais desafios você poderá enfrentar ao tentar representar conhecimento e realizar inferências em seu ambiente de trabalho? Reflita sobre como organizar e utilizar o conhecimento não apenas facilitarão a resolução de problemas complexos, mas também aumentarão a sua capacidade de inovar e liderar projetos que requerem tomada de decisão informada. Considere, também, a importância de garantir que as inferências feitas pelos sistemas de IA sejam justas, precisas e benéficas para todos os envolvidos.

**PLAY NO CONHECIMENTO**

Neste episódio do nosso podcast, exploramos os métodos de representação de conhecimento e inferência em Inteligência Artificial (IA) e como eles estão transformando diversos setores. Venha descobrir como a IA está revolucionando algumas áreas, tais como saúde, tecnologia e negócios, e impactando positivamente a sociedade! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para garantir que você esteja preparado para compreender e aplicar os conceitos de representação de conhecimento e inferência em IA, vamos relembrar alguns conteúdos essenciais. Primeiro, é importante entender a **teoria dos grafos**, que é fundamental para a representação de conhecimento em IA. A teoria dos grafos inclui conceitos, como nós, arestas e caminhos, que são essenciais para modelar redes de conhecimento. Além disso, é útil reforçar os fundamentos de **programação em Python**, que serão necessários para implementar sistemas de inferência simples. Saiba mais no link a seguir.

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À REPRESENTAÇÃO DE CONHECIMENTO EM IA

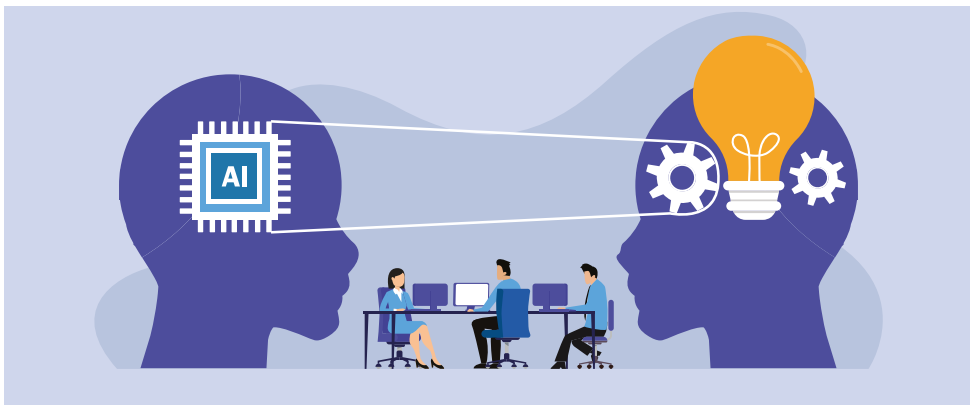
A **representação de conhecimento** é um dos pilares essenciais da Inteligência Artificial (IA), permitindo que sistemas computacionais não apenas armazenem dados, mas também os utilizem de maneira inteligente para realizar inferências, tomar decisões informadas e resolver problemas complexos (Luger, 2021). Organizar e estruturar informações de forma que a IA possa compreender e manipular esses dados são atitudes cruciais para estabelecer conexões entre conceitos e gerar novos insights.

A representação de conhecimento é fundamental para o desenvolvimento de sistemas de IA que interagem de maneira inteligente com o mundo, transcendendo o simples armazenamento de dados (Oliveira; Carvalho, 2008). Métodos, como redes semânticas, frames, ontologias e lógica proposicional, oferecem diferentes abordagens para modelar o conhecimento (Russell; Norvig, 2013). Além de organizarem as informações, esses métodos capacitam a IA a inferir novos dados e a tomar decisões, aplicando regras lógicas para gerar conclusões a partir do conhecimento existente.

MÉTODOS DE REPRESENTAÇÃO DE CONHECIMENTO

A representação de conhecimento é fundamental na Inteligência Artificial (IA), pois sustenta a compreensão, o processamento e a utilização de informações pelos sistemas de IA. As formas como o conhecimento é estruturado e armazenado afetam diretamente a capacidade da máquina de fazer inferências, tomar decisões e resolver problemas de maneira eficiente (Russell; Norvig, 2013). Diversos métodos de representação foram desenvolvidos ao longo do tempo, cada um mais adequado a tipos específicos de problemas e níveis de complexidade.

Em essência, a escolha do método de representação de conhecimento depende da natureza do problema que se deseja resolver, do tipo de informação que precisa ser manipulada e da forma como essa informação será utilizada dentro do sistema de IA. Um método bem escolhido pode aumentar significativamente a eficiência e a eficácia do sistema, enquanto uma escolha inadequada pode levar a problemas de desempenho e até mesmo a conclusões erradas. A seguir, apresentamos os principais métodos de representação de conhecimento utilizados em IA, detalhando as respectivas características, aplicações, vantagens e desvantagens.



Redes semânticas

As redes semânticas são uma das formas mais intuitivas de representação de conhecimento em Inteligência Artificial (IA). Elas oferecem uma maneira estruturada de capturar as relações entre conceitos ou entidades, utilizando grafos como modelo subjacente. Diferentemente das árvores, que têm uma estrutura hierárquica rígida e não permitem ciclos, os grafos são estruturas mais flexíveis, possibilitando múltiplas conexões entre os nós e a existência de relações não hierárquicas e cíclicas. Isso torna os grafos ideais para representar redes semânticas, nas quais as relações entre conceitos podem ser complexas e interconectadas. A estrutura de uma rede semântica é baseada em grafos, em que (Russell; Norvig, 2013):

Nós (Vértices): cada nó em uma rede semântica representa um conceito ou entidade. Esses conceitos podem variar de objetos físicos, como “Cão” ou “Carro”, a conceitos abstratos, como “Justiça” ou “Velocidade”. A capacidade de representar uma ampla gama de conceitos é o que torna as redes semânticas tão flexíveis (Russell; Norvig, 2013).

Arestas (Conexões): as arestas representam as relações entre os conceitos. Existem vários tipos de relações que podem ser representadas, dependendo do contexto e do objetivo da rede (Russell; Norvig, 2013):

- **Hierarquia (“é-um”):** indica que um conceito é um sub-tipo de outro. Por exemplo, a aresta entre “Cão” e “Mamífero” pode representar que “Cão é um tipo de Mamífero”. Observe o exemplo presente na Figura 1.

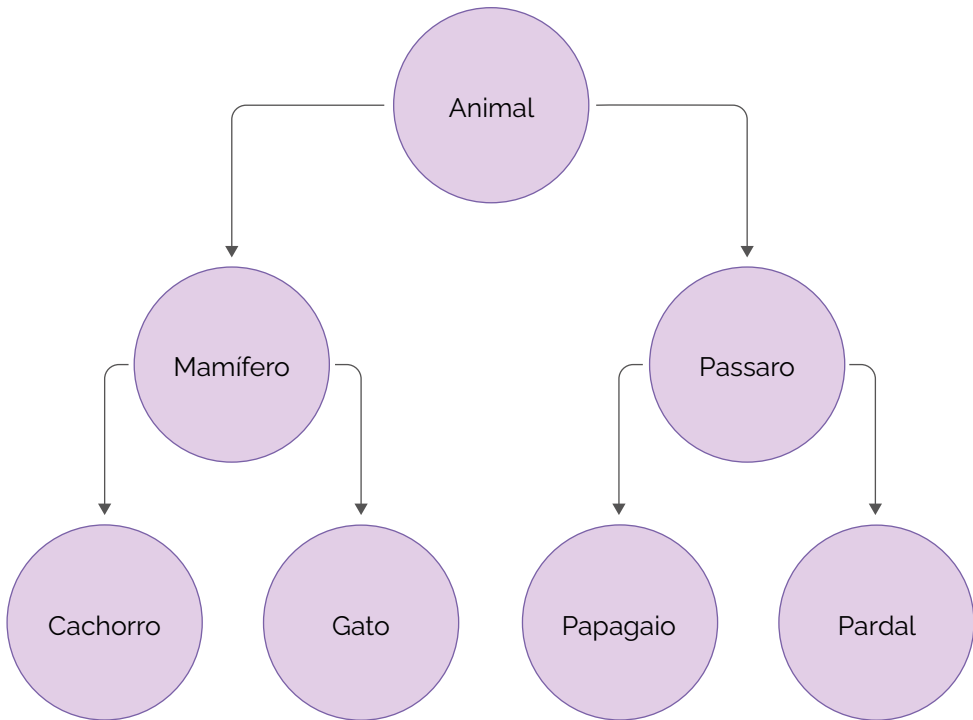


Figura 1 - Rede semântica que organiza conceitos relacionados ao reino animal / Fonte: o autor.

Descrição da Imagem: é exibida uma rede semântica que organiza conceitos relacionados ao reino animal em uma estrutura hierárquica. No topo, "Animal" é o conceito mais geral, que se divide em duas subcategorias: "Mamífero" e "Pássaro". Dentro da categoria "Mamífero", encontram-se os conceitos "Cachorro" e "Gato", enquanto "Pássaro" abrange "Papagaio" e "Pardal". As setas indicam relações do tipo "é-um", mostrando que cada conceito específico é um tipo da sua categoria superior. Fim da descrição.

- **Parte-todo ("parte-de"):** usada para indicar que um conceito é uma parte de outro. Por exemplo, "Roda" pode ser uma parte de "Carro". Observe o exemplo na Figura 2.

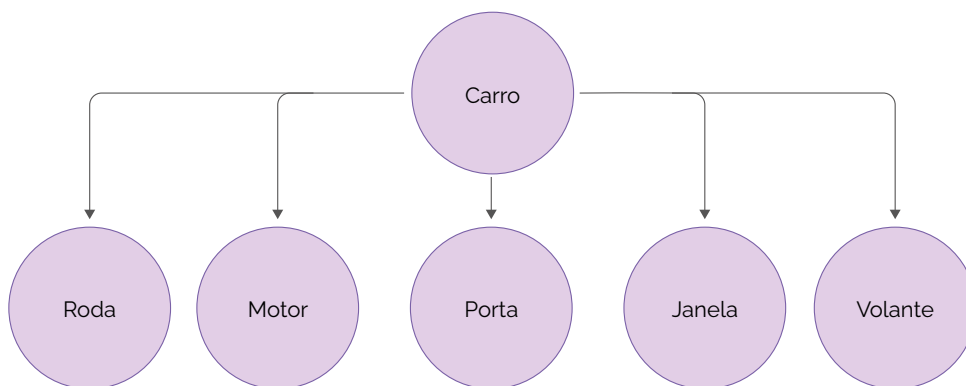


Figura 2 - Rede semântica organizando a relação parte-todo dos componentes de um carro / Fonte: o autor.

Descrição da Imagem: é exibido um diagrama hierárquico de um carro e suas partes. O círculo central é "Carro", conectado a "Roda", "Motor", "Porta", "Janela" e "Volante". É uma representação simples das partes de um carro. Fim da descrição.

- **Associação ("associado-com"):** representa uma relação associativa entre conceitos. Por exemplo, "Médico" pode estar associado a "Hospital". Observe o exemplo na Figura 3.

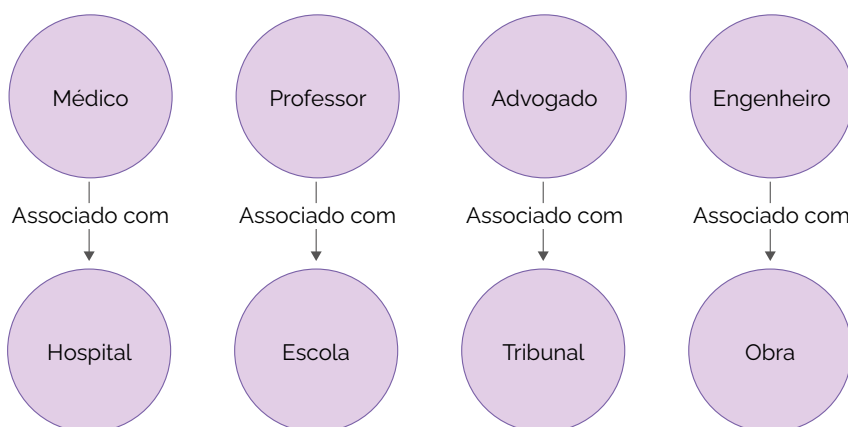


Figura 3 - Rede semântica que representa uma relação associativa entre dois conceitos / Fonte: o autor.

Descrição da Imagem: é exibido um diagrama com quatro pares de círculos conectados por setas. Cada círculo superior contém uma profissão: "Médico", "Professor", "Advogado" e "Engenheiro."

Cada círculo inferior contém um local de trabalho associado: "Hospital", "Escola", "Tribunal" e "Obra." Fim da descrição.

Essas categorias não se limitam a hierarquias rígidas e permitem a construção de modelos complexos, como redes associativas entre múltiplas entidades.

Frames

Frames são estruturas de dados amplamente utilizadas em Inteligência Artificial (IA) para organizar, representar e manipular informações de maneira estruturada e eficiente. Eles funcionam como “quadros” que agrupam um conjunto de dados relacionados, representando estereótipos de objetos, situações, eventos ou qualquer outra entidade que precise ser modelada no contexto de um sistema de IA (Oliveira; Carvalho, 2008).

Cada frame é essencialmente uma estrutura modular que encapsula um conhecimento relevante sobre determinado conceito, facilitando o acesso, a manipulação e a atualização dessas informações pela IA. Diferentemente das redes semânticas, em que cada nó representa um conceito simples, os frames representam conceitos ou situações complexas, cada um com diversas propriedades que podem ser especificadas ou herdadas por padrão (Oliveira; Carvalho, 2008).

A estrutura de um frame pode ser comparada a um formulário ou uma ficha, no qual cada campo (*slot*) contém uma peça de informação relevante sobre o objeto ou situação em questão. Esses *slots* são predefinidos e representam os atributos mais importantes que caracterizam o objeto ou evento modelado (Oliveira; Carvalho, 2008).

- **Nome do frame:** o nome do frame identifica de forma única o conceito ou entidade que o frame representa. Por exemplo, o nome do frame pode ser “Carro”, “Paciente”, “Evento”, dentre outros.
- **Slots:** os *slots* de um frame armazenam os valores ou atributos associados a um conceito. Por exemplo, um frame para o conceito “Carro” pode incluir *slots*, como “Marca”, “Modelo”, “Cor”, “Ano”, “Tipo de Combustível”, e “Número de Portas”.

- **Nome do slot:** cada *slot* tem um nome que indica o tipo de informação que ele armazena, como “Marca”, “Modelo” e “Cor” em um frame de “Carro”.
- **Valor do slot:** o *slot* armazena um valor específico ou uma referência a outro frame. Por exemplo, no *slot* “Marca”, o valor poderia ser “Toyota” ou, se o *slot* se refere a uma entidade complexa, poderia apontar para outro frame.
- **Facetas:** *slots* podem ter facetas, que são informações adicionais sobre o *slot*, como valores-padrão, restrições ou condições. Por exemplo, uma faceta pode indicar que o *slot* “Ano” deve conter um valor numérico entre 1886 e o ano atual.

▪ Herança:

Frames podem herdar propriedades de outros frames. Por exemplo, um frame “Veículo” pode ter *slots* básicos, como “Marca” e “Ano”, enquanto frames mais específicos, como “Carro” ou “Caminhão”, podem herdar esses *slots* e adicionar outros específicos ao tipo de veículo.

- **Superframe:** um frame pode herdar atributos de uma entidade mais geral chamada superframe. Por exemplo, o frame “Carro” pode herdar *slots*, como “Número de Rodas” e “Fonte de Energia”, do superframe “Veículo”.
- **Subframes:** um frame pode ter subframes que herdam suas características, mas com informações mais detalhadas ou específicas. Por exemplo, “Caminhão” e “Motocicleta” poderiam ser subframes de “Veículo”.

Na Figura 4, é exposto um exemplo de um frame mostrando como os *slots* e suas facetas se organizam dentro de um quadro, bem como as possíveis relações de herança entre diferentes frames.



Figura 4 - Estrutura de frames representando um “Carro” e um “Sedan” / Fonte: o autor.

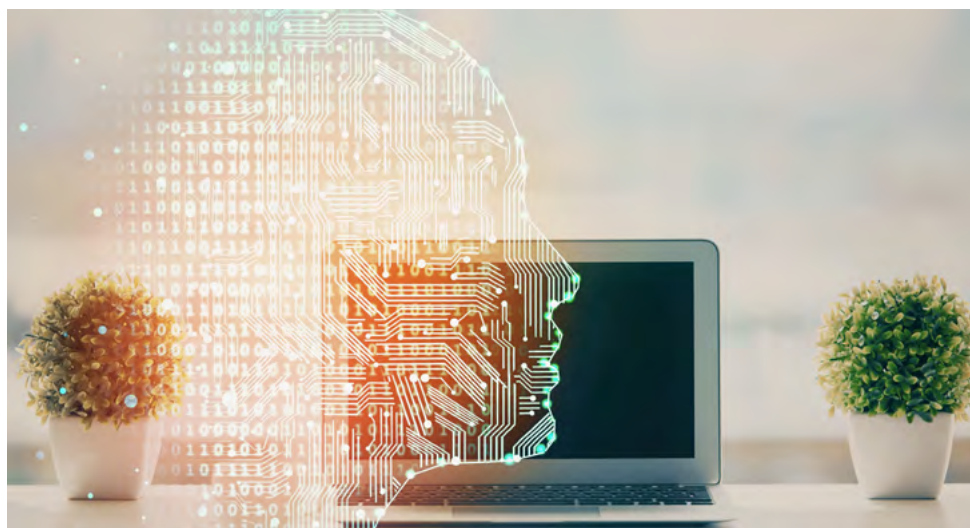
Descrição da Imagem: é exibido um diagrama com dois quadros principais. O quadro superior, “Frame: Carro,” lista atributos, como Marca (Toyota), Modelo (Corolla), Ano (2020), Tipo de Combustível (Gasolina), Cor (Prata), Número de Portas (4) e Consumo (12 km/l). O quadro inferior, “Frame: Sedan,” herda esses atributos e adiciona Capacidade do Porta-malas (470 litros). Fim da descrição.

Frames são amplamente utilizados em sistemas de IA para lidar com informações estruturadas e repetitivas. Por exemplo, em sistemas de configuração de produtos, como computadores, frames permitem personalizar características, como “Processador” e “Memória RAM”.

Ontologias

As ontologias são uma forma avançada e poderosa de representação de conhecimento em Inteligência Artificial (IA). Elas proporcionam uma maneira robusta de organizar, estruturar e interligar informações, sendo especialmente valiosas em sistemas que requerem uma visão integrada e interconectada de várias áreas de conhecimento (Luger, 2021). Ao definir um conjunto de conceitos e as relações entre eles, uma ontologia cria uma representação detalhada e coerente de um domínio específico, facilitando a compreensão e a utilização do conhecimento armazenado.

As ontologias são constituídas por três elementos fundamentais: classes (ou conceitos), propriedades (ou relações) e instâncias (ou exemplos específicos). Esses elementos interagem de maneira complexa para criar uma representação detalhada e rica de um domínio específico (Luger, 2021).



- **Classes:** as classes são categorias ou grupos de objetos, eventos ou ideias em um domínio que definem características comuns entre seus membros (Russell; Norvig, 2013). Elas são organizadas em hierarquias, com classes gerais, como “Animal”, e subclasses específicas, como “Mamífero”. Cada classe tem atributos que definem as propriedades, como “Raça” e “Tamanho” na classe “Cão”.

- **Propriedades:** as propriedades descrevem as conexões entre classes e instâncias, podendo ser hierárquicas (“é-um”) ou associativas (“tem-parte”) (Russell; Norvig, 2013). Elas permitem a construção de hierarquias e a definição de relações entre conceitos, como “Cão” sendo um “Mamífero” ou um “Carro” tendo “Rodas”.
- **Instâncias:** as instâncias são exemplos concretos de uma classe, exemplificando os tipos definidos pelas classes (Russell; Norvig, 2013). Por exemplo, “Rex” é uma instância da classe “Cão”, com atributos específicos, como “Raça: Pastor Alemão”. Instâncias herdam todas as características e restrições das respectivas classes.

Na Figura 5, é apresentada uma ontologia que organiza o conhecimento sobre animais, mostrando uma estrutura hierárquica que parte da classe geral “Animal” e se ramifica em subclasses, como “Mamífero”, “Ave” e “Réptil”.

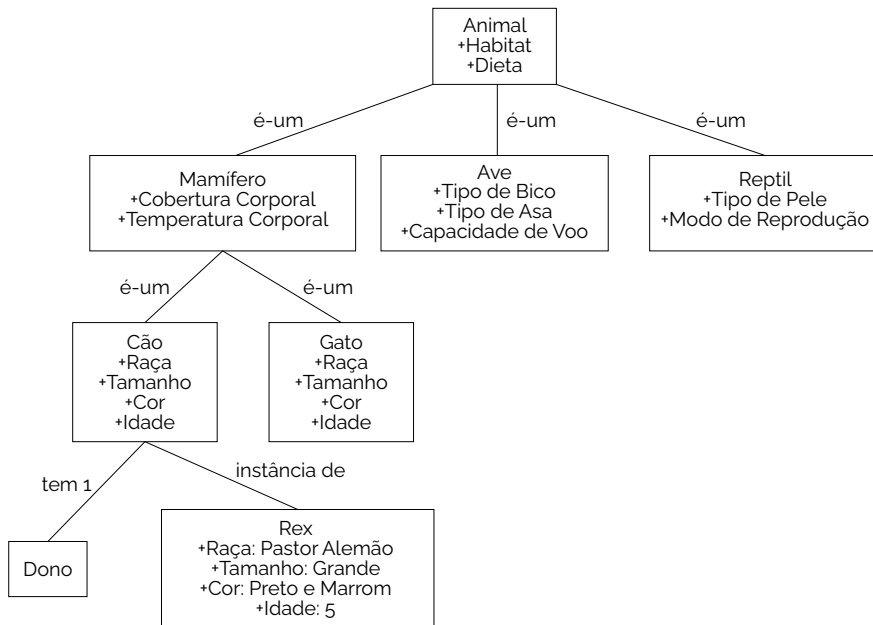


Figura 5 – Ontologia de animais mostrando a hierarquia de classes (animal, mamífero, ave e réptil) e as respectivas relações, incluindo atributos específicos e uma instância do conceito “Cão” chamada “Rex”. Fonte: o autor.

Descrição da Imagem: trata-se de um fluxograma que categoriza diferentes tipos de animais e as respectivas características.

No topo, há uma categoria única chamada “Animal”, que se ramifica em três subcategorias: “Mamífero”, “Ave” e “Réptil”. Cada uma dessas subcategorias se divide em atributos específicos, como “+ TemperaturaCorporal” para mamíferos, “+ TipoDeBico” para aves e “+ TipoDePele” para répteis. Mais abaixo, na categoria de mamíferos, há dois exemplos: “Cão” e “Gato”, cada um com atributos adicionais, como “+ Raça”, “+ Cor” e “+ Idade”. Há também uma instância específica sob a categoria de cães, que menciona um cão chamado Rex: um pastor alemão de cor preta e marrom com cinco anos de idade. O fluxograma inclui outros elementos como pontos de decisão indicando se um animal é doméstico ou não (“+ Doméstico”), o habitat (“+ Habitat”), a dieta (“+ Dieta”), o tipo de vaso (“+ TipoDeVaso”) para aves, o tipo de reprodução (“+ ModoDeReprodução”) para répteis, entre outros. Fim da descrição.

Lógica proposicional e de predicados

A **lógica proposicional** e a **lógica de predicados** são métodos formais essenciais para a representação de conhecimento em sistemas de Inteligência Artificial (IA) (Russell; Norvig, 2013). Eles permitem realizar inferências lógicas precisas, o que é fundamental para sistemas que precisam tomar decisões baseadas em regras explícitas e bem definidas.

Lógica proposicional

A lógica proposicional trabalha com **proposições simples** que podem ser **verdadeiras ou falsas**. Essas proposições são combinadas utilizando conectivos lógicos, como **AND** (E), **OR** (OU), **NOT** (NÃO), entre outros, para formar proposições compostas (Russell; Norvig, 2013). Essas proposições compostas são avaliadas conforme as regras da lógica proposicional. Por exemplo, considere a proposição “Se está chovendo, então, a rua está molhada”. Essa proposição pode ser expressa formalmente e avaliada como verdadeira ou falsa com base em observações.

Lógica de predicados

A lógica de predicados é uma **extensão da lógica proposicional**. Ela introduz **variáveis** e **quantificadores** (como “todos” ou “alguns”), além de **predicados**, que são funções que retornam verdadeiro ou falso (Russell; Norvig, 2013). Isso torna a lógica de predicados mais expressiva e capaz de representar relações mais complexas entre entidades. Por exemplo, a afirmação “Todos os pássaros têm asas” pode ser representada, em lógica de predicados, como “Para todo x, se x é um pássaro, então, x tem asas”.

GRAFOS E LÓGICA NA REPRESENTAÇÃO E INFERÊNCIA EM IA

A **teoria dos grafos** e a **lógica** são duas abordagens matemáticas fundamentais que, quando combinadas, constituem a espinha dorsal da representação de conhecimento e da realização de inferências em Inteligência Artificial (IA). Juntas, essas duas disciplinas fornecem uma estrutura poderosa que permite aos sistemas inteligentes não apenas armazenar e organizar informações de maneira eficiente, mas também deduzir novos conhecimentos, tomar decisões informadas e resolver problemas complexos com base em raciocínios lógicos (Russell; Norvig, 2013).

Teoria dos grafos: representação estruturada do conhecimento

A **teoria dos grafos** é uma ferramenta matemática que modela a relação entre diferentes entidades por meio de grafos. Um grafo é composto por **vértices** (ou nós), que representam entidades ou conceitos, e **arestas** (ou arcos), que representam as relações entre esses vértices (Boaventura Netto; Jurkiewicz, 2021). Para um melhor entendimento, considere o seguinte grafo:

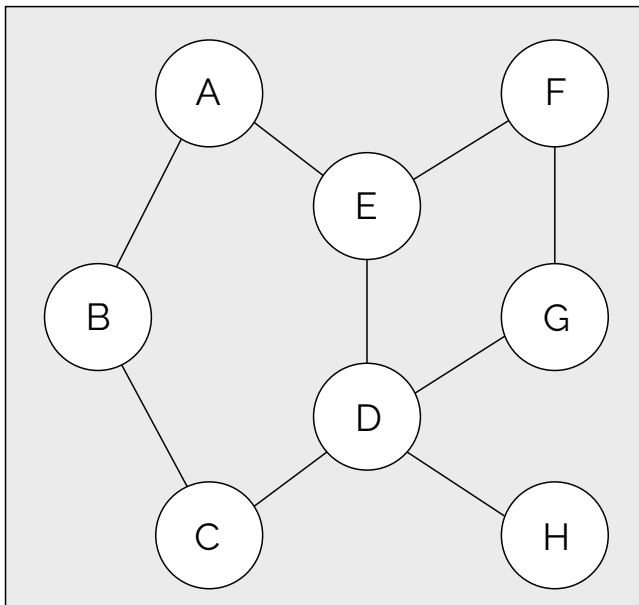


Figura 6 – Exemplo de grafo /
Fonte: adaptada de Castillo,
Gutierrez e Hadi (1997).

Descrição da Imagem: é exibido um grafo, no qual os vértices são representados por círculos rotulados com letras (A, B, C, D, E, F, G e H), e as arestas são representadas por linhas que conectam pares de vértices. Fim da descrição.

A partir do grafo apresentado na Figura 6, podemos explorar as propriedades fundamentais de um grafo, que são essenciais para a compreensão e a aplicação.

Dois vértices são considerados **adjacentes** quando estão diretamente conectados por uma aresta (Castillo; Gutierrez; Hadi, 1997). Essa propriedade é fundamental para identificar conexões imediatas no grafo. Por exemplo, no grafo apresentado na Figura 6, o vértice **A** é adjacente a **B** e **E**. Essas conexões diretas são utilizadas em diversas aplicações, como encontrar vizinhos próximos em redes sociais ou analisar rotas de menor custo em sistemas de transporte.

O **grau de um vértice** corresponde ao número de arestas conectadas a ele (Boaventura Netto; Jurkiewicz, 2021). Em grafos não direcionados, o grau é a soma de todas as conexões do vértice, enquanto, em grafos direcionados, é dividido em **grau de entrada** e **grau de saída**. No grafo apresentado na Figura 6, o vértice **A** possui grau 2, por estar conectado a **B**, e **E** e o vértice **D** possui grau 4, pois está conectado a **C**, **E**, **G** e **H**. O grau de um vértice indica a centralidade desse vértice no grafo (Castillo; Gutierrez; Hadi, 1997). Vértices com maior grau, como **D** e **E**, são mais importantes, pois têm mais conexões e, consequentemente, maior influência ou acessibilidade.

Um **caminho** em um grafo é uma sequência de vértices conectados por arestas (Castillo; Gutierrez; Hadi, 1997). Em outras palavras, ele representa a rota para ir de um vértice a outro dentro da estrutura. Por exemplo, no grafo fornecido, um caminho de **A** a **H** seria $A \rightarrow E \rightarrow D \rightarrow H$.

Em IA, grafos estruturam dados para identificar e manipular **interconexões** entre elementos. Por exemplo, em redes de transporte, vértices são estações, e as arestas são as rotas. Grafos são essenciais para organizar e analisar grandes volumes de dados, permitindo operações, como busca por caminhos curtos.

Integração entre grafos e lógica

A **integração entre a teoria dos grafos e a lógica** possibilita que sistemas de Inteligência Artificial (IA) vão além do simples armazenamento e organização de dados, permitindo que eles raciocinem de forma lógica e eficiente sobre essas informações (Russell; Norvig, 2013). Essa combinação permite que os sistemas modelem a estrutura de problemas complexos com grafos, enquanto a lógica é aplicada para deduzir soluções ou prever comportamentos com base nessa estrutura (Luger, 2021).

Essa combinação é extremamente poderosa, pois, enquanto os grafos estruturam as informações de forma clara e visualmente compreensível, a lógica fornece os mecanismos para explorar e analisar essas estruturas, gerando novos insights e possibilitando a tomada de decisões informadas (Russell; Norvig, 2013). Juntas, essas ferramentas permitem que sistemas inteligentes funcionem de maneira mais eficaz em uma ampla gama de aplicações, desde o planejamento logístico e a otimização de rotas até a análise de redes sociais e a modelagem de sistemas biológicos complexos.

PROGRAMAÇÃO EM PYTHON: SISTEMAS DE INFERÊNCIA SIMPLES

A implementação de sistemas de inferência em Python é uma etapa crucial para explorar o poder da lógica e da modelagem de conhecimento em Inteligência Artificial (IA). Essa implementação envolve a criação de uma rede semântica básica e a implementação de um motor de inferência baseado em lógica proposicional.

Em seguida, ao implementar o motor de inferência, você será capaz de aplicar regras de lógica proposicional para deduzir novos fatos a partir de proposições já conhecidas. Esse processo de inferência é uma das capacidades centrais de sistemas inteligentes, permitindo que eles não apenas processem informações, mas também gerem novos conhecimentos e tomem decisões com base em dados existentes.

Em sistemas de Inteligência Artificial (IA), frequentemente precisamos modelar relações entre diferentes conceitos ou entidades de maneira hierárquica. Por exemplo, em um sistema que classifica animais, podemos ter relações, como “Cão é um Mamífero” e “Mamífero é um Animal”. Essas relações podem ser representadas em uma estrutura de grafo, em que cada nó é um conceito (como “Cão” ou “Mamífero”) e cada aresta direcionada representa uma relação hierárquica (“é-um”). Dado um conceito específico (como “Cão”), queremos inferir a que categorias hierárquicas ele pertence. Por exemplo, dado que “Cão” é um “Mamífero” e que “Mamífero” é um “Animal”, o código precisa inferir que “Cão” também pertence às categorias “Mamífero” e “Animal”. Passos sugeridos:

1. Criação de uma rede semântica:

Inicialmente, a rede semântica foi modelada como um grafo direcionado, visto que esse tipo de estrutura é particularmente adequado para representar relações hierárquicas, como as do tipo “é-um” (Russell; Norvig, 2022). A rede construída organiza os conceitos de “Animal”, “Mamífero” e “Ave” e exemplos específicos, como “Cão” e “Pinguim”. As arestas do grafo representam a relação hierárquica “é-um”, conectando os conceitos conforme ilustrado a seguir:

```
import networkx as nx
import matplotlib.pyplot as plt

# Criando uma rede semântica simples
G = nx.DiGraph() # Grafo direcionado para representar hierarquias
G.add_edge("Animal", "Mamífero", relation="é-um")
G.add_edge("Animal", "Ave", relation="é-um")
G.add_edge("Mamífero", "Cão", relation="é-um")
G.add_edge("Mamífero", "Gato", relation="é-um")
G.add_edge("Ave", "Pinguim", relation="é-um")
```

2. Visualização da rede:

Para facilitar a compreensão das conexões, foi gerado um grafo visual utilizando o algoritmo **spring_layout**, que distribui os nós, de forma a minimizar sobreposições e maximizar a clareza visual. Além disso, as relações “é-um” foram destacadas nas arestas do grafo, permitindo que as conexões semânticas fossem facilmente identificáveis.

```
# Visualizando a rede semântica
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=3000, node_color='lightgreen', font_size=10)
```

```
labels = nx.get_edge_attributes(G, 'relation')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.show()
```

3. Implementação de um motor de inferência:

Com a rede semântica criada, vamos implementar um motor de inferência simples para identificar todas as categorias hierárquicas às quais um conceito pertence. Essa funcionalidade é alcançada por meio de uma função recursiva que percorre os predecessores do conceito-alvo, verificando se a relação “é-um” está presente.

```
# Implementando um motor de inferência baseado na rede se-
mântica

def inferir_categoria(grafo, conceito):

    categorias = []

    for predecessor in grafo.predecessors(conceito): # Usa
predecessors ao invés de ancestors

        if grafo.has_edge(predecessor, conceito) and grafo[pre-
decessor][conceito].get('relation') == 'é-um':

            categorias.append(predecessor)

            categorias.extend(inferir_categoria(grafo, predecessor))
# Chama a função recursivamente para continuar subindo na
hierarquia

    return categorias
```

4. Resultados:

Ao aplicar o motor de inferência aos conceitos “Cão” e “Pinguim”, os seguintes resultados foram obtidos:

```
# Testando a inferência lógica usando o grafo

conceito1 = "Cão"

conceito2 = "Pinguim"
```

```
# Inferir todas as categorias a que "Cão" e "Pinguim"
pertencem

categorias_cao = inferir_categoria(G, conceito1)

categorias_pinguim = inferir_categoria(G, conceito2)

print(f"{conceito1} pertence às categorias: {'\n'.join(-
categorias_cao)}")

print(f"{conceito2} pertence às categorias: {'\n'.join(-
categorias_pinguim)}")
```

O código modela uma rede semântica simples utilizando um grafo direcionado e implementando um sistema de inferência que, dado um conceito, determina todas as categorias hierárquicas superiores a que esse conceito pertence. Esse tipo de sistema é útil para estruturar e raciocinar sobre relações hierárquicas em sistemas de conhecimento, como taxonomias, ontologias ou classificações biológicas.



EM FOCO

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

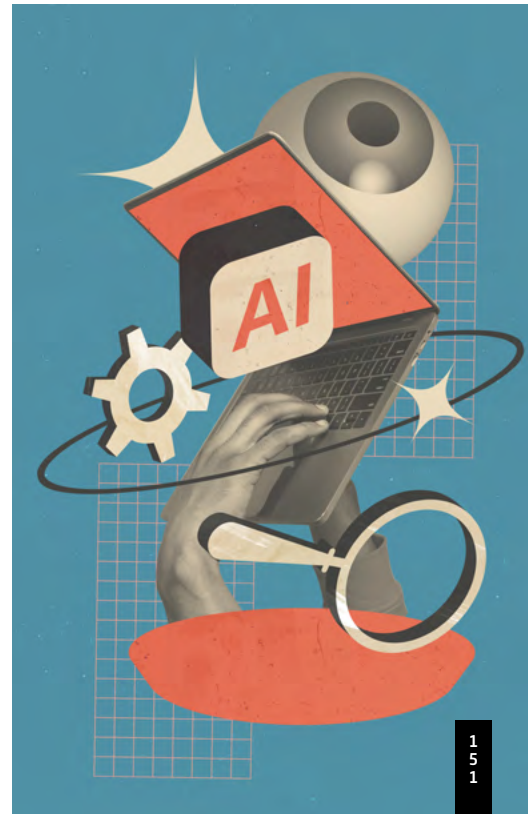
A exploração dos métodos para representar o conhecimento em Inteligência Artificial (IA) por meio da teoria dos grafos e da lógica, associada à programação em Python, é essencial para o **desenvolvimento profissional** em um ambiente cada vez mais orientado pela tecnologia. Essas ferramentas permitem resolver problemas complexos de forma estruturada e eficiente, sendo fundamentais para a criação de sistemas inteligentes e para o suporte à decisão em diversas áreas, como desenvolvimento de software, Inteligência Artificial, engenharia de dados e ciência de dados.

A modelagem de conhecimento utilizando grafos, combinada com a lógica proposicional e de predicados, fornece uma base sólida para a organização e a inferência de dados complexos. Python, com as poderosas bibliotecas, facilita a implementação de sistemas de inferência que automatizam processos e otimizam a tomada de decisões. Essa capacidade de representar e manipular conhecimento é crucial para o desenvolvimento de soluções eficazes em um mercado que valoriza a automação e a inteligência na análise de dados.

Além disso, a integração desses métodos com tecnologias emergentes, como aprendizado de máquina e Internet das Coisas (IoT), amplia ainda mais as respectivas aplicações, permitindo a criação de sistemas híbridos que combinam inferência lógica com modelos de aprendizado adaptativo. Esse tipo de abordagem é particularmente valorizado em setores, como automação, TI e Indústria 4.0, nos quais a capacidade de integrar conhecimento de diferentes fontes e inferir novos insights em tempo real podem ser grandes diferenciais competitivos.

À medida que a tecnologia avança, a habilidade de conectar teoria e prática torna-se indispensável. Profissionais que dominam a teoria dos grafos, a lógica e a **programação em Python** estarão mais bem equipados para enfrentar desafios complexos e desenvolver soluções inovadoras. Essas competências são cada vez mais requisitadas à medida que a Inteligência Artificial e as tecnologias relacionadas continuam a se expandir, exigindo um entendimento profundo tanto das bases teóricas quanto das aplicações práticas.

Como esses métodos podem impactar a sua área de atuação? Reflita sobre como você pode aplicar o conhecimento adquirido para otimizar processos e resolver problemas de forma mais eficiente. Considere também as implicações éticas, garantindo que as soluções desenvolvidas sejam justas, transparentes e compreensíveis para os usuários. Ao dominar esses conceitos, você estará bem-posicionado para atender às demandas do mercado e contribuir significativamente para o avanço da tecnologia em sua área.



AUTOATIVIDADE

1. A integração de grafos com lógica proposicional em sistemas de Inteligência Artificial é fundamental para a modelagem de relações complexas e a inferência de novos conhecimentos a partir dessas relações. Grafos são usados para representar estruturas, como redes semânticas, em que os nós representam conceitos e as arestas indicam as relações entre esses conceitos.

Qual das alternativas a seguir é uma característica correta da integração entre grafos e lógica em IA?

- a) Grafos não podem ser utilizados para inferir novos conhecimentos.
 - b) A lógica proposicional é inadequada para representar relações em grafos.
 - c) Grafos podem ser usados para representar hierarquias de conceitos, e a lógica proposicional pode inferir novas informações a partir dessas relações.
 - d) Grafos são adequados apenas para representar redes semânticas, sem suporte para lógica.
 - e) A inferência lógica em grafos é limitada a problemas de pequena escala.
2. Grafos são uma das ferramentas centrais utilizadas em Inteligência Artificial para a representação de conhecimento, especialmente em sistemas que lidam com grandes volumes de dados interconectados. Eles permitem a modelagem de relações complexas entre conceitos, o que é essencial para aplicações, como redes semânticas, análise de redes sociais e sistemas de recomendação.

Considerando o uso de grafos na representação de conhecimento em IA, analise as afirmativas a seguir:

- I - Grafos podem representar redes hierárquicas de conceitos.
- II - Grafos direcionados são usados para modelar relações assimétricas, como hierarquias.
- III - Grafos ponderados permitem incluir informações adicionais, como pesos ou custos nas relações.
- IV - Grafos são limitados a representações simétricas e não podem modelar relações complexas.

AUTOATIVIDADE

É correto o que se afirma em:

- a) I e II, apenas.
- b) II e III, apenas.
- c) I, II e III, apenas.
- d) II, III e IV, apenas.
- e) I, II, III e IV.

3. Em sistemas de Inteligência Artificial, Python oferece diversas bibliotecas que facilitam a criação de grafos e a implementação de sistemas de inferência lógica. `networkx` é uma biblioteca popular para a criação e a manipulação de grafos, enquanto `matplotlib` é frequentemente usada para visualizar esses grafos.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - O uso de Python facilita a implementação de sistemas de inferência lógica baseados em grafos.

PORQUE

II - Python permite a criação e a visualização de grafos complexos que representam relações entre conceitos em uma rede semântica.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

BOAVENTURA NETTO, P. O.; JURKIEWICZ, S. **Grafos**: introdução e prática. São Paulo: Editora Blucher, 2021.

CASTILLO, E.; GUTIERREZ, J. M.; HADI, A. S. **Expert Systems and Probabilistic Network Models**. New York: Springer-Verlag, 1997.

LUGER, G. F. **Knowing Our World**: An Artificial Intelligence Perspective. [S. l.]: Springer, 2021.

OLIVEIRA, H. C. de; CARVALHO, C. L. de. **Gestão e Representação do Conhecimento**. [S. l.: s. n.], 2008.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence**: A Modern Approach. [S. l.]: Prentice Hall, 2013.

RUSSELL, S.; NORVIG, P. **Inteligência artificial**: uma abordagem moderna. 4. ed. Rio de Janeiro: GEN; LTC, 2022.

GABARITO

1. Opção C.

Grafos são usados para representar hierarquias e relações entre conceitos, e a lógica proposicional pode inferir novos conhecimentos a partir dessas relações. As outras alternativas estão incorretas, pois subestimam a capacidade dos grafos e da lógica proposicional na modelagem e na inferência.

2. Opção C.

As afirmativas I, II e III estão corretas, pois descrevem o uso de grafos na representação de conhecimento em IA. Já a afirmativa IV está incorreta, pois grafos não são limitados a representações simétricas: eles podem modelar uma ampla variedade de relações complexas.

3. Opção A.

Python facilita a implementação de sistemas de inferência lógica baseados em grafos, porque permite a criação e a manipulação de grafos complexos que representam relações em uma rede semântica.



TEMA DE APRENDIZAGEM 6

AGENTES INTELIGENTES

MINHAS METAS

- Compreender o conceito de agentes inteligentes.
- Explorar a percepção e a atuação dos agentes.
- Modelar matematicamente o comportamento de agentes.
- Formalizar os comportamentos de agentes.
- Programar agentes em Python.
- Aplicar agentes inteligentes em problemas reais.
- Conectar conceitos teóricos às atividades práticas.

INICIE SUA JORNADA

Em um mundo onde a autonomia e a capacidade de adaptação das máquinas são cada vez mais exigidas, a criação de sistemas que percebem e interagem com o ambiente de forma inteligente é fundamental. Contudo, como podemos garantir que esses sistemas consigam interpretar informações complexas e tomar decisões eficazes em tempo real? Os agentes inteligentes, capazes de aprender e agir de forma autônoma, representam a resposta para essa demanda crescente em diversas indústrias.

Imagine um cenário em que você está desenvolvendo um sistema de gerenciamento de energia para uma cidade inteligente. Esse sistema precisa ajustar automaticamente o consumo de energia em diferentes áreas da cidade com base em dados, tais como o clima, a demanda energética e os padrões de uso. Como você configuraria um agente inteligente para monitorar essas variáveis e otimizar o uso de energia, evitando desperdícios e garantindo a eficiência? Quais estratégias e algoritmos você usaria para que o agente tome decisões rápidas e precisas?

Para ilustrar melhor, pense em como você já abordou problemas que exigiam **adaptação rápida a novas condições**, como a reorganização de um projeto devido a mudanças inesperadas. Da mesma forma, os agentes inteligentes utilizam algoritmos de aprendizado para analisar o ambiente e ajustar as próprias ações conforme necessário. Imagine estar no comando de um projeto que exige a criação de um agente capaz de prever falhas em uma linha de produção e tomar medidas preventivas antes que os problemas ocorram. Como você definiria o ambiente de atuação do agente e as regras que ele deve seguir para tomar decisões eficazes? Ao aplicar esses conceitos de forma prática, você poderá visualizar como os agentes inteligentes transformam problemas complexos em soluções automatizadas.

Agora, pense em um cenário no qual você está trabalhando em um assistente virtual que precisa interagir com os usuários de forma natural e eficiente. Como você aplicaria os princípios de agentes inteligentes para que o assistente entenda as solicitações dos usuários, aprenda com cada interação e ofereça respostas cada vez mais precisas? Desenvolva um estudo de caso em que o assistente virtual melhora continuamente o próprio desempenho, considerando a coleta de dados, o processamento da linguagem natural e a adaptação ao comportamento do usuário. Esse exercício prático te permitirá compreender como os agentes inteligentes podem ser aplicados para criar interações mais eficientes e personalizadas.

**PLAY NO CONHECIMENTO**

Neste episódio do nosso podcast, mergulhamos no universo dos agentes inteligentes e nas respectivas aplicações, que vão além do que você estudará neste tema. Exploramos como essas tecnologias estão sendo utilizadas para criar cidades inteligentes, automatizar processos industriais e revolucionar o atendimento ao cliente. Além disso, discutimos os desafios éticos e as tendências futuras da IA. Venha descobrir como os agentes inteligentes estão redefinindo a nossa interação com o mundo! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para que você compreenda e aplique os conceitos de agentes inteligentes, é importante revisar alguns conteúdos essenciais. Primeiro, relembre os algoritmos de busca, como a busca em largura e a busca A*, que são fundamentais para a tomada de decisão em ambientes dinâmicos. Além disso, reforçar os fundamentos de programação em Python, incluindo a manipulação de estruturas de dados, será essencial para a implementação desses agentes. Saiba mais no link a seguir.

<https://medium.com/@rsorage/grafos-3-entendendo-e-implementando-o-algoritmo-de-pesquisa-em-largura-469e9c3544bd>

DESENVOLVA SEU POTENCIAL

O QUE SÃO AGENTES INTELIGENTES?

Vamos começar a explorar em detalhes o conceito de agentes inteligentes e a respectiva importância no cenário atual. Um agente inteligente é um sistema capaz de perceber o ambiente em que está inserido, processar essas informações e agir de forma autônoma para alcançar os próprios objetivos. Imagine um robô que detecta obstáculos pelo caminho e escolhe a melhor rota para chegar ao destino

desejado. Assim como esse robô, os agentes podem se adaptar a diferentes situações e tomar decisões que otimizam o desempenho em suas tarefas.

Contudo, o que torna um agente verdadeiramente inteligente? Vamos explorar essas características fundamentais, diferenciando os tipos de agentes, como os reativos, deliberativos e híbridos, e discutindo como eles são utilizados em aplicações reais, como assistentes virtuais e sistemas de recomendação.

Os agentes inteligentes são componentes fundamentais da Inteligência Artificial (IA) concebidos para atuar autonomamente em ambientes complexos. Eles têm a capacidade de perceber o ambiente ao redor, processar informações e tomar decisões, influenciando o ambiente por ações específicas.

Em outras palavras, Russell e Norvig (2021) definem um **agente inteligente** como qualquer entidade capaz de perceber o próprio ambiente por meio de sensores e agir sobre ele por meio de atuadores. Observe como ocorre essa interação na Figura 1. Essa atuação é orientada para alcançar metas definidas, reagindo conforme as percepções adquiridas. Por exemplo, um agente humano tem olhos, ouvidos e outros órgãos como sensores, enquanto mãos, pernas, boca e outras partes do corpo atuam como atuadores.



Já um agente robótico pode utilizar câmeras e sensores infravermelhos para captar informações do ambiente e motores para realizar as próprias ações. Por outro lado, um agente de software recebe informações mediante sequências de teclas, conteúdo de arquivos e pacotes de rede, e atua no ambiente exibindo dados na tela, gravando em arquivos e transmitindo pacotes pela rede.

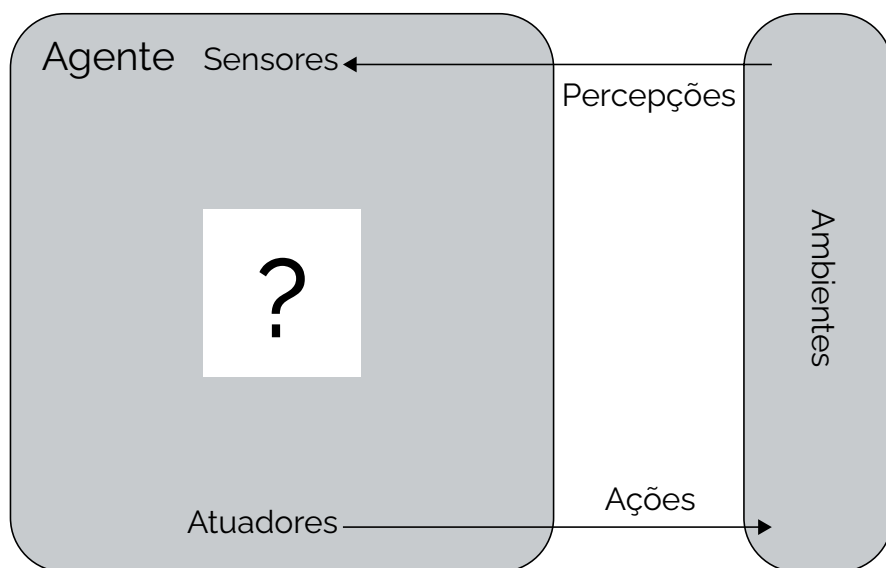


Figura 1 - Agentes interagem com ambientes por meio de sensores e atuadores
Fonte: adaptada de Russell e Norvig (2021).

Descrição da Imagem: é exibido um diagrama que representa a interação entre um agente e o respectivo ambiente. O diagrama é dividido em duas seções principais por uma linha vertical. Parte esquerda (agente): há um quadrado com um ponto de interrogação dentro, indicando o agente cujos detalhes internos não são especificados. Três setas estão conectadas a esse quadrado: uma seta é proveniente da esquerda, rotulada como "Sensores", apontando para um ponto de interrogação. Outra seta sai da parte inferior do quadrado, rotulada como "Atuadores". Uma terceira seta faz um loop dos "Atuadores" de volta para os "Sensores", sugerindo um ciclo de feedback. Parte direita (ambiente): essa seção é rotulada como "Ambiente". Duas setas conectam essa seção com o lado do agente: uma seta aponta do "Ambiente" para os "Sensores", indicando a entrada no agente como percepções. Outra seta aponta dos "Atuadores" para o "Ambiente", indicando a saída como ações. Fim da descrição.

As características que distinguem um agente inteligente estão descritas na Tabela 1.

CARACTERÍSTICA	DESCRIÇÃO	EXEMPLO
AUTONOMIA	Capacidade do agente de operar sem intervenção contínua, tomando decisões com base em suas percepções sobre o ambiente e o estado interno.	Um robô de limpeza que decide quando e como limpar uma sala com base nas informações dos próprios sensores.
	Sensibilidade a mudanças no ambiente, ajustando o comportamento para alcançar os objetivos.	Um carro autônomo que reage a sinais de trânsito, pedestres e outros veículos para manter um trajeto seguro.
	Capacidade de tomar iniciativas e se antecipar a eventos futuros, planejando e escolhendo ações que maximizem a chance de alcançar os objetivos.	Um sistema de recomendação que sugere conteúdos ao usuário, antecipando os interesses com base no histórico.
	Habilidade de incorporar mecanismos de aprendizado para melhorar o desempenho ao longo do tempo com base em experiências passadas.	Um assistente virtual que aprende a responder melhor às solicitações do usuário, aprimorando a precisão das respostas.

Tabela 1 - Descrições de agentes inteligentes / Fonte: o autor.

A interação dos agentes com o ambiente é fundamental para o respectivo funcionamento. Diferentes tipos de ambientes influenciam a forma como um agente age e toma decisões (Russell; Norvig, 2021):

- **Ambiente Determinístico *versus* Estocástico:** em ambientes determinísticos, as ações do agente levam a resultados previsíveis (como jogo de xadrez).

A interação dos agentes com o ambiente é fundamental

Já em ambientes estocásticos, há incerteza nos resultados das ações, como no mercado financeiro.

- **Ambiente Estático *versus* Dinâmico:** em ambientes estáticos, o estado não muda enquanto o agente decide (como um jogo por turnos). Em ambientes dinâmicos, o estado pode mudar continuamente, como nas ruas em que um carro autônomo dirige.
- **Ambiente Acessível *versus* Inacessível:** em um ambiente acessível, o agente pode obter todas as informações relevantes para tomar decisões precisas. Em um ambiente inacessível, algumas informações estão indisponíveis, como um robô explorando um planeta desconhecido.

EXPLORANDO A PERCEPÇÃO E A ATUAÇÃO DOS AGENTES

Para um agente inteligente tomar decisões adequadas, ele precisa perceber o ambiente ao redor e agir sobre ele. A percepção se refere às entradas sensoriais que o agente recebe em um dado instante (Russell; Norvig, 2021). Essa percepção pode incluir dados sobre temperatura, umidade ou até informações de tráfego em tempo real. Os agentes utilizam sensores para coletar essas informações, construindo uma “história” completa de tudo o que já perceberam. A escolha da ação em um determinado instante depende da sequência inteira de percepções acumuladas ao longo do tempo.

Imagine um robô que está explorando um ambiente desconhecido. Ele começa no ponto A e percebe um obstáculo à frente. Em seguida, ele se move para o ponto B, onde percebe um caminho livre à direita. A escolha de qual direção seguir não depende apenas da percepção atual (caminho livre à direita), mas também de todas as percepções anteriores, incluindo os obstáculos que encontrou no passado.

Em termos matemáticos, o comportamento do agente é descrito por uma função de agente, que mapeia qualquer sequência de percepções para uma ação específica (Russell; Norvig, 2021). Podemos imaginar a função de agente como uma tabela que relaciona todas as possíveis sequências de percepções do agente com as ações que ele deve tomar. Para a maioria dos agentes, essa tabela seria muito grande ou até mesmo infinita, exceto se colocarmos um limite nas sequências que queremos considerar. Internamente, em um agente artificial, essa função é implementada pelo programa do agente, que executa em um sistema físico.



Essa função é um mecanismo que mapeia cada sequência possível de percepções em uma ação. Pense na função do agente como uma grande tabela, em que cada linha representa uma sequência de percepções acumuladas ao longo do tempo, e cada coluna indica a ação correspondente que o agente deve realizar (Russell; Norvig, 2021). Por exemplo, no caso do robô que explora um ambiente, a função do agente poderia incluir:

- Se a sequência de percepções indica que o robô encontrou um obstáculo, ele deve mudar de direção.
- Se o caminho estiver livre, ele deve continuar em frente.

Para ilustrar esses conceitos, vamos usar o exemplo do **mundo do aspirador de pó**. Imagine um ambiente muito simples, com apenas dois locais: os quadrados A e B (Figura 2). O agente aspirador de pó pode **perceber** em qual **quadrado está** e se há sujeira presente. Ele tem quatro possíveis ações: mover-se para a esquerda, mover-se para a direita, aspirar a sujeira ou não fazer nada.

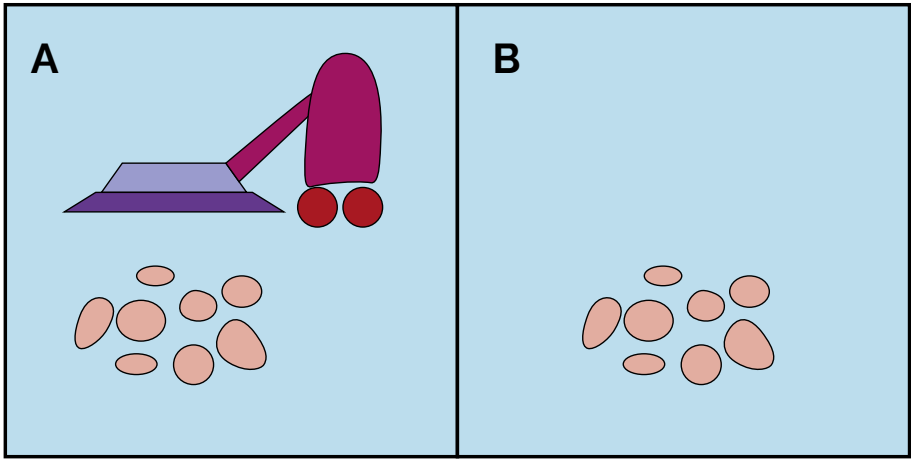


Figura 2 - Mundo do aspirador de pó / Fonte: adaptada de Russell e Norvig (2021).

Descrição da Imagem: é exibido um ambiente simples com dois locais rotulados como quadrados A e B. Quadrado A: à esquerda, há um aspirador de pó com corpo roxo e rodas vermelhas. O bocal do aspirador está estendido em direção ao centro do quadrado. Na parte inferior do quadrado A, há várias partículas de sujeira em forma oval espalhadas. Quadrado B: contém partículas de sujeira semelhantes às do quadrado A, mas não há aspirador de pó presente. Fim da descrição.

Uma função de agente para esse aspirador pode ser definida da seguinte forma:

- Se o quadrado atual estiver sujo, o aspirador deve aspirar.
- Se o quadrado atual estiver limpo, ele deve se mover para o outro quadrado.

Podemos tabular parcialmente essa função, relacionando as percepções possíveis (local atual e presença de sujeira) às ações a serem tomadas. A tabela a seguir nos dá uma visão externa do comportamento do agente. Assim, observe o relacionamento das percepções e ações na Tabela 2.

PERCEPÇÃO	AÇÃO
Está em A, A está sujo	Aspirar
Está em A, A está limpo	Mover-se para a direita

Está em B, B está sujo	Aspirar
Está em B, B está limpo	Mover-se para a esquerda

Tabela 2 – Relacionamento das percepções e ações / Fonte: adaptada de Russell e Norvig (2021).

Essa tabela é uma forma abstrata de descrever o comportamento do agente: ele “percebe” o ambiente e escolhe uma ação com base nessa percepção. No entanto, na prática, o aspirador precisa de um **programa do agente** — um código que implementa essa lógica, processando as percepções em tempo real e escolhendo a ação a ser executada. A **função do agente** é uma descrição abstrata e matemática do comportamento, como observamos na tabela anterior. No entanto, o que realmente faz o agente funcionar é o **programa do agente**, que é a implementação prática dessa função. O programa recebe as percepções do ambiente e calcula a ação a ser tomada com base nas regras estabelecidas.

Por exemplo, o programa do nosso aspirador de pó poderia ser algo assim em pseudocódigo:

Se o quadrado atual está sujo:

Aspirar

Caso contrário:

Se estiver em A:

Mover-se para B

Se estiver em B:

Mover-se para A

Esse código representa a lógica que o aspirador seguirá em tempo real, agindo conforme as percepções que recebe a cada instante. Após decidir qual ação tomar com base nas próprias percepções, o agente precisa **atuar** no ambiente. A atuação é realizada por meio de atuadores, que são os componentes responsáveis por executar as ações. No caso do aspirador de pó, os atuadores podem incluir motores que permitem que ele se mova de um quadrado para outro e um motor de sucção para aspirar a sujeira.

A sequência completa, portanto, é:

1. O agente **percebe** o ambiente por meio de sensores.
2. O programa do agente decide a ação com base na função do agente.
3. O agente **atua** no ambiente usando seus atuadores.

Compreender essa sequência é fundamental para o desenvolvimento de agentes inteligentes. A percepção define o que o agente “sabe”, a função e o programa do agente determinam como ele “pensa”, enquanto a atuação revela como ele “age”.



INDICAÇÃO DE FILME

Eu, Robô

Esse filme é baseado nas histórias de Isaac Asimov e aborda temas que se relacionam diretamente com agentes inteligentes, robótica e tomada de decisões autônomas. No filme, os robôs são projetados com Inteligência Artificial para interagir com humanos e seguir um conjunto de regras rígidas, conhecidas como as “Três Leis da Robótica”. Ao assistir, reflita sobre como os agentes do filme tomam decisões, percebem o ambiente e interagem com os humanos, explorando os dilemas éticos e desafios da IA em ambientes dinâmicos.



MODELAGEM MATEMÁTICA DO COMPORTAMENTO DE AGENTES

Uma das etapas mais importantes na construção de agentes inteligentes é a modelagem de comportamento. Você aprenderá a representar e a gerenciar diferentes estados do ambiente e do próprio agente utilizando funções de percepção e ação. Vamos discutir como os agentes mapeiam as próprias percepções em ações por meio de expressões matemáticas e algoritmos. Em seguida, aplicaremos o conceito de máquinas de estados finitos para modelar comportamentos complexos, ilustrando com exemplos práticos de navegação em ambientes.

Representação de estados

A representação de estados é a base para a modelagem de agentes inteligentes. Um **estado** é uma descrição do ambiente em um dado momento, incluindo todas as informações relevantes que o agente pode usar para decidir as próprias ações (Luger, 2021). A maneira como os estados são definidos depende do problema que o agente precisa resolver.

EXEMPLIFICANDO

Exemplo: no caso do aspirador de pó, um estado pode incluir:

- A posição do aspirador (se ele está no quadrado A ou B).
- A condição dos quadrados (se estão limpos ou sujos).

Nesse exemplo, um estado poderia ser representado como uma dupla (**posição**, **condição**), em que:

- **posição** é "A" ou "B".
- **condição** é uma lista indicando se cada quadrado está sujo ou limpo, por exemplo, [(A: sujo), (B: limpo)].

Os agentes usam essas representações de estado para determinar qual ação tomar em cada situação. A cada nova percepção, eles atualizam o estado interno, o que os ajuda a planejar as ações futuras.

Função de percepção e ação

A função de percepção e ação é um dos elementos centrais na modelagem do comportamento do agente (Russell; Norvig, 2021). Essa função estabelece o modo como o agente deve reagir a uma determinada sequência de percepções, mapeando as percepções para ações específicas. Em termos matemáticos, ela é descrita como uma função:

$$f:P \rightarrow A$$

Em que:

- P é o conjunto das possíveis percepções do agente.
- A é o conjunto das ações possíveis.

Exemplo: no mundo do aspirador de pó, a função de percepção e ação pode ser definida da seguinte forma:

- $f(A, \text{sujo}) = \text{aspirar}$
- $f(A, \text{limpo}) = \text{mover para a direita}$
- $(B, \text{limpo}) = \text{mover para a esquerda}$

Essa função descreve o comportamento do agente, dizendo o que ele deve fazer dependendo das percepções atuais. A implementação dessa lógica é feita mediante um programa de agente, que processa as percepções e escolhe as ações correspondentes.

Máquinas de estados finitos

As máquinas de estados finitos (FSM - Finite State Machines) são modelos matemáticos que permitem representar o comportamento de agentes em ambientes com um número limitado de estados e transições (Luger, 2021). Em uma FSM, o agente muda de estado com base em percepções e executa as ações de acordo com essas transições.

Elementos principais de uma máquina de estados finitos (Luger, 2021):

CONJUNTO DE ESTADOS
Descreve todas as possíveis condições em que o agente pode estar.
CONJUNTO DE TRANSIÇÕES
Regras que determinam quando o agente muda de um estado para outro.
FUNÇÃO DE TRANSIÇÃO
Define para qual estado o agente deve mudar quando uma percepção específica é recebida.
AÇÕES
Atividades que o agente realiza ao entrar em um estado ou durante uma transição.

Exemplo: para o aspirador de pó, a FSM pode ser representada como:

- Estados: “No quadrado A”, “No quadrado B”.
- Transições:
 - Se o estado é “No quadrado A” e a percepção é “sujo”, então, a ação é «aspirar».
 - Se o estado é “No quadrado A” e a percepção é “limpo”, então, a ação é «mover para a direita» e transição para o estado «No quadrado B».
 - E assim por diante para os outros estados e transições.

Esse modelo é especialmente útil para sistemas com um conjunto finito e bem definido de estados, em que o comportamento pode ser detalhado em cada um desses estados.

Outros métodos

Além das abordagens clássicas de modelagem, existem outros métodos que podem ser usados para modelar o comportamento dos agentes, especialmente em ambientes mais complexos (Taulli, 2019):

- **Aprendizado de máquina:** para ambientes com estados complexos ou desconhecidos, os agentes podem usar técnicas de aprendizado de máquina, como redes neurais, para aprender a mapear percepções em ações com base em exemplos e dados históricos (Taulli, 2019).
 - Exemplo: em sistemas de recomendação, um agente pode usar aprendizado supervisionado para identificar padrões de comportamento do usuário e sugerir produtos.
- **Redes de Markov:** para agentes que operam em ambientes estocásticos, em que o resultado de uma ação não é totalmente previsível, as redes de Markov podem ser utilizadas para modelar as transições de estados, considerando a probabilidade de ocorrência de cada evento (Russell; Norvig, 2021).
 - Exemplo: um robô explorando um ambiente desconhecido pode utilizar Processos de Decisão de Markov (MDPs) para decidir a melhor ação com base nas probabilidades de alcançar um estado desejado.

Esses métodos oferecem flexibilidade e sofisticação na modelagem de agentes, permitindo que eles lidem com ambientes mais complexos e tomem decisões adaptativas e inteligentes.

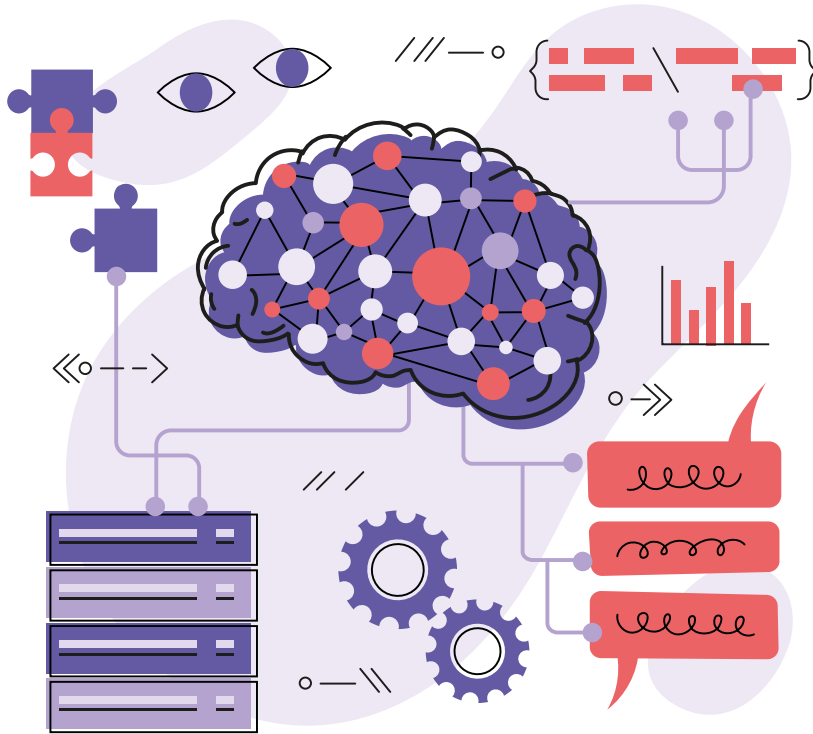
A modelagem matemática do comportamento de agentes é fundamental para o respectivo funcionamento. A representação de estados define o contexto em que o agente atua: a função de percepção e ação descreve como ele deve reagir às entradas; e modelos, como máquinas de estados finitos e aprendizado de máquina, fornecem as ferramentas necessárias para implementar comportamentos que variam desde tarefas simples até problemas complexos em ambientes imprevisíveis.

FORMALIZAÇÃO DE COMPORTAMENTOS

Para que um agente inteligente tome decisões de maneira eficaz, é essencial que o comportamento dele seja formalizado e estruturado. A formalização define o modo como o agente deve raciocinar e atuar com base nas próprias percepções e no conhecimento do ambiente. Nesta seção, veremos como a lógica e os algoritmos são utilizados para formalizar essas decisões, além de como os agentes podem planejar ações para alcançar objetivos complexos.

Formalização com lógica

A lógica é uma ferramenta poderosa para representar e formalizar os comportamentos de um agente. Existem dois tipos principais de lógica usados para esse propósito: a **lógica proposicional** e a **lógica de predicados**.



- **Lógica proposicional:** esse é o tipo mais simples de lógica, em que as afirmações podem ser verdadeiras ou falsas. Em termos de agentes inteligentes, podemos representar as percepções e ações do agente através de proposições (Souza, 2020).

Exemplo: para o agente aspirador de pó, podemos usar proposições, como:

- p_1 : “O quadrado A está sujo.”
- p_2 : “O aspirador está no quadrado A.”
- A partir dessas proposições, podemos criar regras lógicas que definem o comportamento do agente:
 - **Regra 1:** se p_1 for verdadeira e p_2 for verdadeira, então, a ação é «aspirar». Isso pode ser representado como:

Lógica de predicados: a lógica de predicados é mais expressiva que a lógica proposicional, pois permite representar objetos e as respectivas propriedades (Souza, 2020). Ela usa predicados, que são expressões, como $Sujo(A)$ (o quadrado A está sujo) ou $Posicao(Aspirador, A)$ (o aspirador está no quadrado A).

Exemplo: para o agente aspirador, podemos formalizar as regras de comportamento usando lógica de predicados:

- **Regra:** Se $Sujo(A)$ e $Posicao(Aspirador, A)$, então, a ação é «aspirar».

Essa formalização é útil para criar regras de comportamento claras e determinar as ações do agente em diversas situações. A lógica é particularmente eficaz em ambientes que são estáticos e onde as condições são bem definidas.

Algoritmos de tomada de decisão

Além da lógica, os agentes precisam de algoritmos específicos para tomar decisões em ambientes dinâmicos e complexos. Alguns dos principais algoritmos incluem:

- **Árvores de decisão:** uma árvore de decisão é uma estrutura que representa escolhas e as possíveis consequências (Taulli, 2019). Cada nó da árvore representa uma decisão baseada em uma condição, e as ramificações mostram os diferentes caminhos que podem ser seguidos com base nas percepções do agente.

Exemplo: uma árvore de decisão para o aspirador de pó poderia começar com a condição “O quadrado atual está sujo?”. Se a resposta for “Sim”, a ação é “aspirar”. Se for “Não”, a árvore continua para verificar em qual quadrado o aspirador está e, com base nisso, decide se ele deve se mover para a esquerda ou para a direita.

- **Sistemas de regras:** um sistema de regras é um conjunto de instruções do tipo “se-então” que orientam as ações do agente (Russell; Norvig, 2021). As regras são aplicadas sequencialmente até que uma delas seja satisfeita.

Exemplo: para o agente aspirador, o sistema de regras pode incluir:

- Regra 1: Se o quadrado atual estiver sujo, então, aspirar.
- Regra 2: Se o quadrado atual estiver limpo e for o quadrado A, então, mover para a direita.
- Regra 3: Se o quadrado atual estiver limpo e for o quadrado B, então, mover para a esquerda.

Os sistemas de regras são especialmente úteis para agentes que operam em ambientes com várias condições e nos quais as decisões dependem de um conjunto de fatores.

PROGRAMANDO AGENTES EM PYTHON

A programação de agentes inteligentes é uma forma prática de aplicar os conceitos teóricos que discutimos, como percepção, atuação, modelagem e formalização de comportamentos. Python é uma linguagem muito adequada para esse tipo de implementação devido à simplicidade, legibilidade e vasta gama de bibliotecas disponíveis para simulação, aprendizado de máquina e Inteligência Artificial.

Vamos implementar um agente básico para o exemplo do aspirador de pó, que opera em um ambiente com dois quadrados (A e B). Esse agente vai utilizar a lógica definida anteriormente: se o quadrado em que ele está estiver sujo, ele irá aspirar; caso contrário, ele se moverá para o outro quadrado.

Passo 1: Definir o ambiente

Primeiro, configuramos o ambiente com dois quadrados, A e B, que podem estar em dois estados: “sujo” ou “limpo”. Para representar isso, utilizamos um dicionário no Python. Inicialmente, o quadrado A está sujo, e o quadrado B está limpo.

```
ambiente = {'A': 'sujo', 'B': 'limpo'}
```

Passo 2: Definir o agente

O agente precisa saber a posição atual (quadrado A ou B) e perceber o estado do quadrado em que está. Ele aspirará o quadrado atual se este estiver sujo. Caso contrário, deslocar-se-á para o outro quadrado. Essa lógica está implementada na função a seguir:

```
def aspirador_agente(local_atual, ambiente):
```

```
    if ambiente[local_atual] == 'sujo':
```

```
        print(f"Aspirando o quadrado {local_atual}")
```

```
        ambiente[local_atual] = 'limpo'
```

```
    else:
```

```

if local_atual == 'A':

    print("Movendo-se para o quadrado B")

    local_atual = 'B'

else:

    print("Movendo-se para o quadrado A")

    local_atual = 'A'

return local_atual

```

Passo 3: Executar o programa do agente

Agora, vamos criar uma simulação para executar o agente por algumas etapas. Para a execução da simulação, o agente começa posicionado no quadrado A. A cada etapa, o ambiente é atualizado, de forma que ambos os quadrados (A e B) são considerados sujos antes do agente agir. Em seguida, o agente percebe o estado do ambiente, toma as ações necessárias e atualiza a própria posição. O estado do ambiente é exibido após cada ação do agente. O código a seguir realiza essa simulação por cinco etapas:

```

local_atual = 'A'

for _ in range(5):

    ambiente['A'] = 'sujo'

    ambiente['B'] = 'sujo'

```

```
print(f"Agente está no quadrado {local_atual}")
```

```
local_atual = aspirador_agente(local_atual, ambiente)
```

```
print(f"Estado do ambiente (após agir): {ambiente}")
```

```
print("-" * 30)
```

A saída vai mostrar o agente aspirando quando necessário e se movendo entre os quadrados:

```
Agente está no quadrado A
Aspirando o quadrado A
Estado do ambiente: {'A': 'limpo', 'B': 'sujo'}
-----
Agente está no quadrado B
Aspirando o quadrado B
Estado do ambiente: {'A': 'limpo', 'B': 'limpo'}
-----
Agente está no quadrado B
Movendo-se para o quadrado A
Estado do ambiente: {'A': 'limpo', 'B': 'limpo'}
-----
Agente está no quadrado A
Movendo-se para o quadrado B
Estado do ambiente: {'A': 'limpo', 'B': 'limpo'}
-----
Agente está no quadrado B
Movendo-se para o quadrado A
Estado do ambiente: {'A': 'limpo', 'B': 'limpo'}
-----
```

Ao longo da implementação, entendemos que o comportamento do agente pode ser facilmente modificado ou expandido, o que abre a possibilidade para a criação de agentes mais complexos e adaptativos. Esse é o primeiro passo para compreender como agentes mais avançados, como assistentes virtuais ou sistemas de controle em casas inteligentes, são construídos. Compreender a lógica de agentes simples, tais como o aspirador de pó, é essencial para aprofundar-se em tópicos mais avançados de Inteligência Artificial, como aprendizado por reforço e ambientes dinâmicos.

**EM FOCO**

Gostou do que discutimos até aqui? Tenho mais para conversar com você a respeito deste tema. Vamos lá? **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A compreensão e a implementação de agentes inteligentes envolvendo percepção, atuação, modelagem matemática e formalização de comportamentos são fundamentais para o desenvolvimento profissional em um mercado cada vez mais orientado pela automação e Inteligência Artificial. Essas ferramentas permitem a criação de sistemas autônomos que resolvem problemas complexos de maneira estruturada e eficiente, sendo aplicáveis em diversos campos, tais como robótica, automação industrial, logística e serviços digitais.

A modelagem de agentes inteligentes, baseada na representação de estados e na definição de funções de percepção e ação, fornece uma estrutura sólida para o desenvolvimento de sistemas que interagem com o ambiente e tomam decisões de forma autônoma. **Python**, com as bibliotecas poderosas, facilita a implementação desses agentes, possibilitando a criação de soluções que automatizam processos e apoiam a tomada de decisões. Essa capacidade de transformar conhecimento em ação é essencial em um mercado que valoriza a eficiência e a otimização de operações.

Além disso, a integração dessas técnicas com tecnologias, como aprendizado de máquina, expande ainda mais as respectivas aplicações, permitindo que os agentes aprendam e se adaptem ao longo do tempo. Em setores, como a Indústria 4.0, TI e cidades inteligentes, a habilidade de combinar lógica com adaptabilidade se torna um diferencial competitivo.

À medida que a tecnologia avança, a conexão entre teoria e prática se torna cada vez mais indispensável. Profissionais que dominam a modelagem de agentes, a lógica formal e a programação em Python estarão mais bem equipados para enfrentar os desafios da automação e da análise de dados. Essas competências são requisitadas em um mercado que exige soluções inteligentes e inovadoras.

Pense sempre nas implicações éticas desse trabalho, garantindo que os sistemas desenvolvidos sejam justos, transparentes e compreensíveis para os usuários. Ao dominar esses conceitos, você estará preparado para atender às demandas do mercado e contribuir para o avanço da tecnologia em sua área.



AUTOATIVIDADE

1. Um agente inteligente é um sistema computacional projetado para operar em ambientes variados. Ele deve ser capaz de perceber o entorno e realizar ações que o movam em direção a um objetivo específico. Esses agentes são utilizados em diferentes contextos, como automação, sistemas de recomendação e jogos eletrônicos, sendo um dos pilares do estudo em Inteligência Artificial.

O que define um agente inteligente em um ambiente?

- a) Um sistema que reage apenas a comandos pré-programados.
 - b) Um sistema capaz de perceber o ambiente e agir para alcançar objetivos.
 - c) Um programa que processa dados e fornece análises sem interação.
 - d) Um hardware que opera sem nenhuma interação com o ambiente.
 - e) Um sistema que armazena informações, mas não toma decisões.
2. Os agentes inteligentes podem ser classificados de acordo com a forma como processam informações e tomam decisões. Alguns agentes operam de maneira simples, reagindo diretamente a estímulos do ambiente, enquanto outros possuem mecanismos mais complexos, levando em consideração diversos fatores antes de agir.

Considerando os agentes inteligentes, analise as afirmativas a seguir:

- I - Um agente inteligente sempre tem autonomia para agir.
- II - Agentes deliberativos podem planejar as próprias ações para atingir objetivos.
- III - A reatividade é uma característica exclusiva de agentes não inteligentes.
- IV - Agentes híbridos combinam elementos reativos e deliberativos.

É correto o que se afirma em:

- a) I e II, apenas.
- b) II e IV, apenas.
- c) I, III e IV, apenas.
- d) I, II e IV, apenas.
- e) I, II, III e IV.

AUTOATIVIDADE

3. Os agentes inteligentes interagem com o ambiente por meio de percepções que recebem mediante sensores. A partir dessas percepções, os agentes realizam um processamento de informações para definir as próprias ações. A capacidade de um agente em responder de forma adequada a diferentes cenários depende de como ele processa as informações recebidas e da programação ou aprendizado.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - Os agentes inteligentes tomam decisões com base nas percepções do ambiente.

PORQUE

II - A função de percepção e ação mapeia as percepções para ações adequadas.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

LUGER, G. F. **Knowing Our World**: An Artificial Intelligence Perspective. [S. l.]: Springer, 2021.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence**: A Modern Approach. [S. l.]: Pearson, 2021.

SOUZA, J. N. de. **Lógica para Ciência da Computação e Áreas Afins**: Uma introdução concisa sobre os fundamentos da Lógica. Rio de Janeiro: [s. n.], 2020.

TAULLI, T. **Artificial Intelligence Basics**: A Non-Technical Introduction. [S. l.]: Apress, 2019.

GABARITO

1. Opção B.

Um agente inteligente é definido pela capacidade de perceber o ambiente e agir de forma autônoma para alcançar os próprios objetivos. A alternativa A está incorreta, pois agentes inteligentes não reagem apenas a comandos pré-programados: eles adaptam o comportamento com base nas percepções em tempo real. A alternativa C não caracteriza um agente inteligente, pois a interação com o ambiente é uma parte essencial da definição de "agente". A alternativa D está equivocada, porque agentes inteligentes precisam interagir ativamente com o ambiente, enquanto a alternativa E está incorreta, já que armazenar informações não é o suficiente para ser considerado um agente inteligente: a capacidade de tomar decisões é fundamental.

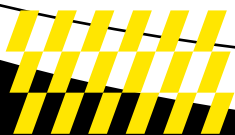
2. Opção D.

Agentes inteligentes são caracterizados pela autonomia (I) e, no caso dos agentes deliberativos, pela capacidade de planejar ações para atingir objetivos (II). Agentes híbridos (IV) combinam aspectos dos agentes reativos e deliberativos, adaptando-se a uma variedade maior de ambientes. A afirmativa III está incorreta, porque a reatividade é, na verdade, uma característica dos agentes inteligentes, especialmente dos agentes reativos que respondem a estímulos imediatos no ambiente. Essa distinção é fundamental para entender as diferentes maneiras como os agentes podem operar.

3. Opção A.

As duas asserções são verdadeiras e a segunda asserção (II) serve como uma justificativa correta para a primeira (I). Os agentes inteligentes precisam processar as informações que recebem do ambiente para tomar decisões. Isso é feito mediante a função que mapeia essas percepções em ações adequadas. Essa é a base do comportamento dos agentes: coletar informações do ambiente e, com base nessas informações, escolher a ação mais adequada. Alternativas, como B, C, D e E, estão incorretas, porque não reconhecem a relação fundamental entre a percepção do ambiente e as ações tomadas pelo agente, bem como o papel da função que realiza esse mapeamento.

unidade





TEMA DE APRENDIZAGEM 7

REDES SEMÂNTICAS EM IA SIMBÓLICA

MINHAS METAS

- Entender o conceito de redes semânticas.
- Compreender a importância da representação do conhecimento.
- Estudar a abordagem matemática com a teoria dos grafos.
- Conhecer a construção e a organização de redes semânticas.
- Desenvolver a implementação prática com Python.
- Explorar a manipulação e a análise de dados em redes semânticas.
- Aplicar os conteúdos estudados em problemas reais e relevância profissional.

INICIE SUA JORNADA

Imagine um cenário em que máquinas não apenas processam dados, mas também compreendem as conexões entre informações, estabelecendo relações de forma lógica e coerente. Com os avanços da Inteligência Artificial, especialmente no campo da Inteligência Artificial (IA) Simbólica, essa possibilidade se torna cada vez mais real. No centro desse progresso, encontram-se as redes semânticas, uma tecnologia que permite a representação do conhecimento de maneira estruturada e relacional.

VOCÊ SABE RESPONDER?

Como construir sistemas que "entendem" as relações complexas entre conceitos? Quais são os **desafios** de organizar conhecimento de forma que seja útil para máquinas e humanos?

A teoria dos grafos fornece a base matemática para as redes semânticas, permitindo que conceitos sejam mapeados e conectados de forma visual e lógica. Cada conceito é representado como um nó, enquanto as conexões e as propriedades são arestas que estruturam o conhecimento de maneira acessível e eficiente. Esse modelo é essencial em áreas, como IA aplicada, visto que ele permite que sistemas realizem inferências e respondam a perguntas complexas de forma automatizada.

Suponha que você está trabalhando em uma plataforma de suporte ao cliente e precisa desenvolver um sistema de recomendação que entenda as necessidades dos usuários e ofereça soluções personalizadas. Como as redes semânticas poderiam ajudar a mapear as dúvidas e as respostas comuns, relacionando-as com o perfil de cada usuário? Esse tipo de aplicação exemplifica o modo como a **organização de conhecimento** pode ser utilizada para representar dados complexos e facilitar a personalização e a precisão das recomendações.

Refletir sobre o impacto do domínio dessas técnicas é essencial para entender o respectivo valor profissional. Ao adquirir habilidades em redes semânticas e organização de conhecimento, você se posiciona na linha de frente de uma área em crescimento, com aplicações que vão desde o atendimento ao cliente até sistemas de aprendizado personalizados. Além disso, o conhecimento prático em IA simbólica permite que você contribua para soluções tecnológicas mais transparentes, acessíveis e eficazes, moldando a interação entre humanos e máquinas de forma mais inteligente e ética.



PLAY NO CONHECIMENTO

Neste episódio do nosso podcast, mergulhamos nas redes semânticas e descobrimos como elas possibilitam que máquinas organizem e interpretem conhecimento de maneira conectada e significativa. Venha entender como essa tecnologia está moldando aplicações em áreas, como atendimento ao cliente, saúde e análise de dados, oferecendo novas possibilidades para a interação entre humanos e IA. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para que você esteja preparado para explorar as redes semânticas e a respectiva aplicação na IA simbólica, vamos revisar alguns conceitos fundamentais. Primeiramente, é essencial relembrar a teoria dos grafos, com foco no modo como nós e arestas representam os conceitos e as devidas relações. Esse conhecimento básico permitirá que você entenda a estrutura das redes semânticas. Além disso, é importante revisar estruturas de dados em Python, as quais serão utilizadas para implementar e manipular redes semânticas de forma eficiente. Familiarize-se também com bibliotecas, como o NetworkX, que auxilia na criação e visualização de grafos em Python. Para reforçar esses conteúdos, assista ao vídeo introdutório sobre grafos e redes, disponível no link a seguir. **<https://medium.com/computando-arte/an%C3%A1lise-de-redes-complexas-em-python-com-networkx-e-pyvis-bd8d67889g18>**

DESENVOLVA SEU POTENCIAL

REDES SEMÂNTICAS: FUNDAMENTOS E APLICAÇÕES EM IA

As redes semânticas são um método poderoso para representar conhecimento em IA simbólica, permitindo que os conceitos e as respectivas relações sejam visualizados e manipulados de forma estruturada (Russell; Norvig, 2013). Usando grafos, as redes semânticas representam cada conceito como um nó, enquanto as relações entre eles são modeladas por arestas. Esse método possibilita que sistemas de IA realizem inferências automáticas e organizem dados complexos, o que é essencial para resolver problemas práticos e estruturados.

A abordagem das redes semânticas é fundamental para sistemas de IA que necessitam de uma representação clara das relações entre informações, como em sistemas de recomendação, diagnósticos médicos e atendimento ao cliente. Entender como esses grafos estruturam o conhecimento é o primeiro passo para projetar sistemas capazes de conectar informações e gerar novas inferências. A importância reside na capacidade de **estruturar e armazenar informações** de modo que a máquina possa raciocinar sobre elas, realizar inferências e responder perguntas com base nas relações definidas (Luger, 2021).

Por exemplo, ao estruturar conceitos, como “Animal”, “Mamífero” e “Ave”, e relacioná-los hierarquicamente, é possível construir uma base de conhecimento em que perguntas, como “Todo mamífero é um animal?”, possam ser respondidas automaticamente.



CONSTRUINDO E EXPLORANDO REDES SEMÂNTICAS

A teoria dos grafos oferece a base matemática que sustenta a construção de redes semânticas (Russell; Norvig, 2013). Cada conceito representado é um nó, e as conexões entre esses conceitos são as arestas (Gonzales *et al.*, 2013). A estrutura de grafos permite representar diferentes tipos de relações entre conceitos, tais como hierárquicas (como “é-um”), de associação (“associado-com”) e de parte-todo (“parte-de”). Essa abordagem gráfica facilita a modelagem e a organização do conhecimento de maneira que sistemas de IA possam realizar inferências e manipular informações de maneira eficiente.

Para construir redes semânticas eficientes, é fundamental entender a estrutura básica dos grafos e o modo como eles podem ser aplicados à modelagem de conhecimento. Em uma rede semântica, os elementos principais são (Boaventura Netto; Jurkiewicz, 2021):



ZOOM NO CONHECIMENTO

- **Nós (vértices):** cada nó representa um conceito ou entidade na rede. Em uma rede sobre o domínio médico, por exemplo, um nó poderia representar “Sintoma”, “Doença” ou “Tratamento”. Esses nós são os pontos centrais que se conectam a outros conceitos dentro do grafo.
- **Arestas (conexões):** as arestas são as ligações entre os nós, indicando a relação entre conceitos. As conexões podem ser de diferentes tipos, dependendo do contexto e da necessidade da rede. Por exemplo, em uma rede de atendimento ao cliente, uma aresta pode representar a relação entre um “Cliente” e um “Produto” que ele comprou.

Nas redes semânticas, as arestas permitem a representação de relações comuns, fundamentais para modelar e organizar o conhecimento de forma estruturada (Gonzales *et al.*, 2013). Essas conexões vão além de uma simples associação, permitindo que os conceitos se relacionem de maneira hierárquica, funcional ou associativa, com objetivos diferentes de uso. Essas relações são especialmente úteis em redes semânticas, pois ajudam a IA a “entender” o mundo da forma como os humanos o categorizam, atribuindo significado e contexto a diferentes tipos de dados.

Ao mapear a relação entre conceitos, o sistema de IA pode, por exemplo, inferir atributos de categorias mais amplas, decompor objetos em suas partes constituintes ou associar entidades que frequentemente interagem, mesmo sem uma relação de dependência direta. As principais relações usadas em redes semânticas são (Russell; Norvig, 2013):

- **Hierarquia (“é-um”):**

A relação de hierarquia, ou “é-um”, expressa uma conexão de tipo ou categoria entre dois conceitos, em que um conceito específico pertence a uma categoria mais ampla. Esse tipo de relação é fundamental para organizar o conhecimento de maneira sistemática e categórica, permitindo que o sistema de IA construa árvores de classes e sub-classes, de forma que dados possam ser buscados e categorizados de maneira lógica e eficiente. A relação “é-um” também facilita a herança de atributos, pois uma entidade mais específica herda todas as características da categoria à qual pertence, criando uma estrutura hierárquica que ajuda na categorização e na generalização de dados.

Por exemplo, em uma rede que representa espécies animais, a afirmação “Cão é um Mamífero” indica que o conceito “Cão” pertence à categoria “Mamífero”. Essa conexão permite que o sistema entenda que todos os atributos gerais atribuídos à “Mamífero” também se aplicam à “Cão”, como ser de sangue quente, ter pelos e amamentar os filhotes. Observe a Figura 1.

“é-um”, expressa uma conexão de tipo ou categoria entre dois conceitos

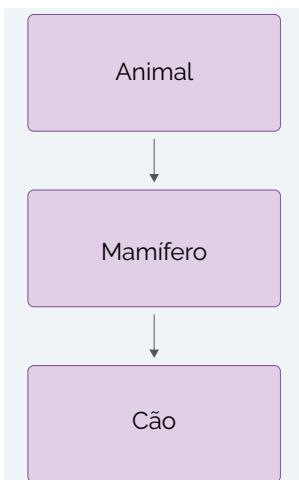


Figura 1 – Rede semântica com conexão hierárquica
Fonte: o autor.

Descrição da Imagem: é exibido um diagrama hierárquico com três caixas conectadas por setas apontando para baixo. A caixa superior contém o texto “Animal”. A caixa do meio contém o texto “Mamífero”, enquanto a caixa inferior contém o texto “Cão”. Fim da descrição.

Esse tipo de hierarquia é especialmente útil em domínios em que é importante classificar itens em classes mais amplas e fazer inferências a partir dessas classificações, como em biologia, onde os seres vivos são categorizados em reinos, famílias, gêneros e espécies. Graças à hierarquia, o sistema de IA pode navegar por dados em diferentes níveis de generalidade ou especificidade, respondendo a perguntas com base na árvore de classificação.

▪ **Parte-todo (“parte-de”):**

A relação parte-todo indica que um conceito ou entidade é um componente de outro conceito maior. Essa relação é essencial em contextos nos quais é necessário entender a composição de objetos ou entidades em suas partes constitutivas (Russell; Norvig, 2013). A estrutura parte-todo permite que o sistema compreenda a organização interna de objetos complexos, o que possibilita que problemas possam ser localizados e resolvidos em níveis mais específicos. Essa relação também é especialmente útil em domínios, como engenharia e medicina, nos quais é preciso entender a composição interna de sistemas, máquinas ou o corpo humano.

Por exemplo, em uma rede semântica que representa a estrutura de uma casa, a afirmação **“Telhado é parte de Casa”** indica que o telhado é um dos componentes fundamentais da construção. Essa relação parte-todo é essencial para que o sistema compreenda a composição integral da casa, ao compreender que cada elemento desempenha um papel específico e interdependente para a funcionalidade do todo.

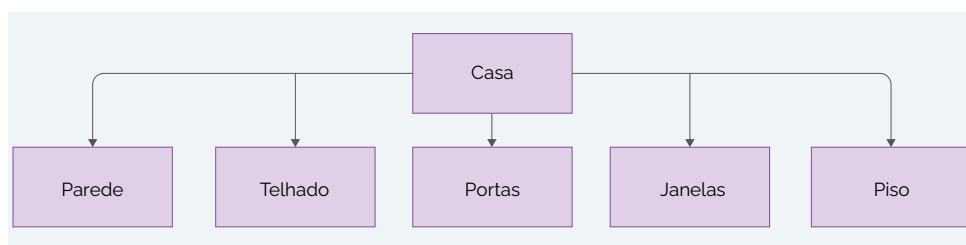


Figura 2 - Rede semântica com conexão parte-todo / Fonte: o autor.

Descrição da Imagem: é exibido um diagrama hierárquico que representa a estrutura de uma casa e os respectivos componentes principais. No topo do diagrama, há um retângulo com a palavra “Casa”. A partir desse retângulo, há cinco setas que apontam para baixo, cada uma conectada a um retângulo menor. Esses retângulos menores contêm as palavras “Parede”, “Telhado”, “Porta”, “Janelas” e “Piso”. Fim da descrição.

Em aplicações práticas, essa relação é amplamente utilizada para modelar sistemas complexos, como maquinário em engenharia ou sistemas orgânicos em medicina, permitindo uma análise eficiente de cada parte conforme o respectivo papel no funcionamento completo da entidade.

- **Associação (“associado-com”):**

A relação associativa estabelece uma conexão entre dois conceitos, mesmo que não haja uma hierarquia ou dependência direta entre eles. Em muitas aplicações de IA, é necessário vincular conceitos de forma relacional para que o sistema possa entender as conexões contextuais e identificar padrões de comportamento ou preferências, permitindo a criação de recomendações personalizadas. A associação é essencial em domínios, como na análise de redes sociais, em que as conexões entre os nós não são de subordinação, mas de interação ou influência mútua. A relação “associado-com” possibilita que o sistema de IA encontre padrões relacionais, conectando conceitos ou entidades que frequentemente interagem.

Por exemplo, a relação “Médico associado a Hospital” indica que o conceito “Médico” está vinculado ao conceito “Hospital”, mesmo que um médico não seja parte do hospital, nem o hospital seja uma categoria a que o médico pertença. Em uma rede semântica de um sistema de saúde, essa associação é útil para identificar onde um médico exerce a prática, quais são as especialidades oferecidas em determinado hospital e como a presença de certos profissionais pode estar relacionada à localização.

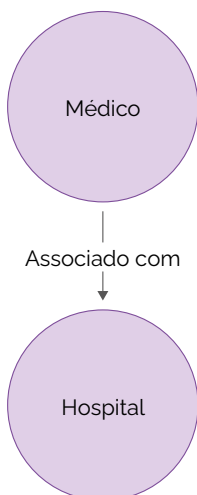


Figura 3 - Rede semântica com conexão de associação / Fonte: o autor.

Descrição da Imagem: são exibidos dois círculos roxos conectados por uma seta apontando para baixo. O círculo superior contém o texto “Médico”, enquanto o círculo inferior contém o texto “Hospital”. Entre os dois círculos, há uma seta com o texto “Associado com”. Fim da descrição.

Ela é comumente usada em contextos nos quais é necessário vincular conceitos de maneira relacional, permitindo que o sistema compreenda associações que não envolvem uma relação de inclusão ou subordinação.

Construção e organização de redes semânticas

As redes semânticas são estruturas de dados que representam conhecimento por meio de nós e arestas. Nelas, os nós indicam conceitos ou entidades e as arestas representam as relações entre esses conceitos (Gonzales *et al.*, 2013). Essas redes possibilitam a criação de sistemas de Inteligência Artificial que podem navegar e entender a relação entre diferentes elementos, sendo amplamente usadas em áreas, como processamento de linguagem natural, organização de conhecimento em sistemas de IA, motores de recomendação e diagnósticos automáticos (Luger, 2021). Para construir e organizar uma rede semântica, é essencial seguir etapas claras:

1. DEFINIÇÃO DOS CONCEITOS E DAS ENTIDADES

O primeiro passo para construir uma rede semântica é identificar os conceitos centrais que ela irá representar (Russell; Norvig, 2013). Esses conceitos são os nós da rede e variam de acordo com o contexto do sistema. Por exemplo, em uma rede semântica sobre medicina, os nós podem representar "Paciente", "Sintoma", "Doença", "Tratamento" e "Medicação". A escolha dos conceitos é fundamental, pois eles formam a base sobre a qual a rede semântica será estruturada.

2. ESTABELECIMENTO DAS RELAÇÕES

Após identificar os conceitos, é necessário definir as relações entre eles, que formam as arestas do grafo e variam conforme o tipo de rede e a complexidade do sistema. Relações comuns incluem: a hierarquia, que representa a classificação de um conceito em uma categoria mais ampla, como "Cão é um Mamífero"; a relação parte-todo, a qual define que um conceito faz parte de outro, como "Motor é parte de Carro"; e a associação, que conecta conceitos relacionados, mas sem uma hierarquia direta, como "Médico associado a Hospital". Essas relações criam uma rede estruturada na qual o sistema de IA pode navegar e inferir características ou conexões entre os conceitos.

3. DEFINIÇÃO DE PROPRIEDADES E ATRIBUTOS

Cada nó (conceito) ou aresta (relação) pode conter propriedades que adicionam mais detalhes. Por exemplo, o conceito "Paciente" pode ter propriedades, como "Idade" e "Histórico Médico". Esses atributos são úteis para enriquecer o contexto e facilitar a análise de dados e inferências. No caso das relações, a propriedade "é-um" pode ser marcada em todas as conexões hierárquicas para definir uma regra de herança de características.

4. ORGANIZAÇÃO DA ESTRUTURA DE REDE

Com os nós, as arestas e as propriedades definidos, a rede pode ser organizada em uma estrutura hierárquica ou associativa. Para redes complexas, é comum usar uma organização baseada em grafos direcionados, em que a direção das arestas indica a relação entre conceitos, como uma relação "filho" para entidades que pertencem a categorias mais amplas. Por exemplo, "Mamífero" seria o nó pai de "Cão", e o sistema poderia navegar a partir de "Mamífero" para "Cão" ou vice-versa.

5. IMPLEMENTAÇÃO DE REGRAS DE INFERÊNCIA

As redes semânticas são ainda mais úteis quando têm regras de inferência que permitem deduzir novas informações com base nas conexões existentes. Em um sistema médico, por exemplo, se "Febre" é associada à "Infecção", e "Infecção" está associada à "Tratamento com Antibióticos", o sistema pode inferir automaticamente que uma febre pode indicar a necessidade de um tratamento específico. A implementação de regras permite que o sistema faça inferências automáticas, tornando-o mais dinâmico e útil.

Esse processo de modelagem, que inclui a definição de conceitos, o estabelecimento de relações e a implementação de regras de inferência, torna os sistemas de IA capazes de compreender e responder a dados complexos em aplicações variadas, como recomendações personalizadas e diagnósticos automáticos. Assim, redes semânticas não só aprimoram a capacidade dos sistemas de IA de lidar com a informação de forma lógica e precisa, como também pavimentam o caminho para interações mais transparentes e inteligentes entre humanos e máquinas.



EXEMPLO PRÁTICO: IMPLEMENTANDO REDES SEMÂNTICAS EM PYTHON

A seguir, é apresentado um passo a passo para construir uma rede semântica. Usa-se, como exemplo, uma rede para representar a estrutura de uma casa e alguns de seus cômodos e características.

Passo 1: Identificar os conceitos centrais

Os conceitos centrais são os elementos principais que serão representados como **nós** da rede.

Exemplo de conceitos:

- Casa
- Quarto
- Sala
- Cozinha
- Banheiro

Passo 2: Definir as relações entre os conceitos

Agora, conectamos os conceitos com **arestas**, que representam as relações entre eles.

Tipos de relações comuns e aplicação na rede:

1. **Parte-todo (parte-de)**: mostra que um conceito faz parte de outro.
 - Exemplo: “Quarto é parte de Casa”.
2. **Hierarquia (é-um)**: mostra que um conceito é uma categoria de outro.
 - Exemplo: Se estivéssemos incluindo diferentes tipos de cômodos, como “Quarto” e “Suíte”, poderíamos dizer “Suíte é um Quarto”.
3. **Associação (associado-com)**: conecta conceitos relacionados sem hierarquia.
 - Exemplo: “Banheiro associado à Sistema de Água”, para indicar que o banheiro está conectado ao sistema de água da casa.

Passo 3: Adicionar propriedades e atributos

Cada conceito pode ter atributos adicionais para enriquecer a rede. Aqui, adicionamos as propriedades que detalham cada elemento.

Exemplo de atributos:

- **Casa**: “Tamanho”, “Número de Cômodos”
- **Quarto**: “Número de Janelas”, “Área em m²”
- **Sala**: “Tipo de Piso”, “Área em m²”

Esses atributos ajudam a definir mais informações que serão úteis para o sistema de IA, ao buscar ou inferir dados sobre a casa.

Passo 4: Organizar a estrutura da rede

Agora, organizamos a rede para que cada nó e aresta seja colocado na posição correta.

1. **Casa** é o nó principal.
2. **Quarto, Sala, Cozinha, e Banheiro** são conectados à Casa por relações de **parte-de**.
3. **Banheiro** está conectado à **Sistema de Água** por uma relação **associado-com**.

Passo 5: Implementar regras de inferência

Regras de inferência permitem que o sistema de IA deduza novas informações automaticamente.

Exemplo de inferência: se Banheiro está associado ao Sistema de Água, o sistema pode inferir que Banheiro precisa de Fornecimento de Água.

Passo 6: Implementar e visualizar a rede

Agora que já realizamos o passo a passo para a criação da rede semântica, podemos implementar essa rede semântica utilizando Python. Com Python e a biblioteca NetworkX, você pode criar e manipular redes semânticas de forma eficiente. O código a seguir ilustra a criação da nossa rede semântica simples e a implementação de um sistema de inferência que determina as relações hierárquicas em uma rede. Primeiramente, criamos um grafo direcionado para representar as conexões hierárquicas entre os conceitos centrais. Os nós da rede representam os cômodos e o sistema de água, enquanto as arestas definem as relações entre esses conceitos. Por exemplo, “Casa” é composta por “Quarto”, “Sala”, “Cozinha” e “Banheiro”, e o “Banheiro” está associado ao “Sistema de Água”.

```
import networkx as nx
```

```
import matplotlib.pyplot as plt
```

```
G = nx.DiGraph()
```

```
G.add_node("Casa", tamanho="150 m²", num_comodos=4)
```

```

G.add_node("Quarto", num_janelas=2, area="12 m²")

G.add_node("Sala", tipo_piso="Porcelanato", area="20 m²")

G.add_node("Cozinha", tem_gas=True, area="10 m²")

G.add_node("Banheiro", tem_agua=True, area="8 m²")

G.add_node("Sistema de Água")

G.add_edge("Casa", "Quarto", relation="parte-de")

G.add_edge("Casa", "Sala", relation="parte-de")

G.add_edge("Casa", "Cozinha", relation="parte-de")

G.add_edge("Casa", "Banheiro", relation="parte-de")

G.add_edge("Banheiro", "Sistema de Água", relation="associado-com")

```

Em seguida, implementamos uma função de inferência que determina se um conceito específico requer água com base na associação ao “Sistema de Água”. Essa função verifica se há uma aresta conectando o conceito ao sistema e imprime o resultado.

```

def inferir_necessidade_agua(grafo, conceito):

    if grafo.has_edge(conceito, "Sistema de Água"):

        print(f"{conceito} precisa de fornecimento de água.")

```

```
else:
```

```
    print(f"{conceito} não requer fornecimento de água  
diretamente.»)
```

```
inferir_necessidade_agua(G, "Banheiro")
```

Além disso, foi desenvolvida uma função para visualizar o grafo. Essa função utiliza a biblioteca Matplotlib para exibir o grafo, incluindo os atributos dos nós (como tamanho e área) e os rótulos das arestas, que representam as relações entre os conceitos.

```
def desenhar_grafo_com_atributos(grafo):
```

```
    pos = nx.spring_layout(grafo)
```

```
    nx.draw(grafo, pos, with_labels=True, node_size=3000,  
            node_color="lightblue", font_size=10)
```

```
    labels_nos = {node: "\n".join([f"{k}: {v}" for k, v in  
grafo.nodes[node].items()]) for node in grafo.nodes}
```

```
    for node, (x, y) in pos.items():
```

```
        plt.text(x + 0.15, y, labels_nos[node], fontsize=8,  
                 ha="left", color="black")
```

```
    labels_arestas = nx.get_edge_attributes(grafo, 'relation')
```

```
nx.draw_networkx_edge_labels(grafo, pos, edge_labels=labels_arestas, font_color="red")
```

```
plt.show()
```

```
desenhar_grafo_com_atributos(G)
```

O resultado da implementação pode ser visualizado na Figura 4. A partir da associação entre o banheiro e o sistema de água, realizamos uma inferência se o banheiro precisa de água. Considerando a rede semântica e a implementação de inferência da rede, temos o seguinte resultado: “Banheiro precisa de fornecimento de água.”

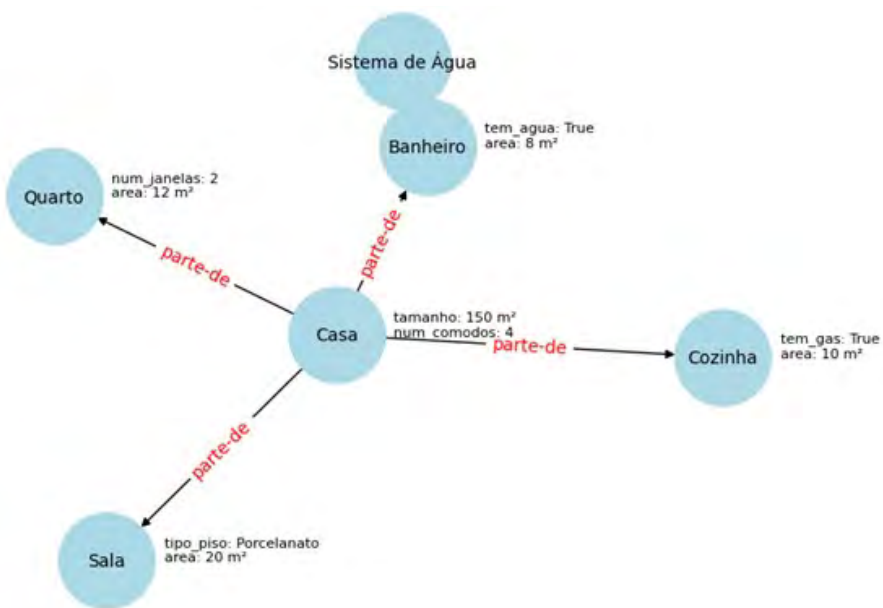


Figura 4 - Estrutura da rede semântica de uma casa / Fonte: o autor.

Descrição da Imagem: é exposto um diagrama representando a estrutura e os atributos de uma casa e os respectivos cômodos. O nó central é rotulado como «Casa», com conexões para outros quatro nós: «Quarto», «Sala», «Banheiro» e «Cozinha». Cada nó de cômodo tem atributos específicos: Casa, Tamanho: 150 m², Número de cômodos: 4, Quarto, Número de janelas: 2, Área: 12 m², Relação: parte de (part of) Casa, Sala: Tipo de piso: Porcelanato, Área: 20 m², Relação: parte de (part of) Casa, Banheiro, Tem água: Sim, Área: 8 m², Relação: parte de (part of) Casa, Conectado ao “Sistema de Água”: Cozinha, Tem gás: Sim, Área: 10 m² Relação: parte de (part of) Casa. Fim da descrição.

Por meio de ferramentas, como a biblioteca NetworkX, é possível desenvolver e visualizar redes semânticas em Python de maneira prática e eficaz. Esse processo, que envolve desde a definição dos nós e das relações até a implementação de regras de inferência, é essencial para áreas que exigem compreensão automatizada de dados complexos, tais como atendimento ao cliente, diagnóstico médico e sistemas de recomendação. Ao dominar a construção de redes semânticas, os profissionais podem desenvolver soluções que alinham a precisão da IA com uma lógica relacional, aproximando máquinas do raciocínio humano e trazendo possibilidades de inovação para diversas indústrias.

**EM FOCO**

Gostou do que discutimos até aqui? Tenho mais para conversar com você a respeito deste tema. Vamos lá? **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A construção e a organização de redes semânticas, aliadas à teoria dos grafos e à programação em Python, tornaram-se habilidades essenciais em um mercado que valoriza a automação e a inteligência na análise de dados. Essas ferramentas permitem estruturar e resolver problemas complexos de forma organizada, além de possibilitar a criação de sistemas inteligentes em áreas, como ciência de dados, engenharia de software, análise de mercado e consultoria empresarial.

O uso de redes semânticas para modelar conhecimento, em conjunto com a lógica proposicional e de predicados, oferece uma base robusta para organizar informações e fazer inferências. Python, com a biblioteca NetworkX, facilita a implementação desses sistemas, permitindo a criação de estruturas de dados que suportam análises e decisões fundamentadas. A habilidade de construir e manipular redes de conhecimento é especialmente valiosa para quem busca automatizar processos, aprimorar o diagnóstico de problemas e personalizar recomendações de forma eficiente.

A combinação de redes semânticas com modelos de aprendizado expande ainda mais as **possibilidades de aplicação**.

Essa integração permite o desenvolvimento de sistemas híbridos que não apenas inferem relações lógicas, mas também se ajustam de forma contínua a novos dados. Essa abordagem se torna particularmente valiosa em setores que exigem insights em tempo real, como análise financeira, monitoramento de saúde e inteligência de mercado.

À medida que a tecnologia avança, a capacidade de aplicar teoria na prática é um diferencial competitivo. Profissionais que dominam redes semânticas, teoria dos grafos e Python estarão mais bem preparados para enfrentar os desafios e desenvolver soluções inovadoras e estratégicas. Essas competências são requisitadas à medida que a IA e as tecnologias de análise de

dados continuam a se expandir, exigindo uma compreensão profunda tanto dos fundamentos teóricos quanto das aplicações práticas.

Como esses métodos podem transformar a sua área de atuação? Reflita sobre o modo como o conhecimento de redes semânticas e lógica pode otimizar processos e resolver problemas de maneira mais eficiente. Pense também nas implicações éticas e na importância de desenvolver soluções transparentes e compreensíveis para todos os usuários.



AUTOATIVIDADE

1. As redes semânticas são amplamente usadas para estruturar conhecimento em sistemas de IA, permitindo que a máquina entenda relações entre informações. Essa representação facilita recomendações personalizadas em serviços, como atendimento ao cliente, e auxilia diagnósticos médicos automatizados.

Qual é a principal vantagem das redes semânticas em sistemas de recomendação?

- a) Elas analisam dados rapidamente sem qualquer relação lógica.
 - b) Representam conceitos sem considerar a hierarquia.
 - c) Facilitam a inferência por meio das conexões lógicas entre conceitos.
 - d) Substituem o processamento de linguagem natural em recomendações.
 - e) Mantêm dados de forma não estruturada.
2. Redes semânticas podem apresentar diferentes tipos de relações, como hierárquicas (é-um), parte-todo (parte-de) e associativas (associado-com). Cada tipo de relação cumpre uma função específica, permitindo à IA organizar e inferir dados complexos.

Considerando os tipos de relações em redes semânticas, analise as afirmativas a seguir:

- I - "Parte-de" indica que um conceito é um componente de outro.
- II - "É-um" conecta conceitos de forma associativa sem hierarquia.
- III - "Associado-com" estabelece uma relação não hierárquica entre conceitos.
- IV - "É-um" define uma conexão de tipo ou categoria entre dois conceitos.

É correto o que se afirma em:

- a) I e II, apenas.
- b) II e III, apenas.
- c) I, III e IV, apenas.
- d) III e IV, apenas.
- e) I, II e IV, apenas

3. A inferência automática em redes semânticas é possível devido às conexões lógicas entre conceitos. Relações, como hierarquia e associação, auxiliam a IA a identificar padrões e realizar deduções com base nas relações mapeadas.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - Redes semânticas permitem a inferência automática por meio de suas relações.

PORQUE

II - II. A organização hierárquica é o único tipo de relação que permite inferências em redes semânticas.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

BOAVENTURA NETTO, P. O.; JURKIEWICZ, S. **Grafos**: introdução e prática. São Paulo: Blucher, 2021.

GONZALES, C. *et al.* (ed.). **Graph Structures for Knowledge Representation and Reasoning**: Third International Workshop. [S. l.]: GKR, 2013.

LUGER, G. F. **Knowing Our World**: An Artificial Intelligence Perspective. [S. l.]: Springer, 2021.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence**: A Modern Approach. [S. l.]: Prentice Hall, 2013.

GABARITO

1. Opção C.

A vantagem das redes semânticas em sistemas de recomendação é a capacidade de inferir informações por intermédio de conexões lógicas entre conceitos. As demais alternativas estão incorretas, pois não refletem a função estruturada e relacional das redes semânticas.

2. Opção C.

As afirmativas I, III e IV descrevem corretamente as relações "parte-de", "associado-com" e "é-um". A afirmativa II está incorreta, porque "é-um" representa uma conexão hierárquica, não associativa.

3. Opção C.

A primeira asserção é verdadeira, já que as redes semânticas permitem inferências automáticas. A segunda asserção é falsa, pois existem outros tipos de relações que também possibilitam inferências, como associações.



TEMA DE APRENDIZAGEM 8

FRAMES E SISTEMAS BASEADOS EM CONHECIMENTO

MINHAS METAS

- Entender o conceito de frames na IA Simbólica.
- Realizar a análise estrutural e herança em frames.
- Estudar a abordagem matemática para estruturas de frames.
- Compreender a importância dos frames na representação do conhecimento.
- Desenvolver a implementação prática de frames com Python.
- Explorar a manipulação e a análise de dados em estruturas de frames.
- Aplicar frames em problemas reais e analisar a respectiva relevância profissional.

INICIE SUA JORNADA

Imagine sistemas que, além de processarem dados, consigam compreender objetos e as respectivas interações quase como seres humanos. Na área da Inteligência Artificial, a IA Simbólica abre caminhos para organizar e representar o conhecimento de forma lógica e estruturada. Uma opção poderosa para alcançar essa capacidade é o uso de frames: estruturas de dados amplamente aplicadas para organizar e manipular informações de forma estruturada e eficiente. Funcionam como “molduras” que agrupam dados relacionados, representando características de objetos, situações, eventos ou qualquer entidade a ser modelada em um sistema de IA (Oliveira; Carvalho, 2008).

Com frames, máquinas podem interpretar dados de forma contextualizada e próxima ao raciocínio humano, permitindo uma compreensão mais rica e eficiente das informações.

VOCÊ SABE RESPONDER?

Como criar sistemas que realmente compreendam os dados, relacionando informações de maneira eficiente? Quais são os passos para organizar o conhecimento de modo que facilite o raciocínio automatizado e seja útil tanto para máquinas quanto para pessoas?

Os frames oferecem uma maneira **prática e poderosa** de representar objetos e as respectivas características por meio de “slots”, que armazenam propriedades e inter-relações. Esse modelo é particularmente útil em setores que demandam precisão na recomendação e análise de dados, ajudando os sistemas a fazerem inferências e previsões com base em padrões complexos.

Agora, pense em uma plataforma de educação digital em que o conteúdo precisa ser personalizado para cada aluno, levando em conta interesses e progresso individual. Como os frames poderiam estruturar as informações sobre o histórico de aprendizado, as preferências e as habilidades específicas, ajudando o sistema a recomendar materiais de estudo adequados para cada perfil? Essa

aplicação mostra como a estruturação de conhecimento por frames facilita a personalização, otimizando a experiência de quem utiliza o sistema.

Ao dominar essas técnicas, você adquire uma habilidade valiosa, essencial para atuar no desenvolvimento de IA de última geração. Com o conhecimento em frames, você estará preparado para criar soluções tecnológicas mais eficazes, acessíveis e relevantes, contribuindo para uma interação mais fluida e intuitiva entre pessoas e sistemas.



PLAY NO CONHECIMENTO

Neste episódio do nosso podcast, vamos entender como os frames auxiliam a IA na organização e na análise de grandes volumes de dados. Exploramos como essas estruturas ajudam sistemas inteligentes a identificarem padrões e extrair informações relevantes em áreas, como pesquisa científica, marketing e gestão de informações corporativas. Venha descobrir como os frames estão tornando a IA mais eficaz na filtragem e na compreensão de dados complexos, permitindo uma tomada de decisão mais rápida e precisa. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para que você esteja preparado para estudar frames e a respectiva aplicação na IA Simbólica, vamos revisar alguns conceitos essenciais. Primeiramente, é importante relembrar os conceitos de **herança e modularidade** nas estruturas de dados, similar ao que ocorre em programação orientada a objetos. Em frames, a herança permite que um frame específico, como "Carro", herde características de um frame mais geral, como "Veículo". Essa estrutura hierárquica ajuda a organizar e a atualizar informações de forma eficiente. Para reforçar esses pontos, sugerimos um vídeo introdutório sobre herança em estruturas de dados e programação orientada a objetos, disponível no link a seguir. <https://www.youtube.com/watch?v=dPFOoAPFowE>

DESENVOLVA SEU POTENCIAL

FRAMES: FUNDAMENTOS E APLICAÇÕES EM IA SIMBÓLICA

Frames são estruturas de dados amplamente utilizadas na IA Simbólica para representar conhecimento por meio de “quadros” que contêm informações organizadas sobre determinado conceito, assim como os atributos e os valores associados a ele (Russell; Norvig, 2013). Cada frame é semelhante a um “esqueleto” que armazena dados específicos e permite que sistemas de IA realizem inferências e organizem informações complexas de maneira estruturada. Essa abordagem é essencial para sistemas que necessitam organizar e acessar informações em diferentes níveis de detalhe.

Por exemplo, ao representarmos um “Animal”, podemos incluir características, como “espécie”, “habitat” e “alimentação”, criando um frame que permite que perguntas sobre esses aspectos sejam respondidas automaticamente. O conceito de frames permite que sistemas de IA façam inferências e associem informações automaticamente.



ESTRUTURA E HERANÇA EM FRAMES

A estrutura de um frame é composta por elementos que formam um conjunto coeso de dados sobre um conceito específico (Russell; Norvig, 2013). Esses elementos incluem nome do frame, slots e facetas, que, juntos, proporcionam uma estrutura rica e flexível (Oliveira; Carvalho, 2008). Além disso, frames operam em uma estrutura hierárquica, o que permite a herança entre frames, possibilitando a criação de uma árvore de conhecimento eficiente e modular.

Estrutura de um Frame

A estrutura de um frame é comparável a um formulário ou ficha em que cada campo armazena uma informação específica sobre o conceito representado. Esses campos são chamados de slots, e cada slot contém um dado ou característica específica, compondo o que se conhece como uma “ficha” detalhada de um objeto ou entidade (Oliveira; Carvalho, 2008).

1. Nome do Frame:

O nome identifica de forma única o conceito ou a entidade que o frame representa (Oliveira; Carvalho, 2008). É a “cabeça” do frame, descrevendo o tipo de objeto ou a situação modelada, como “Animal”, “Carro”, “Paciente” ou “Evento”. Em um sistema de diagnóstico médico, por exemplo, um frame nomeado “Gripe” pode representar a condição médica e os respectivos atributos associados, enquanto um frame “Paciente” representa o indivíduo afetado pela condição.

2. Slots:

Slots são os campos de dados dentro do frame que armazenam informações específicas, funcionando como uma ficha detalhada sobre o conceito representado (Lieto, 2021). Cada slot contém um nome que descreve o tipo de informação e um valor que preenche o campo. Esses slots são flexíveis e personalizáveis, podendo representar desde características básicas até relações complexas entre entidades.



ZOOM NO CONHECIMENTO

- Nome do slot: cada slot é identificado por um nome que representa a informação contida no slot. No frame "Carro", por exemplo, slots, como "Marca", "Modelo" e "Ano", indicam informações detalhadas sobre o veículo.
- Valor do slot: o valor é a informação específica atribuída ao slot. Esse valor pode ser um dado simples (por exemplo, "Marca: Toyota") ou uma referência a outro frame para representar uma relação complexa (por exemplo, "Alimentação" em um frame de "Leão" pode apontar para o frame "Carnívoro").

A organização em slots proporciona consistência na estrutura dos dados, permitindo que cada conceito seja descrito de forma uniforme e acessível.

3. Facetas:

Facetas são atributos adicionais que agregam camadas de significado e controle aos slots, tornando os frames mais robustos e precisos (Oliveira; Carvalho, 2008). Facetas permitem a inclusão de valores padronizados, restrições e condições, garantindo que as informações sejam preenchidas e acessadas de maneira consistente e segura (Brachman; Levesque, 2004).

VALORES-PADRÃO

Definem um valor automático para o slot, caso ele não seja explicitamente preenchido. Esse recurso é útil em sistemas em que alguns atributos possuem valores comuns ou padrão.

RESTRIÇÕES DE VALOR

Impõem limitações nos valores aceitáveis para o slot, garantindo a integridade dos dados. Por exemplo, um slot "Idade" em um frame de "Paciente" poderia ter uma restrição para aceitar apenas valores numéricos positivos, impedindo a entrada de dados incoerentes.

CONDIÇÕES

Definem regras específicas para o preenchimento e o uso do slot. Uma faceta pode, por exemplo, restringir o slot "Alimentação" em um frame "Animal" para aceitar apenas valores, como "Herbívoro", "Carnívoro" ou "Onívoro", de acordo com o tipo de alimentação do animal.

As facetas tornam o frame mais flexível, permitindo que o sistema de IA controle a entrada e a manipulação de dados, além de definirem comportamentos padronizados que ajudam a organizar e a validar informações.

4. Demons:

Os demons são procedimentos acoplados a frames que executam ações automáticas, geralmente quando determinados valores são fornecidos ou modificados (Aragão, 2002). Eles oferecem flexibilidade ao sistema, pois permitem cálculos dinâmicos e inferências sem a necessidade de armazenar todas as informações explicitamente.

Por exemplo, em um **frame** que representa um quadrado, o **demon** pode ser utilizado para calcular automaticamente a área do quadrado a partir do comprimento de um dos lados. Isso elimina a necessidade de armazenar a área como um atributo fixo, já que ela pode ser calculada sempre que necessário.

Herança em frames

A herança é um dos recursos mais poderosos dos frames, permitindo a criação de uma estrutura hierárquica que facilita a reutilização de informações. Além disso, reduz a redundância e melhora a organização do conhecimento (Patterson, 1990). A herança entre frames permite que frames mais específicos herdem slots e facetas de frames mais gerais, criando uma árvore de conhecimento que estrutura conceitos de maneira lógica e eficiente.



Superframe: a base genérica

Um **superframe** é um frame que define características gerais e comuns para uma categoria de conceitos. Ele funciona como a base sobre a qual frames mais específicos, chamados de **subframes**, são construídos (Oliveira; Carvalho, 2008). Por exemplo, um superframe chamado “**Veículo**” pode incluir slots básicos, como:

- **Número de rodas:** 2, 4 ou mais.
- **Fonte de energia:** combustível, elétrico ou híbrido.

Os subframes, como “**Carro**” e “**Moto**”, herdam esses slots, garantindo consistência e evitando a duplicação de informações. Assim, todos os slots gerais definidos em “**Veículo**” são automaticamente aplicados aos respectivos descendentes, enquanto estes podem adicionar detalhes próprios.

Por que o superframe é importante? Essa estrutura modular permite que conceitos amplos sirvam como alicerce para especializações, economizando tempo no desenvolvimento e promovendo uma organização lógica. A reutilização de informações comuns elimina a necessidade de redefinir atributos repetidamente, tornando o sistema mais eficiente.

Subframes: a personalização do conhecimento

Os **subframes** representam a especialização de frames mais gerais, conhecidos como superframes (Oliveira; Carvalho, 2008). Eles herdam todas as propriedades do superframe, mas podem adicionar novas características, modificar slots herdados ou criar exceções para representar de forma precisa os conceitos específicos. Essa abordagem facilita a criação de uma estrutura hierárquica que organiza o conhecimento de maneira lógica e eficiente. Vamos explorar o exemplo de um superframe “**Animal**”:

1. Superframe: Animal
 - **Slots gerais:**
 - **Locomoção:** terrestre, aquático ou aéreo.
 - **Habitat:** variável.
2. Subframe: Mamífero
 - **Herdado de “Animal”:**
 - Locomoção.
 - Habitat.
 - **Adicionado:**
 - **Amamentação:** Sim.
3. Subframe: Ave
 - **Herdado de “Animal”:**
 - Locomoção.
 - Habitat.
 - **Adicionado:**
 - **Capacidade de Voo:** Sim/Não.

Essa estrutura hierárquica permite que o conhecimento seja organizado de forma lógica e eficiente. Informações gerais, tais como tipos de locomoção e habitats, são mantidas no superframe “**Animal**”, enquanto detalhes específicos, como a capacidade de voo ou a característica de amamentar, são adicionados nos subframes “**Mamífero**” e “**Ave**”, respectivamente. Dessa forma, os sistemas de IA podem navegar pela hierarquia utilizando informações gerais ou específicas conforme necessário, promovendo uma abordagem flexível e escalável para o gerenciamento do conhecimento.

Essa abordagem é especialmente poderosa, porque combina consistência e adaptabilidade. Um frame descendente pode tanto reutilizar as características do frame pai quanto sobrescrevê-las para atender às especificidades de uma subcategoria.

Por exemplo, enquanto todos os animais têm um habitat básico, subframes, como “Cachorro”, podem modificar esse valor para refletir a convivência doméstica. Da mesma forma, as regras de herança permitem a criação de exceções, como no caso do “Avestruz”, que, embora faça parte do grupo das aves, não possui capacidade de voo. Essa combinação de herança, modificações e exceções torna os frames uma ferramenta essencial para a modelagem do conhecimento em sistemas de IA.

A Tabela 1 resume esses conceitos, exemplificando como a hierarquia e as adaptações funcionam em um sistema que organiza informações sobre seres vivos.

ASPECTO	DESCRIÇÃO	EXEMPLO
Hierarquia de Herança e Especialização	Frames se tornam mais específicos conforme descemos na hierarquia, permitindo a organização do conhecimento em níveis de generalidade e especificidade.	Frame “Ser Vivo”: slots, como “Ciclo de Vida” e “Capacidade de Reprodução”. Frame “Animal”: herda de “Ser Vivo” e adiciona “Locomoção” e “Habitat”. Frame “Mamífero”: adiciona “Temperatura Corporal” e “Amamentação”. Frame “Cão”: Adiciona “Raça” e “Latido”.
Navegação na Hierarquia	A hierarquia permite que o sistema acesse informações gerais ou específicas, dependendo da necessidade.	Pergunta: “Todos os mamíferos têm temperatura corporal quente?” Resposta: Sim, baseado no superframe “Mamífero”. Pergunta: “O que caracteriza um cão?” Resposta: “Latido” e “Raça”, encontrados no subframe “Cão”.

ASPECTO	DESCRIÇÃO	EXEMPLO
Modificações de Slots Herdados	Frames descendentes podem sobrescrever slots herdados para refletir características específicas de uma subcategoria.	Slot "Habitat" em "Animal": valor padrão "Terrestre". Subframe "Cachorro": modifica para "Doméstico".
Regras de Herança e Exceções	Nem todas as entidades seguem as regras gerais; exceções podem ser definidas para ajustar propriedades herdas.	Slot "Capacidade de Voo" em "Ave": valor padrão "Voa". Subframe "Avestruz": sobrescreve para "Não Voa".

Tabela 1 - Resumo dos conceitos de herança em frames / Fonte: o autor.

Essa estrutura de herança e especialização é particularmente poderosa em IA Simbólica, em que o conhecimento precisa ser categorizado de maneira lógica e inter-relacionada. As capacidades dos frames de herdar e personalizar características em uma hierarquia permitem que os sistemas de IA organizem e interpretem dados complexos com maior precisão e consistência.

INFERÊNCIA EM FRAMES: UMA VISÃO FORMAL

A inferência em frames é o processo em que sistemas de IA deduzem novas informações a partir dos dados explícitos armazenados em um frame. Essa capacidade é fundamental para automatizar o raciocínio e a tomada de decisões. A seguir, apresentamos formalmente os principais tipos de inferência que podem ocorrer em estruturas de frames (Oliveira; Carvalho, 2008):

1. **Inferência por herança:**

Esse tipo de inferência ocorre quando um frame específico herda dados ou propriedades de um frame mais geral. Por exemplo, o frame “Mamífero” herda automaticamente características gerais definidas no frame “Animal”, como “Locomoção” e “Habitat”. Isso evita redundâncias e facilita a organização do conhecimento.

- **Exemplo:**

- Superframe: Animal (Locomoção: Variável, Habitat: Variável).
- Subframe: Mamífero (herda slots de “Animal” + slots próprios como “Amamentação”).

2. **Inferência baseada em condições e facetas:**

Essa inferência utiliza restrições definidas nas facetas dos slots. Facetas, como “valores padrão” e “restrições”, orientam a inferência, ao preencherem automaticamente dados quando informações não são fornecidas ou quando condições específicas são atendidas.

- **Exemplo:** se um frame de “Paciente” possui o slot “Temperatura” com um valor padrão de 36,5°C, esse valor será assumido automaticamente quando não for informado um valor específico.

3. **Inferência por procedimentos associados (demons):**

Os demons são procedimentos acoplados aos slots que são automaticamente disparados quando determinadas condições são atendidas. Esses procedimentos podem realizar cálculos, gerar alertas ou ativar processos específicos.

- **Exemplo:** no frame de “Veículo”, um demon pode ser ativado quando o slot “Nível de Combustível” cair abaixo de um limite, disparando um alerta de “Reabastecimento Necessário”.

ABORDAGEM MATEMÁTICA PARA ESTRUTURAS DE FRAMES

A estrutura matemática de frames é baseada nos conceitos de **teoria dos grafos** e **lógica formal**, proporcionando uma representação robusta e flexível para modelar conhecimento (Russell; Norvig, 2013). Essa abordagem permite organizar

frames de maneira hierárquica, eficiente e lógica, além de habilitar inferências automáticas baseadas nas relações entre frames. Uma das formas de representar um frame e as respectivas relações é por intermédio de um grafo direcionado acíclico, em que:

- **Nós (vertices):** representam frames (entidades ou conceitos).
- **Arestas (edges):** representam relações entre frames, como a herança (“é um”) ou associações (“relacionado a”).

Formalização

Um grafo G é definido como:

$$G=(N,A)$$

Em que:

- N é o conjunto de nós ($N=\{F1,F2,F3,\dots\}$), representando frames.
- $A \subseteq N \times N$ é o conjunto de arestas direcionadas, representando as relações de herança entre os frames.

Cada frame F é formalmente definido como:

$$F = (N, S, A)$$

Em que:

- N : nome do frame.
- $S = \{(s1, v1), (s2, v2), \dots\}$: conjunto de slots, em que si é o nome do slot e vi é o valor associado.
- A : relações do frame com outros frames.

Considere o seguinte exemplo: o frame F , denominado ‘Mamífero’, e suas características. Esse frame pode ser representado formalmente como:

$$F = ('Mamífero', S, A)$$

Em que:

1. Nome do Frame (N):

$N = \text{'Mamífero'}$

2. Conjunto de Slots (S):

$= \{(\text{'Amamentação', True}), (\text{'Temperatura Corporal', 'Quente'})\}$

- Slot 'Amamentação' com valor True, indicando que mamíferos amamentam.
- Slot 'Temperatura Corporal' com valor 'Quente', representando que mamíferos possuem sangue quente.

3. Relações com Outros Frames (A):

$A = \{\text{'herda de Animal'}\}$

O frame 'Mamífero' está relacionado ao superframe 'Animal', do qual herda propriedades mais gerais, como:

- Locomoção: terrestre, aquática ou aérea.
- Habitat: variável.

A abordagem matemática dos frames proporciona uma base estruturada e rigorosa para modelar conhecimento em sistemas de Inteligência Artificial, unindo a flexibilidade dos conceitos da teoria dos grafos com a precisão da lógica formal. Representar frames como grafos direcionados acíclicos (DAGs) permite organizar informações hierarquicamente, com herança de atributos e relações bem definidas entre os conceitos.

Essa estrutura facilita a inferência automática, garantindo que sistemas inteligentes sejam capazes de deduzir informações a partir das relações estabelecidas. Com essa modelagem, os frames se tornam ferramentas poderosas para representar o conhecimento de forma lógica, escalável e consistente, viabilizando aplicações em áreas, como diagnóstico, recomendação e planejamento.

EXEMPLO PRÁTICO: IMPLEMENTANDO FRAMES EM PYTHON

Nesta seção, abordaremos a implementação prática de **frames em Python** como parte do objetivo de conectar teoria e prática em sistemas baseados em frames. Frames são amplamente utilizados em Inteligência Artificial Simbólica para

modelar conhecimento, representando entidades, propriedades e relações entre elas (Oliveira; Carvalho, 2008).

A implementação será feita de forma que (Oliveira; Carvalho, 2008):

- Cada frame seja uma entidade encapsulada com slots (atributos) e relações com outros frames.
- Haja suporte para **inferência lógica**, permitindo que frames mais específicos herdem propriedades de frames gerais.
- A **visualização gráfica** dos frames e das respectivas conexões seja clara, com os slots exibidos ao lado dos nós correspondentes.

Essa abordagem demonstra como **Python** pode ser utilizado para implementar e manipular frames, além de visualizar as conexões hierárquicas. O código a seguir exemplifica essas funcionalidades, alinhando-se à meta de criar soluções práticas e escaláveis para a representação do conhecimento em IA.

A seguir, explicamos cada parte do código, destacando como ele modela essa estrutura e gera representações gráficas.

Definição da classe frame

Primeiramente, vamos criar a classe `Frame`, que representa um conceito ou entidade. Cada frame possui um nome, uma coleção de atributos (slots) e um dicionário de relações (relations) que pode indicar conexões hierárquicas, como herança de propriedades.

```
import networkx as nx
```

```
import matplotlib.pyplot as plt
```

```
# Definição da classe Frame
```

```
class Frame:

    def __init__(self, name, slots=None, relations=None):

        self.name = name # Nome do frame

        self.slots = slots if slots else {} # Dicionário de slots

        self.relations = relations if relations else {} # Relações
        com outros frames
```

Inferência de slots

A classe Frame também possui um método chamado `infer_slots`, que permite a herança de atributos entre os frames. Esse método percorre as relações do frame e, caso haja uma relação de **herança** (“**herda de**”), ele recupera os atributos do frame pai e os combina com os atributos próprios.

```
# Método para realizar inferências lógicas

def infer_slots(self):

    inherited_slots = {}

    for relation, related_frame in self.relations.items():

        if relation == "herda de":

            inherited_slots.update(related_frame.infer_slots()) #
            Propaga slots herdados

    return {**inherited_slots, **self.slots} # Combina slots
    herdados com slots locais
```

Geração da representação gráfica

A seguir, criamos uma função chamada `plot_detailed_frames`, que utiliza a biblioteca `NetworkX` para desenhar a estrutura dos frames e as respectivas relações. A variável `G` é um grafo direcionado, pois as relações entre os frames têm direção (por exemplo, “Mamífero herda de Animal”). Em seguida, adicionamos os **nós** (frames) e as **arestas** (relações) ao grafo.

```
# Função para criar a representação visual dos frames

def plot_detailed_frames(frames):

    G = nx.DiGraph() # Grafo direcionado para representar frames

    # Adicionando nós e arestas ao grafo

    for frame in frames:

        G.add_node(frame.name, slots=frame.slots) # Adiciona o frame como nó

        for relation, related_frame in frame.relations.items():

            G.add_edge(frame.name, related_frame.name, label=relation) # Adiciona arestas com rótulos
```

Para melhorar a visualização, definimos posições fixas para os nós. Essas posições ajudam a organizar o layout do grafo, facilitando a leitura das relações entre os frames.

```
# Configuração do layout do grafo com posições fixas

pos = {
```

```
"Animal": (0, 2),
```

```
"Mamífero": (-1, 0),
```

```
"Ave": (1, 0),
```

```
"Cachorro": (-2, -2),
```

```
}
```

Desenho dos frames e relações

Agora, utilizamos NetworkX e Matplotlib para desenhar os nós e as conexões. Cada **frame** é desenhado como um nó azul-claro, com rótulos para identificá-los. As relações são representadas por **setas pretas** indicando a direção da herança.

```
# Desenha os nós
```

```
nx.draw_networkx_nodes(G, pos, node_color="lightblue",
node_size=2000)
```

```
nx.draw_networkx_labels(G, pos, labels={node: node for
node in G.nodes}, font_size=12)
```

```
# Desenha as arestas com setas visíveis
```

```
nx.draw_networkx_edges(G, pos, edge_color="black", ar-
rows=True, arrowstyle="->", connectionstyle="arc3,rad=0.1")
```

Também adicionamos os rótulos das arestas para indicar as relações. Isso permite que o gráfico exiba claramente qual frame **herda** do outro. Além disso, incluímos os **slots** (atributos dos frames) ao lado de cada nó. Com isso, podemos visualizar os **atributos** de cada frame diretamente no grafo. Por fim, removemos os eixos do gráfico e exibimos a visualização.

```
# Adiciona rótulos às arestas

edge_labels = nx.get_edge_attributes(G, 'label')

nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=10)

# Adicionando os slots ao lado dos nós

for node, (x, y) in pos.items():

    slots = "\n".join([f"{k}: {v}" for k, v in G.nodes[node].get('slots', {}).items()])

    plt.text(x + 0.5, y, slots, fontsize=9, ha='left', color='darkgreen') # Ajustado para posição ao lado

plt.title("Representação Visual dos Frames e Relações")

plt.axis("off") # Remove os eixos

plt.show()
```

Definição de frames e herança

Agora que temos a estrutura pronta, criamos alguns frames de exemplo e estabelecemos as respectivas relações:

```
# Exemplo de frames

animal = Frame("Animal", slots={"Locomoção": "Variável",
                                "Habitat": "Variável"})
```

```
mamifero = Frame("Mamífero", slots={"Amamentação": True,
"Temperatura Corporal": "Quente"}, relations={"herda de":
animal})
```

```
ave = Frame("Ave", slots={"Capacidade de Voo": "Sim"}, re-
lations={"herda de": animal})
```

```
cachorro = Frame("Cachorro", slots={"Raça": "Labrador"},
relations={"herda de": mamifero})
```

Nesse exemplo:

- “Animal” é um frame genérico que define “Locomoção” e “Habitat” como variáveis.
- “Mamífero” herda de “Animal” e adiciona “Amamentação” e “Temperatura Corporal”.
- “Ave” também herda de “Animal” e define “Capacidade de Voo”.
- “Cachorro” herda de “Mamífero” e possui um slot específico para definir sua raça.

Esses frames são organizados em uma lista. Assim, passamos essa lista para a função que desenha o grafo:

```
# Lista de frames
```

```
frames = [animal, mamifero, ave, cachorro]
```

```
plot_detailed_frames(frames)
```

Inferência de slots

Por fim, usamos o método `infer_slots()` para exibir os atributos inferidos de cada frame. Esse código percorre todos os frames e imprime os atributos herdados, demonstrando o funcionamento da **inferência hierárquica**.

```
# Exibir inferências no console
```

```
for frame in frames:
```

```
    print(f"Frame: {frame.name}")
```

```
    print("Slots inferidos:", frame.infer_slots())
```

```
    print("-" * 50)
```

A implementação em Python retorna o seguinte resultado:

- Representação visual

Representação visual dos frames e relações

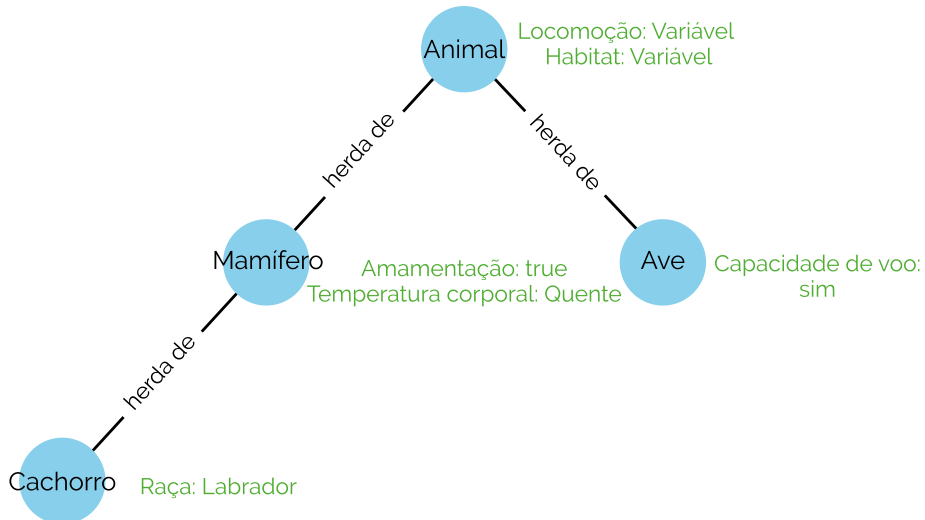


Figura 1 - Estrutura de frames e herança de um sistema que organiza informações sobre seres vivos / Fonte: o autor.

Descrição da Imagem: é exibida uma representação visual de frames e relações entre diferentes categorias de animais. A estrutura é composta por círculos conectados por linhas, indicando a herança de características entre as categorias. No topo, há um círculo rotulado como "Animal", o qual exibe as características: "Locomoção: Variável" e "Habitat: Variável". Esse círculo está conectado a dois outros círculos: "Mamífero" e "Ave".

O círculo "Mamífero" herda do "Animal" e possui as características: "Amamentação: True" e "Temperatura Corporal: Quente". Esse círculo está conectado a um círculo menor rotulado como "Cachorro", que herda de "Mamífero" e possui a característica "Raça: Labrador". O círculo "Ave" também herda do "Animal" e tem a característica "Capacidade de Voo: Sim". Fim da descrição.

■ Inferências

Frame: Animal

Slots inferidos: {'Locomoção': 'Variável', 'Habitat': 'Variável'}

Frame: Mamífero

Slots inferidos: {'Locomoção': 'Variável', 'Habitat': 'Variável', 'Amamentação': True, 'Temperatura Corporal': 'Quente'}

Frame: Ave

Slots inferidos: {'Locomoção': 'Variável', 'Habitat': 'Variável', 'Capacidade de Voo': 'Sim'}

Frame: Cachorro

Slots inferidos: {'Locomoção': 'Variável', 'Habitat': 'Variável', 'Amamentação': True, 'Temperatura Corporal': 'Quente', 'Raça': 'Labrador'}

Exploramos as abordagens prática e teórica de frames em sistemas baseados em conhecimento, uma técnica poderosa na Inteligência Artificial Simbólica. Iniciamos com uma introdução aos conceitos fundamentais de frames, discutindo estruturas, hierarquias, heranças e importância das respectivas aplicações em modelagem de conhecimento. Avançamos para uma análise detalhada da abordagem matemática para estruturas de frames, conectando-as à teoria dos grafos e à lógica formal. A implementação prática em **Python** mostrou como transformar conceitos teóricos em soluções computacionais.

**EM FOCO**

Gostou do que discutimos até aqui? Tenho mais para conversar com você a respeito deste tema. Vamos lá? **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A aplicação de frames, estruturados a partir de conceitos de hierarquia e herança, combinada com a programação em Python, destaca-se como uma competência essencial para o mercado de trabalho atual. Essa abordagem permite organizar e modelar conhecimento de forma eficiente, facilitando a criação de sistemas inteligentes em áreas, como tecnologia educacional, saúde, logística e gestão de informações.

Frames oferecem uma maneira prática de representar objetos e as respectivas características por meio de slots, agregando informações de maneira estruturada e lógica. Em sistemas educacionais, por exemplo, frames podem ser utilizados para modelar o progresso acadêmico de alunos, incluindo informações, como desempenho, interesses e habilidades. Essa estrutura auxilia na personalização de recomendações de materiais de estudo, otimizando a experiência de aprendizado.

A utilização de Python, que apresenta simplicidade e versatilidade, é crucial para transformar a teoria em prática. Ferramentas, como **NetworkX e Matplotlib**, permitem visualizar hierarquias e conexões entre frames, enquanto estruturas de código facilitam a implementação de inferências baseadas em heranças. No caso de um sistema de diagnóstico médico, por exemplo, um superframe “Paciente” pode ser conectado a subframes, como “Gripe” ou “Diabetes”, ajudando médicos a identificar sintomas e planejar tratamentos.

Essa habilidade prática de criar e manipular frames é cada vez mais valorizada em um mercado que exige precisão e eficiência na organização de dados. Profissionais que dominam esses conceitos não apenas ampliam as capacidades técnicas, mas também contribuem para a criação de soluções transparentes e escaláveis. A integração de frames no ambiente profissional promove não apenas o desenvolvimento de ferramentas inovadoras, mas também a tomada de decisões mais informadas e assertivas.

Estudante, para a sua atuação profissional, basta incluir todo o aprendizado adquirido ao analisar como o uso de frames pode otimizar processos em sua área de atuação. A partir dessa abordagem, você poderá entender como uma estrutura bem definida pode resolver problemas complexos, aumentar a produtividade e criar sistemas mais acessíveis e adaptáveis às necessidades reais.



AUTOATIVIDADE

1. Frames são estruturas de dados projetadas para organizar informações de forma lógica e estruturada. Eles permitem modelar conceitos, entidades ou situações com alto grau de detalhamento, facilitando a compreensão e o processamento por sistemas inteligentes. Uma das principais características dos frames é a hierarquia, que permite que informações gerais sejam reutilizadas por subestruturas mais específicas, reduzindo redundâncias e promovendo consistência nos dados representados.

Qual elemento é essencial para a definição de um frame em IA?

- a) Algoritmos de busca.
 - b) Relações de similaridade.
 - c) Slots que armazenam informações específicas.
 - d) Funções de transferência.
 - e) Inferências probabilísticas.
2. A inferência automática em frames envolve a análise de conexões e relações representadas na estrutura dos frames. Essas relações ajudam os sistemas de IA a compreender e a processar informações de forma lógica, utilizando os vínculos definidos entre conceitos.

Considerando as características que permitem inferências automáticas em frames, analise as afirmativas a seguir:

- I - Relações hierárquicas entre frames.
- II - Conexões entre slots e facetas.
- III - Algoritmos probabilísticos externos.
- IV - Organização de dados não estruturados.

É correto o que se afirma em:

- a) I e II, apenas.
- b) III e IV, apenas.
- c) II e IV, apenas.
- d) I e III, apenas.
- e) I, II, III e IV.

3. A inferência em frames é um processo que utiliza informações estruturadas para produzir conclusões. Essa técnica é amplamente aplicada em sistemas de Inteligência Artificial que necessitam conectar informações previamente organizadas.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - Frames permitem inferências automáticas.

PORQUE

II - Apenas relações hierárquicas possibilitam inferências.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

ARAGÃO, J. F. B. **Sistemas especialistas como ferramenta auxiliar para o ensino da disciplina Bases da Técnica Cirúrgica**. Campina Grande: Universidade Federal de Campina Grande, Centro de Ciências e Tecnologia, Coordenação de Pós-Graduação em Informática, 2002.

BRACHMAN, R. J.; LEVESQUE, H. J. **Knowledge Representation and Reasoning**. [S. l.]: Morgan Kaufmann, 2004.

LIETO, A. **Cognitive Design for Artificial Minds**. [S. l.]: Routledge, 2021.

OLIVEIRA, H. C. de; CARVALHO, C. L. de. **Gestão e Representação do Conhecimento**. [S. l.: s. n.], 2008.

PATTERSON, D. W. **Introduction to Artificial Intelligence and Expert Systems**. Englewood Cliffs: Prentice-Hall, 1990.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. [S. l.]: Prentice Hall, 2013.

GABARITO

1. Opção C.

Os slots são os componentes fundamentais de um frame, responsáveis por armazenar informações específicas. As alternativas A, B, D e E estão incorretas, porque descrevem elementos que não fazem parte da estrutura essencial de frames ou que são secundários ao conceito.

2. Opção A.

Relações hierárquicas (I) e conexões entre slots e facetas (II) são características fundamentais para permitir inferências automáticas em frames. As afirmativas III e IV estão incorretas, porque algoritmos probabilísticos externos e dados não estruturados não são usados diretamente para a inferência em frames.

3. Opção C.

A asserção "Frames permitem inferências automáticas" é verdadeira. No entanto, a asserção "Apenas relações hierárquicas possibilitam inferências" é falsa, pois as inferências também podem ocorrer por meio de relações associativas e de parte-todo.



TEMA DE APRENDIZAGEM 9

APLICAÇÕES PRÁTICAS DE IA SIMBÓLICA

MINHAS METAS

- Explorar as aplicações práticas de IA simbólica.
- Introduzir a abordagem matemática para IA simbólica.
- Analisar a otimização de algoritmos simbólicos.
- Implementar projetos em Python.
- Compreender a construção de redes semânticas.
- Utilizar frames em aplicações práticas.
- Fomentar a aplicabilidade profissional de IA simbólica.

INICIE SUA JORNADA

Você já parou para pensar no que está por trás das ferramentas que usamos todos os dias, como assistentes virtuais, motores de busca e sistemas de recomendação? Essas tecnologias, que parecem tão intuitivas, dependem de estruturas inteligentes para compreender, organizar e responder às nossas necessidades. Elas precisam ser capazes de processar grandes volumes de dados de forma eficiente, capturando não apenas informações explícitas, mas também diferenças contextuais, como o momento, as intenções e as preferências do usuário. No entanto, como projetar esses sistemas de maneira eficaz? Como garantir que eles possam lidar com problemas complexos, evoluir com as demandas dos usuários e, ainda assim, ser rápidos e confiáveis?

A Inteligência Artificial (IA) simbólica desempenha um papel essencial na criação dessas ferramentas. **Ela utiliza métodos baseados em lógica, representações formais e matemática para estruturar e processar o conhecimento** (Russell; Norvig, 2013). Por intermédio de redes semânticas, por exemplo, podemos organizar informações em grafos que representam relações entre conceitos. Esses grafos permitem que um sistema compreenda que “um cachorro é um animal” ou que “um tablet pode ser tanto um medicamento quanto um dispositivo eletrônico”, dependendo do contexto (Luger, 2021).

Por outro lado, os frames fornecem molduras para organizar essas informações em contextos específicos (Oliveira; Carvalho, 2008). Imagine um assistente virtual que reconheça a diferença entre “abrir um arquivo” e “abrir uma conta bancária” com base no cenário em que a interação ocorre. Essas ferramentas não apenas viabilizam respostas mais precisas, mas também tornam a experiência do usuário mais fluida e personalizada.

Para **compreender e aplicar** essas ideias, é necessário mergulhar na modelagem matemática e no desenvolvimento computacional. Modelos matemáticos nos ajudam a otimizar algoritmos de busca e inferência, permitindo que sistemas explorem eficientemente as vastas possibilidades de redes semânticas e estruturas simbólicas (Paterson, 1990). Por meio da programação em Python, uma linguagem acessível e poderosa, você terá a oportunidade de criar projetos que combinam esses conceitos teóricos em aplicações práticas. Imagine desenvolver um motor de busca que vá além de palavras-chave, interpretando perguntas complexas, como “Qual é o melhor destino para férias de inverno com crianças

pequenas?”, e retornando respostas realmente relevantes. Outra possibilidade seria criar um sistema de recomendação que, além de sugerir filmes baseados no histórico de visualizações, considere fatores, como o humor do usuário, o horário do dia ou os eventos ao redor.

Essas habilidades podem transformar profundamente a sua atuação profissional. Ao aplicar IA simbólica, você contribui para desenvolver tecnologias que transcendem a simples automação, introduzindo inovações capazes de melhorar a tomada de decisão, a produtividade e a qualidade de vida das pessoas. No entanto, o potencial dessas tecnologias também traz responsabilidades. É essencial refletir sobre questões éticas e sociais, como o impacto do uso de dados pessoais e a necessidade de transparência nos processos decisórios. Como garantir que as soluções criadas sejam inclusivas, justas e livres de vieses prejudiciais? Como equilibrar eficiência e precisão com o respeito à privacidade e aos valores humanos? Essa reflexão sobre o equilíbrio entre técnica e responsabilidade será um diferencial no seu desenvolvimento como profissional de tecnologia.



PLAY NO CONHECIMENTO

Neste episódio do nosso podcast, mergulhamos no fascinante universo da IA simbólica aplicada à personalização de experiências. Descubra como sistemas inteligentes estão criando interações mais humanas e relevantes em áreas, como educação e entretenimento. Venha explorar como a tecnologia pode moldar o futuro da personalização e inspirar novas possibilidades! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**



VAMOS RECORDAR?

Para garantir que você esteja preparado para compreender e aplicar os conceitos de IA simbólica, vamos relembrar alguns conteúdos essenciais. Primeiro, é importante entender como a lógica proposicional e de predicados serve como base para a representação simbólica, permitindo organizar e processar informações de forma estruturada.

Além disso, explorar as bases epistemológicas da IA ajuda a compreender os limites e desafios dessa abordagem. O artigo *Por trás da inteligência artificial: uma análise das bases epistemológicas do aprendizado de máquina* analisa a forma como a indução e a matematização moldam os fundamentos da IA, destacando as respectivas fragilidades e implicações filosóficas. Saiba mais no link a seguir.

DESENVOLVA SEU POTENCIAL

APLICAÇÕES PRÁTICAS DA IA SIMBÓLICA

A Inteligência Artificial Simbólica está transformando a maneira como enfrentamos desafios complexos, apresentando soluções robustas, explicáveis e adaptáveis. A capacidade dela de modelar explicitamente conceitos, regras e condições diferencia essa abordagem de outros paradigmas, como o aprendizado profundo, destacando-se especialmente em áreas nas quais explicabilidade, precisão e contextualização são fundamentais (Russell; Norvig, 2013).

A IA Simbólica utiliza ferramentas essenciais, como redes semânticas, frames e algoritmos simbólicos, para criar sistemas que não apenas compreendem informações, mas também agem com base em **raciocínio lógico** (Gonzales *et al.*, 2013). As redes semânticas organizam os conceitos em estruturas hierárquicas e relacionais. Os frames ajustam essas informações de acordo com o contexto específico, enquanto os algoritmos simbólicos garantem que as inferências sejam otimizadas e direcionadas a soluções práticas (Oliveira; Carvalho, 2008).

Essa abordagem tem sido amplamente usada em sistemas de recomendação, motores de busca e assistentes virtuais. Esses sistemas não apenas resolvem problemas de forma eficiente, mas também criam oportunidades para melhorar a interação humano-máquina e oferecer experiências personalizadas.

Sistemas de recomendação

Os sistemas de recomendação são ferramentas indispensáveis no **cotidiano digital**, ajudando a filtrar informações e a fornecer sugestões personalizadas com base nos interesses e no comportamento dos usuários. A aplicação de IA Simbólica torna esses sistemas ainda mais sofisticados, combinando redes semânticas para mapear relações entre conceitos e frames para adaptar recomendações a contextos específicos.

No caso de um sistema de recomendação, as redes semânticas são usadas para organizar informações em grafos, em que nós representam entidades (como filmes, livros ou músicas) e arestas representam as relações entre essas entidades. Por exemplo, em uma rede semântica de filmes, o nó “comédia” pode estar conectado a títulos, como *O Máskara* e *As Branquelas*, formando um mapa relacional de gêneros, atores e temas.

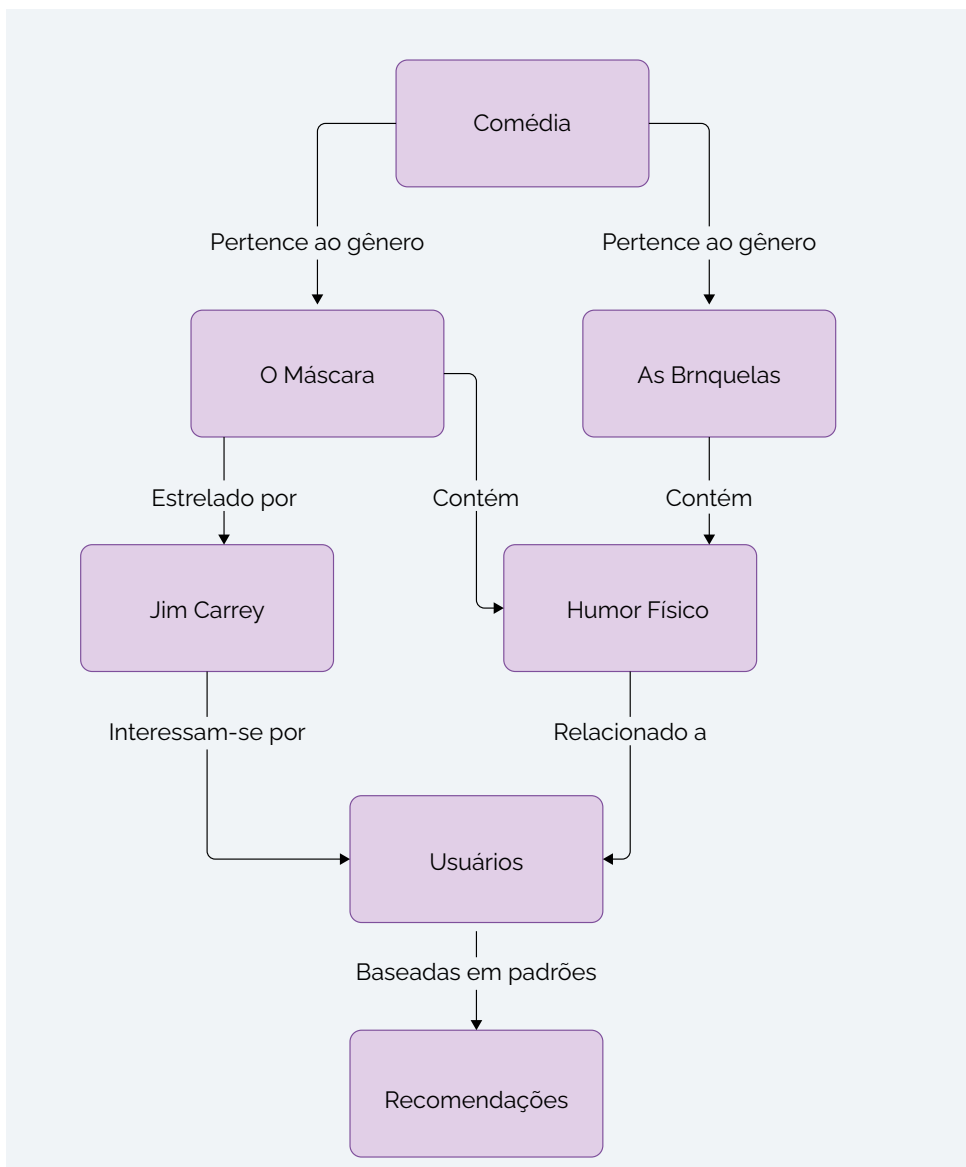


Figura 1 - Rede semântica para recomendação de filmes de comédia / Fonte: o autor.

Descrição da Imagem: é exibido um diagrama de relacionamento no formato de um mapa conceitual sobre o gênero "Comédia". A seguir, estão os elementos e as conexões descritas: Comédia: está no topo do diagrama e é o gênero central. Possui duas ramificações que levam a: O Máscara (um filme), com a informação adicional "Estrelado por Jim Carrey"; e As Branquelas (outro filme), com a característica "Contém Humor Físico". Usuários: relacionam-se com "Humor Físico" (ligado ao filme As Branquelas) e com "Jim Carrey" (ligado ao filme O Máscara). Baseados nesses padrões de interesse, geram Recomendações. Fim da descrição.

Essa estrutura de rede semântica permite que o sistema vá além de simples associações, identificando padrões profundos e conexões entre preferências. Por exemplo, ao analisar que os “usuários que assistem a comédias estreladas por Jim Carrey frequentemente têm interesse em filmes com humor físico”, o sistema pode sugerir títulos similares, ampliando a relevância das recomendações. Os frames tornam os sistemas de recomendação mais flexíveis, ao considerarem variáveis contextuais. Eles estruturam informações relacionadas ao usuário, ao contexto e aos itens recomendados, permitindo uma personalização mais avançada. Por exemplo, um sistema pode ajustar as sugestões com base no horário ou no estado emocional do usuário. Em uma sexta-feira à noite, ele pode recomendar “filmes leves e divertidos”, enquanto, em uma manhã de domingo, pode sugerir “documentários inspiradores”. Observe, a seguir, o **frame do usuário** que modela informações que influenciam diretamente as recomendações, como as preferências e o contexto atual.

ATRIBUTOS	DEFAULT	TIPO	SE-NECESSÁRIO
Nome	Usuário1	símbolo	-
Preferências	Comédia, Ação	lista/símbolo	Obrigatório
Histórico	["O Máskara"]	lista/símbolo	Opcional (pode ser usado para evitar repetições)
Estado Emocional	Animado	símbolo	Dinâmico (pode ser atualizado em tempo real)
Horário	Sexta à noite	símbolo	Dinâmico (atualizado pelo sistema com base no tempo atual)
Sugestão Atual	Filmes leves e divertidos	símbolo	Derivado do estado emocional e do horário

Quadro 1 - Representação do frame do usuário / Fonte: o autor.

Esse modelo de frame permite que o sistema de recomendação capture o contexto e as preferências do usuário em tempo real, oferecendo sugestões personalizadas que atendem às necessidades específicas em diferentes momentos. Isso aumenta a relevância e melhora a experiência geral do usuário ao interagir com o sistema.

Motores de busca

Os motores de busca são ferramentas essenciais no mundo digital, permitindo que os usuários encontrem informações relevantes de forma rápida e eficiente (Lewandowski, 2023). Com a aplicação de IA Simbólica, esses sistemas se tornam ainda mais avançados, utilizando redes semânticas para compreender melhor as relações entre conceitos e frames, a fim de personalizar os resultados com base no contexto e nas intenções do usuário (Russell; Norvig, 2013).

No caso de um motor de busca, as redes semânticas são usadas para organizar informações em grafos, em que nós representam conceitos (como palavras-chave ou tópicos) e arestas representam as relações entre esses conceitos (Russell; Norvig, 2013). Por exemplo, em uma rede semântica para buscas, o termo “carro elétrico” pode estar conectado a conceitos, tais como “sustentabilidade”, “bateria” e “veículos ecológicos”, formando um mapa relacional de sinônimos, tópicos relacionados e categorias.

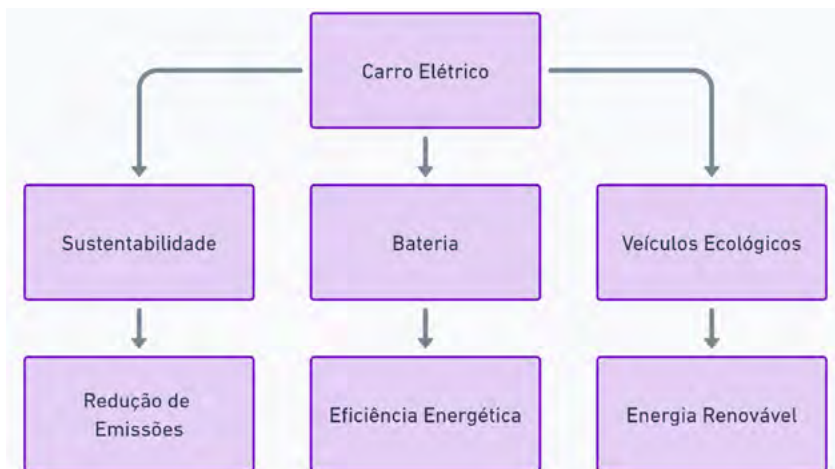


Figura 2 - Rede semântica para a organização de resultados de busca / Fonte: o autor.

Descrição da Imagem: é exibida uma rede semântica que organiza informações relacionadas ao termo central “Carro Elétrico”.

Esse termo principal está conectado diretamente a três subconceitos: “Sustentabilidade”, “Bateria” e “Veículos Ecológicos”. Cada um desses subconceitos, por sua vez, está ligado a conceitos mais específicos. “Sustentabilidade” se relaciona com “Redução de Emissões”, enquanto “Bateria” está associado à “Eficiência Energética”. Já “Veículos Ecológicos” está conectado a “Energia Renovável”. Fim da descrição.

Essa estrutura de rede semântica permite que o motor de busca vá além da correspondência exata de palavras-chave, compreendendo o significado por trás das consultas (Bast; Buchhold; Haussmann, 2016). Por exemplo, ao interpretar uma pesquisa como “melhores opções de transporte ecológico”, o sistema pode conectar “transporte ecológico” a “carro elétrico” e a “bicicleta elétrica”, apresentando resultados mais relevantes para a intenção do usuário.

Os frames tornam os motores de busca mais flexíveis, ao considerarem as variáveis contextuais, como a localização, o dispositivo utilizado ou o histórico de pesquisas (Russell; Norvig, 2013). Eles estruturam as informações relacionadas ao usuário e ao contexto, permitindo uma personalização avançada dos resultados de busca.

ATRIBUTOS	DEFAULT	TIPO	SE-NECESSÁRIO
NOME	Usuário1	símbolo	-
HISTÓRICO DE BUSCA	["carro elétrico", "bicicleta elétrica"]	lista/símbolo	Opcional (pode ser usado para sugerir tópicos relacionados)
LOCALIZAÇÃO	São Paulo	símbolo	Dinâmico (atualizado em tempo real)
DISPOSITIVO	Smartphone	símbolo	Dinâmico (detectado automaticamente)

ATRIBUTOS	DEFAULT	TIPO	SE-NECESSÁRIO
HORÁRIO	Segunda de manhã	símbolo	Dinâmico (atualizado pelo sistema com base no tempo atual)
INTENÇÃO ATUAL	Transporte ecológico	símbolo	Derivado do histórico e da consulta atual

Quadro 2 - Frame: Usuário / Fonte: o autor.

Se o **histórico de busca** incluir termos, como “bicicleta elétrica” e “carro elétrico”, e a **localização** for São Paulo, o motor de busca pode priorizar resultados sobre lojas ou opções disponíveis na cidade. Além disso, se o **dispositivo** for um smartphone, pode otimizar os resultados para serem visualizados em telas menores. Esse modelo de frame para motores de busca permite capturar o contexto e as intenções do usuário em tempo real, otimizando a experiência de busca, ao fornecer resultados que atendem às necessidades específicas em diferentes cenários. A integração de redes semânticas e frames torna os motores de busca mais inteligentes e úteis, melhorando significativamente a relevância dos resultados apresentados (Chen; Décary, 2018).

Assistentes virtuais

Os assistentes virtuais são ferramentas que utilizam a **IA Simbólica** para oferecer suporte personalizado aos usuários. Ao combinarem **redes semânticas** para organizar informações e **frames** para ajustar as interações com base no contexto, esses sistemas conseguem entender comandos complexos, fornecer respostas relevantes e executar tarefas com eficiência (Russell; Norvig, 2013).

No caso de um assistente virtual, as **redes semânticas** ajudam a organizar informações e ações em grafos que conectam conceitos e funcionalidades relacionadas (Gonzales *et al.*, 2013). Por exemplo: um comando, como “*Ligue as luzes do quarto e ajuste a temperatura para 22 graus*”, é interpretado conectando os conceitos:

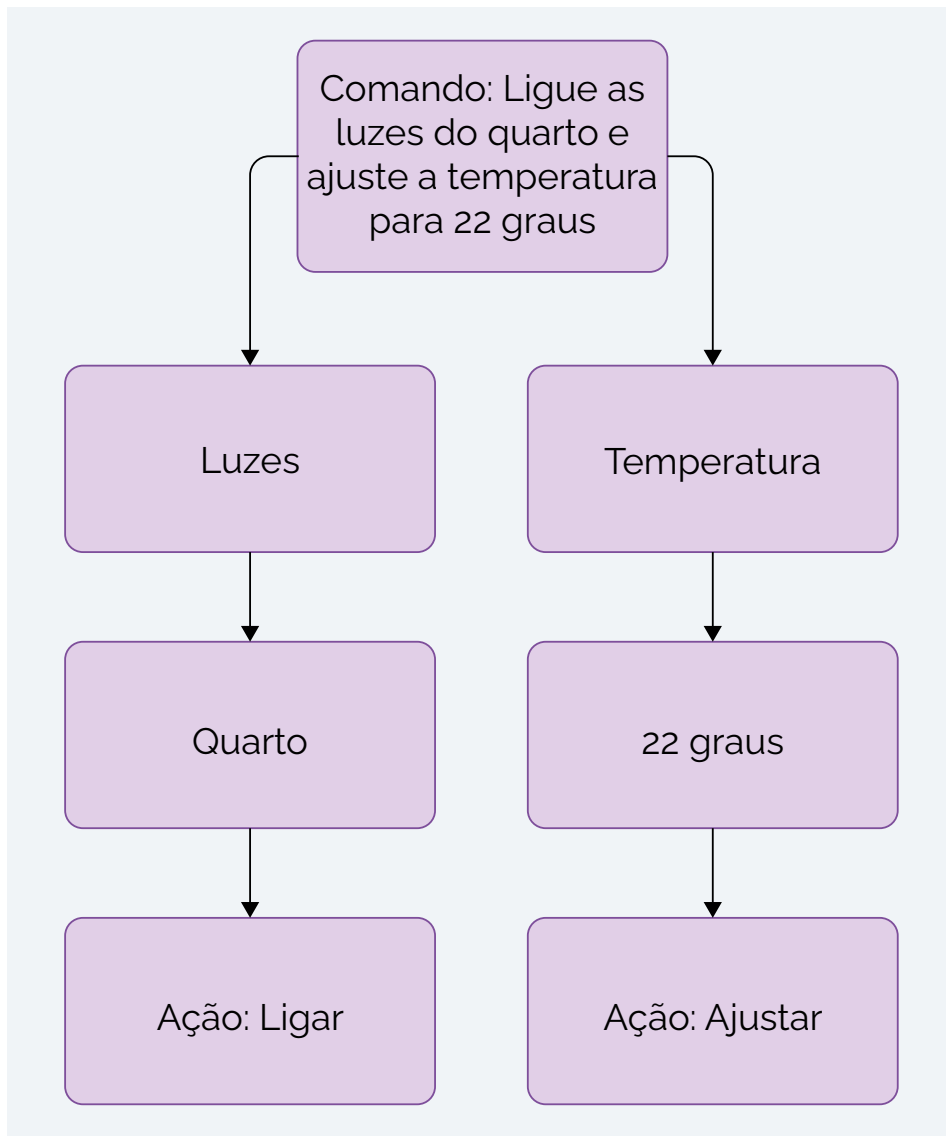


Figura 3 - Rede semântica para assistente virtual / Fonte: o autor.

Descrição da Imagem: é exibida uma rede semântica que organiza o comando de automação “Ligue as luzes do quarto e ajuste a temperatura para 22 graus”. O comando é dividido em dois componentes principais: “Luzes” e “Temperatura”. O componente “Luzes” está associado ao local “Quarto”, que, por sua vez, está vinculado à ação “Ligar”. Já o componente “Temperatura” está relacionado ao valor “22 graus”, o qual está conectado à ação “Ajustar”. Essa estrutura hierárquica demonstra como o comando é interpretado e segmentado em partes específicas, identificando os objetos (luzes e temperatura), os contextos (quarto e 22 graus) e as ações a serem realizadas (ligar e ajustar). Esse modelo facilita a compreensão e a execução de comandos complexos em sistemas de automação. Fim da descrição.

Essas conexões são mapeadas em um grafo semântico, permitindo que o assistente compreenda a intenção do comando e execute múltiplas ações relacionadas.

Os frames tornam os assistentes virtuais mais flexíveis, ao estruturarem as informações do usuário, do ambiente e da tarefa. Eles capturam atributos, como localização, dispositivo, horário e preferências, ajustando a interação com base no contexto (Russell; Norvig, 2013).

ATRIBUTOS	DEFAULT	TIPO	SE-NECESSÁRIO
NOME	Usuário1	símbolo	-
PREFERÊNCIAS	Luzes baixas à noite	lista/símbolo	Opcional (para personalização)
LOCALIZAÇÃO	Quarto	símbolo	Dinâmico (detectado automaticamente)
DISPOSITIVO	Smart Speaker	símbolo	Dinâmico (detectado automaticamente)
HORÁRIO	Sexta à noite	símbolo	Dinâmico (atualizado pelo sistema)
AÇÃO SOLICITADA	Ajustar luz e temperatura	símbolo	Derivado do comando atual

Quadro 3 - Exemplo: Frame do Usuário / Fonte: o autor.

Ao integrarem **redes semânticas** e **frames**, os assistentes virtuais conseguem compreender comandos complexos e oferecer respostas personalizadas que atendem às necessidades específicas dos usuários (Russell; Norvig, 2013). Isso torna as interações mais naturais, eficientes e alinhadas ao contexto, melhorando significativamente a experiência geral com o assistente.

**aprofundando**

Além de motores de busca, sistemas de recomendação e assistentes virtuais, as redes semânticas têm aplicações em outras áreas, tais como educação, conectando tópicos para aprendizagem adaptativa; saúde, relacionando sintomas e tratamentos; setor jurídico, organizando leis e casos; e e-commerce, analisando preferências de clientes. Elas também são úteis na pesquisa científica, descobrindo conexões ocultas entre dados. Essas aplicações destacam o potencial delas em diversos domínios que demandam organização e compreensão contextual de grandes volumes de informações (Russell; Norvig, 2013).

FUNDAMENTOS MATEMÁTICOS E OTIMIZAÇÃO NA IA SIMBÓLICA

A matemática é o alicerce da Inteligência Artificial Simbólica, fornecendo os conceitos e as ferramentas necessários para modelar problemas, representar conhecimento e desenvolver algoritmos eficientes (Russell; Norvig, 2013). Combinando fundamentos lógicos, estruturas matemáticas e técnicas de otimização, a IA simbólica se torna prática e aplicável a diversos cenários complexos.

A matemática é o alicerce da Inteligência Artificial Simbólica

Conceitos fundamentais:

LÓGICA PROPOSICIONAL E DE PREDICADOS

Esses conceitos fornecem a base para a representação do conhecimento em IA simbólica (Russell; Norvig, 2013). A lógica proposicional lida com sentenças simples (como "Se A, então B"), enquanto a lógica de predicados permite a descrição de relações mais complexas, como "Se um usuário gosta de comédia, então, recomende filmes estrelados por Jim Carrey". Esses modelos permitem que sistemas realizem inferências dedutivas e tomem decisões com base em regras explícitas.

GRAFOS E REDES SEMÂNTICAS

Grafos são ferramentas poderosas para estruturar dados em redes de nós (entidades) e arestas (relações). Redes semânticas, como grafos especializados, organizam informações de forma lógica e hierárquica (Luger, 2021). Por exemplo, em um sistema de recomendação, o nó "Comédia" pode estar conectado aos filmes O Máskara e As Branqueelas, que, por sua vez, são vinculados a atores, tais como Jim Carrey, e estilos, como "Humor Físico". Essa estrutura permite que o sistema identifique conexões relevantes e navegue eficientemente por grandes volumes de informações.

ALGORITMOS DE BUSCA

A eficiência na navegação por estruturas simbólicas, como árvores de decisão e redes semânticas, é garantida por algoritmos de busca. Estratégias, como busca em largura, busca em profundidade e algoritmos heurísticos, ajudam a encontrar soluções de forma sistemática, priorizando caminhos mais promissores e relevantes para o problema em questão (Russell; Norvig, 2013).

■ Técnicas de otimização

Para que a IA simbólica seja prática e eficiente, os algoritmos precisam ser otimizados. As principais técnicas incluem (Russell; Norvig, 2013):

Podas Alfa-Beta:

Amplamente utilizadas em árvores de decisão, como em jogos de estratégia, essas podas eliminam caminhos que não impactam o resultado, reduzindo significativamente o número de nós explorados e melhorando o desempenho (Russell; Norvig, 2013). Imagine que você está avaliando várias opções para decidir o melhor caminho para resolver um problema. Durante a análise, você percebe que algumas opções já levam a resultados claramente ruins. Nesse momento, você ignora essas possibilidades e foca nas alternativas mais promissoras, economizando tempo e energia.

Em termos computacionais, as podas Alfa-Beta funcionam assim: quando o sistema encontra um ramo (um conjunto de escolhas ou possibilidades) que não contribui para melhorar o resultado, ele “poda” esse ramo, ou seja, para de analisá-lo (Russell; Norvig, 2013). Isso reduz drasticamente o número de análises necessárias, tornando o processo muito mais rápido e eficiente.

Heurísticas:

Estratégias baseadas em aproximações que direcionam a busca para soluções mais prováveis, economizando tempo e recursos computacionais (Luger, 2021). Por exemplo, em uma rede semântica, heurísticas podem priorizar conexões mais relevantes, tais como gêneros de filmes favoritos ou atores mais assistidos. Em um sistema de recomendação de filmes, uma heurística pode priorizar os gêneros que o usuário mais assiste, como comédias ou documentários, ignorando categorias menos relevantes para aquele perfil. Além disso, ao buscar caminhos em um grafo de rede semântica, a heurística pode priorizar conexões de nós mais “populares”, tais como atores de grande renome ou temas recorrentes, economizando tempo na geração de recomendações.

Avaliação de Complexidade Computacional:

Analisar a eficiência dos algoritmos em termos de tempo e espaço de execução é crucial para garantir que eles possam ser escalados para problemas maiores. Isso inclui ajustar os algoritmos para atender às demandas práticas sem sacrificar o desempenho. Imagine um algoritmo de busca em grafos que utiliza busca em profundidade para encontrar um nó específico. Se o grafo for muito grande, a busca pode se tornar inviável devido à quantidade de nós explorados. Para otimizar, pode-se usar a busca A^* , que combina heurísticas e custos acumulados para priorizar os caminhos mais curtos, garantindo um desempenho melhor em tempo de execução (Russell; Norvig, 2013).

Implementando projetos em Python com IA Simbólica

Python é uma das linguagens mais utilizadas para o desenvolvimento de projetos de IA, incluindo aplicações de IA Simbólica. A simplicidade, a versatilidade e a vasta biblioteca de ferramentas dela tornam a criação de soluções simbólicas prática e eficiente (Joshi, 2017). Nesta seção, exploraremos passo a passo um projeto que combina os conceitos de redes semânticas, algoritmos de busca e personalização.

Objetivo: criar um sistema que analise preferências do usuário em filmes e forneça recomendações personalizadas com base em uma rede semântica de conexões entre gêneros, filmes, atores e estilos de humor.

Passo 1: Construção da rede semântica

O código a seguir cria uma estrutura de grafo utilizando a biblioteca `networkx` para representar uma rede semântica. Um grafo é formado por nós, que representam entidades, e arestas, as quais indicam relações entre essas entidades. No código, o grafo é criado como não-direcionado, ou seja, as conexões entre os nós não têm direção, o que permite que a relação entre dois nós seja bidirecional.

```
import networkx as nx

# Criando o grafo
G = nx.Graph()

# Adicionando nós
nodes = [
    "Comédia", "O Máskara", "As Branquelas",
    "Jim Carrey", "Marlon Wayans", "Humor Físico",
    "Narrativas Familiares"
]
```

```
G.add_nodes_from(nodes)

# Adicionando arestas

edges = [

    ("Comédia", "O Máskara"),

    ("Comédia", "As Branquelas"),

    ("O Máskara", "Jim Carrey"),

    ("As Branquelas", "Marlon Wayans"),

    ("O Máskara", "Humor Físico"),

    ("As Branquelas", "Humor Físico"),

    ("Comédia", "Narrativas Familiares")]

G.add_edges_from(edges)
```

Primeiro, é definida uma lista de nós contendo elementos, como “Comédia”, “O Máskara”, “As Branquelas”, “Jim Carrey”, “Marlon Wayans”, “Humor Físico” e “Narrativas Familiares”. Esses nós representam diferentes categorias, como gêneros de filmes, títulos, atores e estilos de humor. Em seguida, os nós são adicionados ao grafo utilizando o método `add_nodes_from`.

Depois, uma lista de arestas é definida para estabelecer as conexões entre os nós. Essas arestas indicam relações, como o pertencimento de filmes a um gênero, a associação de atores a filmes ou a presença de estilos de humor em produções cinematográficas. Por exemplo, o nó “Comédia” está conectado aos nós “O Máskara” e “As Branquelas”, indicando que ambos os filmes pertencem ao gênero comédia. “O Máskara” está conectado a “Jim Carrey”, representando que ele é o ator principal desse filme, enquanto “As Branquelas” está conectado a “Marlon Wayans” pelo mesmo motivo.

Além disso, os filmes “O Máskara” e “As Branquelas” estão conectados ao nó “Humor Físico”, representando o estilo de humor presente em ambos. Por fim, o nó “Comédia” está conectado a “Narrativas Familiares”, indicando que esse gênero também está associado a elementos narrativos desse tipo.

Passo 2: Implementação de algoritmos de busca

Usaremos um algoritmo de busca simples em profundidade para encontrar conexões entre filmes e gêneros preferidos pelo usuário.

```
def buscar_recomendacao(grafo, genero_preferido):
    conexoes = nx.node_connected_component(grafo, genero_preferido)

    print(f"Filmes recomendados para quem gosta de {genero_preferido}: {conexoes}")

# Testando a função

buscar_recomendacao(G, "Comédia")
```

Ao testarmos a função, temos o seguinte resultado: Filmes recomendados para quem gosta de Comédia: {'Comédia', 'Narrativas Familiares', 'O Máskara', 'Humor Físico', 'As Branquelas', 'Jim Carrey', 'Marlon Wayans'}.

Agora, usaremos um algoritmo com busca heurística, que tem o objetivo de melhorar o desempenho, ao priorizar os nós mais relevantes, como atores populares.

```
def buscar_com_heuristica(grafo, genero_preferido):
    conexoes = [n for n in grafo.neighbors(genero_preferido)]

    recomendacoes = sorted(conexoes, key=lambda x: grafo.degree(x), reverse=True)

    print(f"Recomendações otimizadas: {recomendacoes}")

# Testando com heurística

buscar_com_heuristica(G, "Comédia")
```

Ao testarmos a função, temos o seguinte resultado: Recomendações otimizadas: ['O Máskara', 'As Branquelas', 'Narrativas Familiares'].

Passo 3: Visualização da rede semântica

```
import matplotlib.pyplot as plt
pos = nx.spring_layout(G)
plt.figure(figsize=(10, 7))
nx.draw_networkx_nodes(G, pos, node_color="lightblue",
node_size=2000)
nx.draw_networkx_edges(G, pos, edge_color="gray")
nx.draw_networkx_labels(G, pos, font_size=12)
plt.title("Rede Semântica de Filmes", fontsize=14)
plt.axis("off")
plt.show()
```

Ao executarmos o código, temos a seguinte árvore semântica:

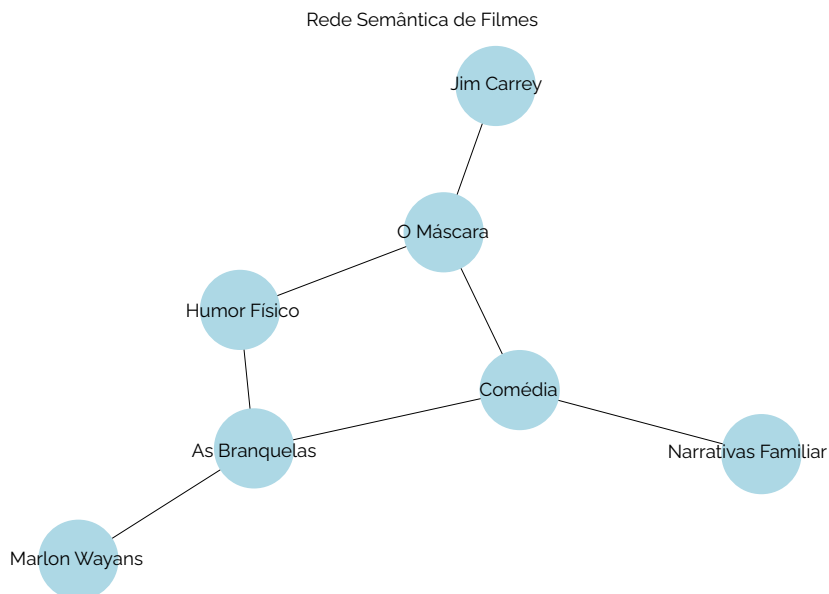


Figura 4 - Rede semântica de filmes / Fonte: o autor.

Descrição da Imagem: é exibida uma rede semântica de filmes que organiza conceitos relacionados ao gênero comédia e elementos associados.

O filme “O Máskara” está conectado ao ator Jim Carrey, destacando o papel principal, e também aos conceitos de “Comédia” e “Humor Físico”, os quais caracterizam o estilo do filme. O conceito de “Humor Físico” se relaciona ao filme “As Branqueiras”, que, por sua vez, está ligado ao ator Marlon Wayans, estabelecendo uma conexão temática entre as produções. Além disso, o nó “Comédia” está associado a “Narrativas Familiares”, representando outro aspecto comum do gênero. Fim da descrição.

Passo 4: Personalização com frames

Agora, vamos realizar a personalização de recomendação no contexto do usuário. A função `recomendacao_contextual` foi projetada para oferecer recomendações adaptadas ao **contexto do usuário**. Aqui, “contexto” pode se referir a situações específicas, como o humor do usuário (*relaxar* ou *aprender*). A função utiliza o grafo para filtrar as conexões mais relevantes do nó “gênero preferido” (por exemplo, “Comédia”) com base no contexto definido.

```
def recomendacao_contextual(grafo, genero_preferido, contexto):

    if contexto == "relaxar":

        recomendacoes = [n for n in grafo.neighbors(genero_preferido) if "Humor Físico" in grafo.neighbors(n)]

    elif contexto == "aprender":

        recomendacoes = [n for n in grafo.neighbors(genero_preferido) if "Narrativas Familiares" in grafo.neighbors(n)]

    else:

        recomendacoes = [n for n in grafo.neighbors(genero_preferido)]

    print(f"Recomendações no contexto '{contexto}': {recomendacoes}")

# Testando a função com diferentes contextos
recomendacao_contextual(G, "Comédia", "relaxar")
recomendacao_contextual(G, "Comédia", "aprender")
```

O resultado da execução da função `recomendacao_contextual` ilustra como é possível personalizar recomendações com base no contexto do usuário utilizando IA simbólica. Essa personalização é alcançada filtrando as conexões no grafo, que representam relações entre entidades, como gêneros de filmes, títulos, estilos de humor e narrativas.

No contexto “relaxar”, a função retorna filmes associados ao gênero “Comédia” que possuem conexões com o nó “Humor Físico”. Nesse caso, os filmes “*O Máskara*” e “*As Branquelas*” são recomendados, porque ambos utilizam o estilo de humor leve e divertido. Esse resultado é consistente com a lógica aplicada, em que a prioridade é sugerir conteúdos descontraídos e alinhados ao estado emocional de relaxamento. Por outro lado, no contexto “aprender”, a função tenta identificar conexões entre “Comédia” e “Narrativas Familiares”. Como os filmes “*O Máskara*” e “*As Branquelas*” não estão diretamente conectados a esse nó, a função não retorna recomendações para esse contexto. Esse comportamento destaca uma limitação na estrutura atual do grafo, que pode ser ajustada para enriquecer as conexões e fornecer resultados mais amplos.



EM FOCO

Gostou do que discutimos até aqui? Tenho mais para conversar com você a respeito deste tema. Vamos lá? **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

A Inteligência Artificial Simbólica conecta teoria e prática de maneira única, abrindo novas oportunidades no mercado de trabalho em setores que exigem soluções **inteligentes, explicáveis e adaptáveis**. Com a capacidade de estruturar conhecimento e realizar inferências lógicas, ela se destaca como uma ferramenta essencial para resolver problemas complexos e impulsionar a inovação. A combinação de fundamentos teóricos, como lógica proposicional, redes semânticas e

teoria dos grafos, com a aplicação prática por meio de ferramentas, como Python, cria um cenário propício para o desenvolvimento de soluções tecnológicas que atendem às demandas de um mercado em constante evolução.

O mercado atual valoriza cada vez mais os profissionais capazes de transformar conceitos abstratos em aplicações práticas que agregam valor às organizações. Por exemplo, o uso de redes semânticas para estruturar informações e frames para contextualizar respostas está revolucionando áreas, como sistemas de recomendação, assistentes virtuais e análise de dados. Empresas que priorizam a personalização de experiências, como plataformas de streaming ou e-commerce, encontram, na IA simbólica, uma base sólida para entender as necessidades dos clientes e oferecer soluções personalizadas e assertivas.

Além disso, a capacidade de otimizar algoritmos simbólicos por meio de técnicas, como podas Alfa-Beta e heurísticas, permite que os profissionais lidem com problemas em larga escala, garantindo eficiência e precisão. Setores, como logística e saúde, já utilizam esses métodos para prever demandas, otimizar rotas ou até mesmo recomendar tratamentos médicos personalizados com base em informações organizadas de forma lógica. A integração de teoria e prática é um diferencial que posiciona os especialistas em IA simbólica como peças-chave para transformar dados em decisões estratégicas.

Os desafios éticos e sociais associados ao uso de IA também abrem novas perspectivas no mercado de trabalho. Profissionais que dominam a IA simbólica têm a oportunidade de desenvolver soluções que sejam transparentes e responsáveis, atendendo às expectativas de regulamentações crescentes e de consumidores cada vez mais conscientes. Isso inclui a criação de sistemas que sejam livres de vieses, explicáveis e alinhados a princípios éticos, características cada vez mais demandadas em setores, como saúde, governança e educação.

A conexão entre teoria e prática, promovida pela IA simbólica, prepara os estudantes para enfrentarem os desafios de um mercado dinâmico e em transformação. Profissionais que compreendem como aplicar redes semânticas, frames, lógica simbólica e programação em Python estarão aptos a desenvolver soluções inovadoras, criar valor estratégico e, ao mesmo tempo, contribuir para um futuro mais ético e sustentável. Essa é a base para transformar o conhecimento acadêmico em impacto profissional, unindo habilidades técnicas e reflexões críticas para moldar o futuro da tecnologia.

AUTOATIVIDADE

1. A lógica proposicional e os grafos são fundamentais para a IA simbólica. Essas ferramentas permitem representar conhecimento e realizar inferências para encontrar soluções de problemas. Outro paradigma em IA, diferentemente do simbólico, envolve técnicas que operam com dados não estruturados e identificam padrões diretamente a partir dos dados. Embora ambos os paradigmas possam ser complementares, eles se diferenciam em sua abordagem de representação e processamento de informações.

Qual das alternativas a seguir apresenta um elemento não associado diretamente à IA simbólica?

- a) Lógica Proposicional.
 - b) Redes Semânticas.
 - c) Algoritmos de Busca.
 - d) Redes Neurais Artificiais.
 - e) Frames.
2. Na Inteligência Artificial Simbólica, a otimização de algoritmos desempenha um papel central, permitindo resolver problemas complexos de maneira mais eficiente. Técnicas de otimização ajudam a reduzir o esforço computacional, ao eliminarem ou priorizarem caminhos, adaptando os métodos de busca às necessidades de cada aplicação.

Considerando as técnicas de otimização na IA simbólica, é correto afirmar:

- I - As podas Alfa-Beta reduzem o número de nós explorados em uma árvore de decisão.
- II - Heurísticas orientam a busca para caminhos mais prováveis de solução.
- III - Algoritmos otimizados sempre garantem a solução exata de um problema.

É correto o que se afirma em:

- a) I e II, apenas.
- b) I, II e III.
- c) II e III, apenas.
- d) III, apenas.
- e) I, apenas.

AUTOATIVIDADE

3. Redes semânticas são estruturas em grafos usadas para representar conceitos e as respectivas relações de maneira lógica. Elas organizam informações de forma que os sistemas possam navegar e inferir conexões entre os dados. Essas redes são amplamente aplicadas em áreas que exigem análise de conhecimento estruturado, como sistemas de recomendação e motores de busca.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

- I - Redes semânticas são amplamente utilizadas em sistemas de recomendação para organizar conhecimento e personalizar sugestões.

PORQUE

- II - Redes semânticas permitem representar e estruturar informações de forma lógica, conectando conceitos relevantes por meio de grafos.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

BAST, H.; BUCHHOLD, B.; HAUSSMANN, E. Semantic Search on Text and Knowledge Bases. **Foundations and Trends in Information Retrieval**, Hanover, v. 10, n. 2-3, p. 119-271, 2016.

CHEN, M.; DÉCARY, M. A cognitive-based semantic approach to deep content analysis in search engines. *In*: INTERNATIONAL CONFERENCE ON SEMANTIC COMPUTING, 12., 2018, [s. l.]. **Proceedings** [...]. [S. l.: s. n.], 2018.

GONZALES, C. *et al.* (ed.). **Graph Structures for Knowledge Representation and Reasoning: Third International Workshop**. [S. l.]: GKR, 2013.

JOSHI, P. **Artificial intelligence with Python**. [S. l.]: Packt Publishing, 2017.

LEWANDOWSKI, D. **Understanding Search Engines**. Cham: Springer International Publishing, 2023.

LUGER, G. F. **Knowing Our World: An Artificial Intelligence Perspective**. [S. l.]: Springer, 2021.

OLIVEIRA, H. C. de; CARVALHO, C. L. de. **Gestão e Representação do Conhecimento**. [S. l.: s. n.], 2008.

PATERSON, D. W. **Introduction to Artificial Intelligence and Expert Systems**. Englewood Cliffs: Prentice-Hall, 1990.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. [S. l.]: Prentice Hall, 2013.

GABARITO

1. Opção D.

Redes Neurais Artificiais pertencem à IA sub-simbólica e são amplamente utilizadas em aprendizado de máquina para identificar padrões em grandes volumes de dados. Diferentemente da IA simbólica, que trabalha com princípios de lógica explícita e representação formal do conhecimento, as redes neurais artificiais operam de maneira menos interpretável e mais orientada por dados. As demais alternativas estão associadas à IA simbólica: a lógica proposicional é utilizada para modelar proposições e raciocínio lógico; as redes semânticas organizam conceitos e relações em grafos, facilitando a inferência e a navegação de informações; os algoritmos de busca são ferramentas cruciais para explorar estruturas simbólicas e encontrar soluções eficientes; e os frames são representações que ajustam informações a contextos específicos, promovendo maior flexibilidade nos sistemas.

2. Opção A.

A afirmativa I é verdadeira, porque as podas Alfa-Beta são técnicas de otimização utilizadas em árvores de decisão para reduzir o número de nós explorados, eliminando caminhos que não influenciam o resultado. Isso torna o processo mais eficiente, sendo amplamente aplicado em contextos, como jogos de tabuleiro e sistemas de decisão. A afirmativa II também é verdadeira, pois heurísticas são estratégias que direcionam a busca em algoritmos para caminhos mais prováveis de solução. Elas utilizam estimativas ou aproximações para priorizar possibilidades mais promissoras, otimizando tempo e recursos computacionais. Por outro lado, a afirmativa III é falsa, já que nem todos os algoritmos otimizados garantem soluções exatas. Técnicas, como as heurísticas, frequentemente priorizam a rapidez e a eficiência em detrimento da exatidão, sendo úteis em cenários onde buscas exaustivas não são viáveis.

3. Opção A.

Ambas as asserções são verdadeiras, e a II justifica corretamente a I. Redes semânticas, ao organizarem informações em grafos e conectar conceitos logicamente, permitem personalizar sugestões, como em sistemas de recomendação. Essas características tornam as redes semânticas essenciais para aplicações que demandam a compreensão de relações complexas e o fornecimento de resultados personalizados.

